SmallFloats.jl

0.3.0

Jeffrey Sarnoff

2026-08-02T19:44Z

Contents

Part I

Home

Chapter 1

SmallFloats.jl

SmallFloats.jl implements the IEEE P3109 draft's small binary formats for Julia, across bitwidths 3 through 16. Its default operations compute the mathematically exact result and project once into the requested format.

```
julia> using SmallFloats

julia> x = Binary8p4se(1.6)
Binary8p4se(1.625 ≡ 0x45)

julia> Add(Binary8p4se, RNE_SN, x, Binary8p4se(1.75))
Binary8p4se(3.5 ≡ 0x4e)
```

The display shows both parts of a stored value: `1.625` is the exact datum and `0x45` is the code point that names it. The addition computes the exact sum, 3.375, then applies the named projection `RNE_SN` once. That relationship—an exact value, a code point, and an explicit projection—is the center of the package and of this manual.

1.1 Choose a route

Start using the package

Begin with Install and Verify, then work through First Session. Read Core Model once before moving into a larger workflow. Together these pages take you from installation to the four distinctions that prevent most mistakes.

Complete an application task

Go directly to a workflow when you already know the core model:

- Choose a Format
- Quantize and Measure a Tensor
- Control Rounding and Overflow
- Run Operations over Arrays
- Use Blocks for Dynamic Range
- Make Stochastic Work Reproducible
- Pack Values for Storage
- Interoperate with Float16 and BFloat16

Each workflow gives one shortest correct procedure, a way to validate the result, and the failure modes most likely to produce a plausible but wrong answer.

Understand the semantics

P3109 in One Chapter introduces the draft standard. Format Anatomy, Values, Code Points, and Conversion, and The Exact-Then-Project Contract form the conceptual spine. The remaining concept pages explain Julia integration, session state, performance, and the difference between defined, stochastic, and approximate results.

Look up a contract

Reference pages are organized by API family. Start with Formats and Value Queries, Projection Specifications, or the Operation Catalog. The Public API Index collects source docstrings after the curated family references.

Inspect evidence or implementation

Examples and Evidence indexes complete applied and verification sessions. Maintainers should begin with Architecture and Invariants, then follow the data path through encoding, projection, the oracle, tables, and block reductions.

1.2 Three promises and one boundary

Defined paths are bit-exact

For a defined operation, the package returns the projection of the mathematically exact result. Table kernels and scalar methods implement the same function; a table is a cache, not an approximate alternate path.

Policy is visible

Rounding and saturation are values in a `ProjSpec`. Convenience operators use the session default, while the spec-named register—`Add(T, ρ, x, y)`, `Exp(T, ρ, x)`, and so on—makes policy explicit and reproducible.

Approximation is declared and measured

Optional approximate kernels live behind a registry. Registration measures their maximum code-point deviation κ and rejects an understated bound.

P3109 is not IEEE 754 in smaller storage

P3109 has one NaN and one zero, defines a different bias rule, and makes signedness and finite/extended domain format parameters. In particular, `Binary16p11se` is not `Float16`, and converting between them is never a reinterpretation. See P3109 in One Chapter before porting IEEE-specific assumptions.

1.3 The rule to remember first

An `Unsigned` constructor argument is a **code point**. Every other `Real` constructor argument is a **value to project**.

```
julia> Binary8p4se(0x02), Binary8p4se(2)
(Binary8p4se(0.001953125 ≡ 0x02), Binary8p4se(2.0 ≡ 0x48))
```

Use `codepoint(x)` and `decode(x)` to make the two interpretations explicit.

1.4 Supported formats

The package implements all 504 legal formats at $K \in 3{:}16$. `using SmallFloats` exports the 120 aliases at $K \le 8$; the other 384 are available through `SmallFloats.Binary…`, `format(K, P, SGN, EXT)`, or `using SmallFloats.Formats`.

1.5 Next

Install and Verify → First Session → Core Model.

Part II

Start Here

Chapter 2

Install and Verify

2.1 Install and Verify

A working install in about five minutes.

Requirements

- **Julia 1.12** or later.
- **`Quadmath`** (libquadmath's `Float128`) is a hard dependency. It is used only inside the oracle and exact-fallback paths — never where it could change a result.

Install

The package is not yet registered, so install by URL:

```
pkg> add https://github.com/JeffreySarnoff/SmallFloats.jl
```

Append `#main` (or a tag/commit) to pin a revision:

```
pkg> add https://github.com/JeffreySarnoff/SmallFloats.jl#main
```

For an editable install, `pkg> dev https://github.com/JeffreySarnoff/SmallFloats.jl` clones into `~/.julia/dev`, or use an existing clone:

```
pkg> dev path/to/SmallFloats.jl
```

Smoke test

Thirty seconds to confirm everything is in place:

```
julia> using SmallFloats

julia> Binary8p4se(1.6) + Binary8p4se(0.25)
Binary8p4se(1.875 ≡ 0x47)

julia> conformance() !== nothing
true
```

The first line proves the projection engine is live (1.6 rounds to 1.625, plus 0.25, lands exactly on 1.875); the second proves the conformance machinery is.

Platform note: disabling Float128

On platforms where libquadmath misbehaves, set an environment variable **before** loading the package:

```
ENV["SmallFloats_Float128"] = "disable"
using SmallFloats
```

This selects the pure-MPFR configuration. Results are **bit-identical** — that equivalence is itself part of the test suite; only oracle and build speed change.

Next

First Session.

Chapter 3

First Session

3.1 First Session

This session establishes the package's working vocabulary and runs one scalar and one array operation. It assumes Install and Verify has succeeded.

```
julia> using SmallFloats
```

Construct a value from a number

```
julia> x = Binary8p4se(1.6)
Binary8p4se(1.625 ≡ 0x45)
```

`Binary8p4se` is an 8-bit, precision-4, signed, extended format. It does not contain the number 1.6. Construction therefore projects 1.6 to the nearest datum, 1.625, and stores its code point, `0x45`.

The display exposes both views. In ordinary application output values may print more compactly; the documentation uses the typed display so that no projection is invisible.

Construct a value from a code point

```
julia> y = Binary8p4se(0x46)
Binary8p4se(1.75 ≡ 0x46)
```

An `Unsigned` argument means **code point**, not numeric value. Compare the two forms:

```
julia> Binary8p4se(0x02), Binary8p4se(2)
(Binary8p4se(0.001953125 ≡ 0x02), Binary8p4se(2.0 ≡ 0x48))
```

Use an unsigned integer when you mean stored identity. Use a signed integer or other `Real` when you mean a value to project. `decode` and `codepoint` recover the two views:

```
julia> decode(x), codepoint(x)
(1.625, 0x45)
```

Compute with an explicit policy

```
julia> Add(Binary8p4se, RNE_SN, x, y)
Binary8p4se(3.5 ≡ 0x4e)
```

Read the call from left to right:

```
operation(result format, projection specification, operands...)
```

The exact sum is 3.375. `RNE_SN` says how that exact result lands in `Binary8p4se`: round to nearest with ties to even, then apply the draft's `SatNone` boundary rules. The result is 3.5 at code point `0x4e`.

The ordinary Julia spelling uses the current default projection:

```
julia> x + y
Binary8p4se(3.5 ≡ 0x4e)
```

Use `+` for exploratory same-format arithmetic. Use `Add(T, ρ, ...)` in code whose policy must be visible and independent of session state.

See projection change an overflow

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);

julia> Multiply(Binary8p4se, RNE_SN, w, two)
Binary8p4se(Inf ≡ 0x7f)

julia> Multiply(Binary8p4se, RNE_SF, w, two)
Binary8p4se(224.0 ≡ 0x7e)
```

Both calls compute the exact product 400. `RNE_SN` produces infinity for this case; `RNE_SF` clamps to the largest finite datum. The projection is part of the operation's meaning.

Apply the same contract to an array

```
julia> A = Binary8p4se.([-1.0, 0.5, 2.0]);

julia> Exp(Binary8p4se, RNE_SN, A)
3-element Vector{Binary8p4se}:
 Binary8p4se(0.375 ≡ 0x34)
 Binary8p4se(1.625 ≡ 0x45)
 Binary8p4se(7.5 ≡ 0x57)
```

The array form uses the same result format, projection, and scalar semantics. Its implementation may use a cached function table, but that does not change the result contract.

Checkpoint

Before running it, predict whether these two expressions have the same intent:

```
Binary8p4se(0x48)
Binary8p4se(72)
```

They do not. The first selects code point `0x48`, whose datum is 2.0. The second projects the numeric value 72 into the format.

What you now know

- a datum is an exact member of a format;
- a code point names that datum;

- an unsigned constructor argument selects a code point;
- computation produces an exact result and projects once;
- explicit operation forms name result format and policy;
- array forms preserve the scalar contract.

Next

Core Model.

Use it

Choose a Format or Quantize and Measure a Tensor.

Look it up

Formats and Value Queries, Projection Specifications, and Operation Catalog.

Chapter 4

Core Model

4.1 Core Model

Five ideas organize the package. Read them as one connected model: a format defines a finite universe, a value names one member of it, computation is exact before projection, projection makes two explicit policy choices, and any departure from the defined path is named and measured.

A format is a finite set of datums

A SmallFloats *format* is not a floating-point type in the IEEE-754 sense of a continuum sampled at machine precision. It is a **finite, enumerated set of exact values** — the *datums* — each with a name, its *code point*. The format `Binary4p2se` (4 bits, precision 2, signed, extended) has 16 code points and they are all of them:

```
julia> decode.(sort(Binary4p2se.(0x00:0x0f)))
16-element Vector{Float64}:
 NaN
 -Inf
  -2.0
  -1.5
  -1.0
  -0.75
  -0.5
  -0.25
   0.0
   0.25
   0.5
   0.75
   1.0
   1.5
   2.0
  Inf
```

That listing *is* the format. There is nothing between the grid points, no hidden state, no approximation in the set itself — the spacing you see (subnormals near zero, binades doubling outward) is the whole structure.

Three consequences fall out immediately:

- **One NaN, no negative zero.** The first entry — NaN sorts below −Inf in the draft's total order (§4.12.1) — is the format's single NaN; the value `0.0` appears exactly once. IEEE-754 spends thousands of code points on NaN payloads and two on zeros; P3109 spends them on datums.
- **The set has edges.** `-Inf`, `Inf`, and `NaN` occupy three of the sixteen names, which is why the largest *finite* value is 2.0 and not something larger. Finite formats (suffix `f` instead of `e`) spend those two infinity code points on more finite datums instead.

- **Larger formats are the same idea with more points.** `Binary8p4se` is 256 names; `Binary16p11se` is 65 536. The 504 formats are just the legal choices of how many names (bitwidth K), how much of the budget goes to significand vs exponent (precision P), and whether the set includes negatives (s/u) and infinities (e/f).

Now you can read: Formats, Aliases & Representations for why the type `Binary8p4se` is a concrete *representation* of the abstract format `Binary{8,4,true,true}` — and why `===` is `false` but `<:` is `true`.

A value is a wrapper around a code point; decode is exact

Construct a value and you hold the code point; ask what it means and you get the datum back, exactly:

```
julia> x = Binary8p4se(1.6)
Binary8p4se(1.625 ≡ 0x45)

julia> codepoint(x)
0x45

julia> decode(x)
1.625
```

The display `1.625 ≡ 0x45` is the idea in one line: the code point `0x45` *means* the datum 1.625, and the `≡` is exact, not approximate. `decode` never rounds — it is a table lookup from name to meaning.

The asymmetry that matters: `decode` is exact, but **construction is a projection**. `Binary8p4se(1.6)` did not store 1.6; it stored the name of the nearest datum, which happens to be 1.625. The value you get back is the value the format can say, not the value you asked for.

This is why the `Unsigned`-means-code-point rule (from the First Session) exists: `codepoint` and the constructor are inverses *by design* —

```
julia> Binary8p4se(codepoint(x)) === x
true
```

— and that round-trip is the format's identity. A value is its code point; everything else is a view.

Now you can read: Values, Code Points, and Conversion for the four ways in and two ways out, and the carriers `decode` returns at wide formats.

Every computation is exact math, then one projection

When you compute, SmallFloats does not simulate low-precision arithmetic step by step. It computes the **mathematically exact result** of the operation on the decoded datums, then applies **one projection** — one rounding plus one saturation decision — to land on a code point:

```
exact result  →  RoundToPrecision  →  Saturate  →  code point
```

`x + y` from the First Session: exact sum 3.375, projected once to 3.5. `200 × 2`: exact product 400 (not representable — projection decides between `Inf`, 224, or `NaN`, and the spec says which).

Two things follow from "one projection, at the end":

- **There is no accumulated rounding error inside an operation.** A fused `FMA(x, y, z)` rounds once, not twice. A `BlockDotProduct` accumulates all 32 lane products exactly and projects once at the end. Intermediate roundings are not "small" — they are *absent*.

- **The default path is bit-exact.** Because the exact math is carried far enough (Float64, Float128, or MPFR enclosures, escalating automatically), the code point you get is *the* defined answer, not a good approximation of it. The package can prove this because the value sets are finite — see the fifth idea.

Now you can read: The Exact-Then-Project Contract for what "exact" means at each carrier width, and the symbolic-sticky trick that lets directed modes land exactly on asymptotes.

Projection = rounding mode × saturation mode

A projection specification is the pair of two independent choices:

	what it decides	examples
rounding mode	which neighbor a between-grid value lands on	nearest-ties-even, toward zero, toward ±∞, stochastic
saturation mode	what an out-of-range result becomes	clamp to finite (SF), propagate infinities (SP), draft rows (SN)

Watch one overflow resolve three ways (`Binary8p4se`, max finite 224):

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);

julia> Multiply(Binary8p4se, RNE_SN, w, two)   # nearest + draft rows → Inf
Binary8p4se(Inf ≡ 0x7f)

julia> Multiply(Binary8p4se, RNE_SF, w, two)   # nearest + clamp → 224
Binary8p4se(224.0 ≡ 0x7e)

julia> Multiply(Binary8p4se, RTZ_SN, w, two)   # toward zero + draft rows → 224
Binary8p4se(224.0 ≡ 0x7e)
```

The first two differ in *saturation* (SN vs SF); the first and third differ in *rounding* (RNE vs RTZ) — and the third row is the subtle one: under `SatNone`, a directed mode that points *away* from the overflow clamps to the extremal finite value, so `RTZ_SN` agrees with `RNE_SF` here for different reasons.

The full deterministic grid is 6 rounding modes × 3 saturation modes = 18 predefined specs (`RNE_SN`, `RNA_SF`, `RTP_SP`, …), plus three stochastic families parameterized by a random-bit budget. `RNE_SN` is the package-wide default — which is exactly why `+`, `exp`, and `T(1.6)` all agree with `Add(T, RNE_SN, …)`, `Exp(T, RNE_SN, …)`, and `Convert(T, RNE_SN, …)`.

The default is a session default

`+` and friends follow `DefaultProjection()`, which is process-global and stays set. Convenience syntax is for exploratory code; anything that must be reproducible names its spec explicitly. The workflow guides teach the safe `with_default_projection` form before showing `DefaultProjection!`.

Now you can read: Rounding and Saturation for the experiments, and Projection Specifications for the complete grid.

Approximation is opt-in, named, and measured

The package's speed comes from the first idea's finiteness: an 8-bit unary operation is a function with 256 inputs, so its *entire* behavior is a 256-byte table, built once through the exact scalar path and then gathered per element (0.27 ns). The exactness is inherited: table and scalar are the same function by construction.

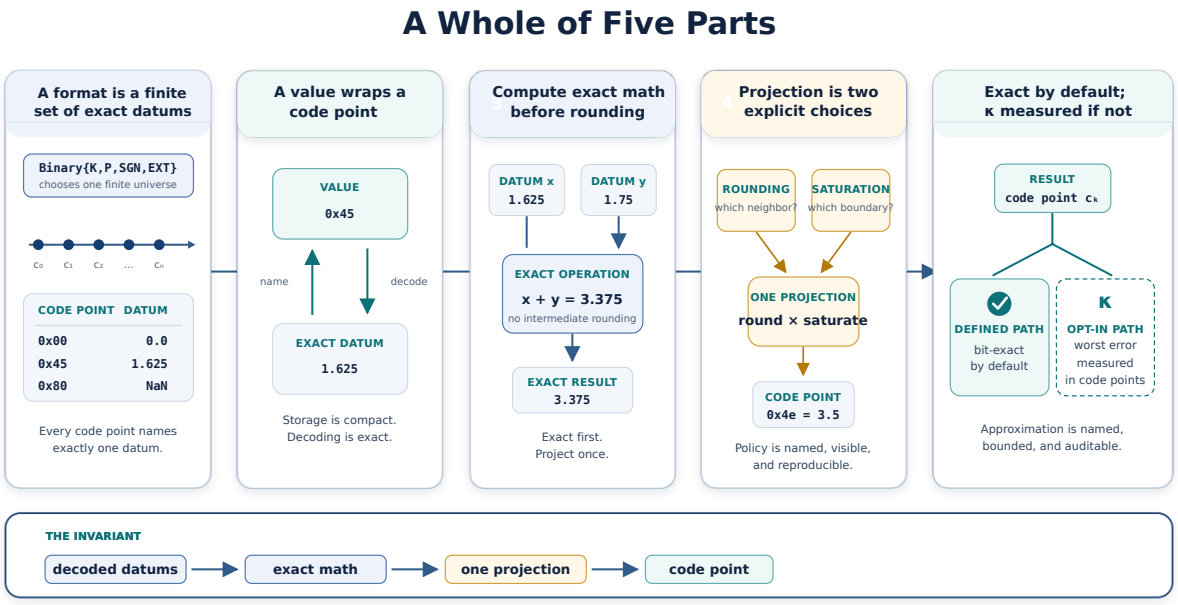

Figure 4.1: A whole of five parts: a finite set of datums, values as code points, exact computation, one controlled projection, and exact or measured results.

The same finiteness makes approximation *honest*. If you register a faster but inexact kernel — say a hardware-style hard-tanh for Tanh — the registry does not take your word for its accuracy. It **measures** the worst-case deviation, exhaustively over every input, as a distance in code points:

```
julia> κ, exhaustive = measure_kappa(
           x -> Clamp(Binary8p4se, RNE_SN, x,
                      Negate(one(Binary8p4se)), one(Binary8p4se)),
           :Tanh, Binary8p4se, (Binary8p4se,), RNE_SN)
(4.0, true)
```

κ = 4 code points, verified over all 256 inputs — that is a measured fact, not a claim. Declare κ smaller than the measurement and registration is *rejected*. Nothing approximate is reachable from the default API; an approximation exists only because you named it, and its κ is part of the conformance declaration the package exports.

This is the package's answer to the problem stated in the Introduction: small formats are easy to implement approximately and hard to implement exactly. SmallFloats implements them exactly, and when you choose otherwise, it makes the choice explicit and quantified.

Now you can read: Register & Measure an Approximation, and Conformance & Approximation Registry.

Visual Synopsis

Read left to right: a value is a name in a finite set (1–2); computing is exact math followed by one projection (3); the projection is your choice of rounding and saturation (4); and the result is exact unless you explicitly register otherwise (5).

Next

Choose a Format applies the model to an application decision. Values, Code Points, and Conversion develops stored identity in more detail.

Look it up

Formats and Value Queries, Projection Specifications, and Conformance and Approximation Registry.

Part III

Workflows

Chapter 5

Choose a Format

5.1 Choose a Format

Pick a `Binary{K,P,SGN,EXT}` format that fits a specific tensor, signal, or quantity.

Ingredients

- A sample of the data (or a plausible generator for it) so you can measure damage before committing.
- `formatname`, `MaxFiniteOf`, `decode` for inspecting candidates.
- `RNE_SF` (or another `ProjSpec`) to project a sample through each candidate.

Recipe

1. Enumerate the format before committing. Small formats are small enough to look at in full. Sorting `Binary4p2se`'s 16 code points shows the subnormal spacing, the binade structure, and the dynamic range in one line:

```
decode.(sort(Binary4p2se.(0x00:0x0f)))
```

```
[-Inf, -2.0, -1.5, -1.0, -0.75, -0.5, -0.25, 0.0,
  0.25, 0.5, 0.75, 1.0, 1.5, 2.0, Inf, NaN]
```

2. At fixed bitwidth, precision trades against range. For a unit-scale Gaussian tensor, RMSE roughly doubles per lost significand bit while `MaxFinite` grows explosively — spend bits on precision unless the data's magnitude actually needs the range:

```
rng = Xoshiro(7); x = randn(rng, 50_000)
for F in (Binary8p5se, Binary8p4se, Binary8p3se, Binary8p2se)
    back = [decode(Convert(F, RNE_SF, xi)) for xi in x]
    println(rpad(formatname(F), 12), " rmse = ", round(sqrt(mean((x .- back).^2)); sigdigits=3),
            "   maxfinite = ", decode(MaxFiniteOf(F)))
end
```

```
Binary8p5se  rmse = 0.0133   maxfinite = 15.0
Binary8p4se  rmse = 0.0266   maxfinite = 224.0
Binary8p3se  rmse = 0.0528   maxfinite = 49152.0
Binary8p2se  rmse = 0.103    maxfinite = 2.147483648e9
```

If your data's dynamic range genuinely exceeds one binade's reach at your precision budget, don't buy that range with a wider element — get it from a block scale instead.

3. Bounded-in-[0,1] quantities want unsigned. A membership degree, a probability, a normalized score — anything that is never negative — is a job for an unsigned finite format, not a signed one spending a code on a sign it never uses. `Binary8p6uf` gives a [0, 15.5] range at 1/32 resolution near 1:

```
F = Binary8p6uf
membership(x, a, b, c, d) = clamp(min((x - a) / (b - a), 1.0, (d - x) / (d - c)), 0.0, 1.0)
warm = F(membership(26.0, 15, 20, 24, 28))   # membership of 26 °C in "warm"
hot  = F(membership(26.0, 24, 30, 100, 101)) # … and in "hot"

(decode(warm), decode(hot),
 decode(Minimum(F, RNE_SN, warm, hot)),      # warm AND hot
 decode(Maximum(F, RNE_SN, warm, hot)),      # warm OR hot
 decode(Multiply(F, RNE_SF, warm, hot)))   # product t-norm
```

```
(0.5, 0.3359375, 0.3359375, 0.5, 0.16796875)
```

4. Pick finite (`f`) when `Inf` would be meaningless. If nothing in your domain should ever legitimately produce or consume an infinity — a scale factor, a bounded score, a probability-like quantity — a finite-domain format spends the code points IEEE-style formats give to ±Inf on two more finite datums instead. Reach for `e` (extended) only when overflow-to-infinity is a real outcome you want to detect downstream.

An unsigned format is not free precision

Going unsigned doubles your positive-side resolution only if the quantity truly never needs a sign. Do not use an unsigned format for anything that can legitimately go negative (gradients, residuals, log-odds) — projecting a negative `Real` into an unsigned format is not a sign flip, it is a domain violation and produces a different failure than you likely intend. Reach for the fuzzy-inference example above only when the quantity is already bounded in `[0, ∞)` by construction.

What can go wrong

Enumeration intuition does not scale past small K

`decode.(sort(F.(0x00:...)))` is a genuine format-choice tool for K small enough to print — beyond roughly `K = 8` the code-point space is too large to eyeball, so switch to the RMSE/MaxFinite sweep instead.

RMSE alone hides overflow

A format can have a low RMSE on the bulk of your data and still overflow its tails to `Inf`. Always check `count(isinf, back) / length(back)` alongside RMSE and max error — see Quantize a Tensor or Model for the full three-number check.

Next steps

Quantize a Tensor or Model, Use Blocks to Track Dynamic Range.

Chapter 6

Quantize and Measure a Tensor

6.1 Quantize and Measure a Tensor

Use this workflow to compare candidate formats on representative data and produce the three measurements that should accompany a quantization choice: root-mean-square error, maximum absolute error, and overflow or clamp rate.

Use a block workflow instead when the data contains groups with very different scales and one bare element format cannot cover them economically.

Prepare representative data

A quantizer can look excellent on synthetic unit-scale data and fail on the tails of a real tensor. Prefer a stable sample from the actual model or signal. The example uses a fixed seed only to make the documentation reproducible.

```
using SmallFloats, Random, Statistics

rng = Xoshiro(42)
w = randn(rng, 50_000) .* 0.25
```

Project explicitly

Name the projection used for evaluation. `RNE_SF` makes the choice clear: round to nearest, ties to even, and clamp outside the finite range.

```
F = Binary8p4se
q = [Convert(F, RNE_SF, x) for x in w]
back = decode.(q)
```

Using `F.(w)` is shorter and follows the session default. The explicit form is better in an evaluation because the report remains meaningful if a default changes elsewhere in the process.

Measure error and boundary use

```
function quantization_report(::Type{F}, x; projection = RNE_SF) where {F}
    q = [Convert(F, projection, xi) for xi in x]
    y = decode.(q)
    finite = isfinite.(y)
    (
        format = formatname(F),
        rmse = sqrt(mean((x[finite] .- y[finite]).^2)),
        max_error = maximum(abs.(x[finite] .- y[finite])),
        nonfinite_rate = count(!isfinite, y) / length(y),
        boundary_rate = count(v -> abs(v) == decode(MaxFiniteOf(F)), y) / length(y),
    )
end

quantization_report(Binary8p4se, w)
```

RMSE describes aggregate damage. Maximum error catches a bad local miss. Nonfinite and boundary rates reveal a range problem that an average can hide. Under `SatFinite`, overflow becomes an extremal finite value rather than `Inf`, so the boundary rate is the relevant companion metric.

Compare candidates under the same policy

```
reports = [quantization_report(F, w) for F in
           (Binary8p5se, Binary8p4se, Binary8p3se, Binary8p2se)]
```

At fixed bitwidth, increasing P improves local resolution and reduces exponent range. Compare formats on the same sample and projection. Do not mix a format decision with a saturation-policy change in the same experiment.

Choose the narrowest format that satisfies an application threshold for all reported metrics. Record the selected format and projection with the model or dataset artifact.

Validate the choice on held-out data

Repeat the report on data that did not participate in the choice. Also inspect the worst few errors:

```
r = quantization_report(Binary8p4se, w)
q = [Convert(Binary8p4se, RNE_SF, x) for x in w]
err = abs.(w .- decode.(q))
worst = partialsortperm(err, 1:10; rev = true)
[(w[i], decode(q[i]), err[i]) for i in worst]
```

A satisfactory average does not excuse systematic failure on rare but important values. If failures cluster by row or channel scale, move to Use Blocks for Dynamic Range.

What can go wrong

MSE can hide the tail

Always report maximum error and boundary/nonfinite rate. A few saturated values can matter greatly while barely moving a global mean.

The constructor hides policy

`F.(x)` is valid, but it uses the current default projection. Use `Convert(F, ρ, x)` in experiments whose policy must be recorded.

Packing is a later decision

First choose and validate the numerical format. Pack the resulting code points only after that choice; packing changes storage, not quantization.

Next

Pack Values for Storage or Use Blocks for Dynamic Range.

Understand it

Format Anatomy and Rounding and Saturation.

Chapter 7

Control Rounding and Overflow

7.1 Control Rounding and Overflow

Use this workflow when a computation must name how between-grid values round and how out-of-range values are handled. The result format and projection spec should be visible in any code whose numerical policy matters.

Select the result format

The result format is the first argument to a spec-named operation:

```
F = Binary8p4se
x, y = F(1.625), F(1.75)
```

Do not rely on cross-format promotion to choose a destination. Convert operands explicitly before combining formats.

Select a deterministic projection

Projection specs combine one rounding mode with one saturation mode. The predefined names use `ROUND_SATURATION`:

```
julia> x, y = Binary8p4se(1.625), Binary8p4se(1.75);

julia> Add(Binary8p4se, RNE_SN, x, y)
Binary8p4se(3.5 ≡ 0x4e)
```

`RNE` is nearest, ties to even. `SN` is `SatNone`, the draft's domain- and direction-dependent boundary policy. Use `SF` when the application requires a finite clamp. Use `SP` when representable infinities should propagate according to `SatPropagate`.

Test a between-grid case

Choose a value that cannot be represented and compare the policies you intend to support:

```
z = 1.6
[(ρ, decode(Convert(Binary8p4se, ρ, z))) for ρ in
 (RNE_SN, RNA_SN, RTP_SN, RTN_SN, RTZ_SN, RTO_SN)]
```

Do not infer a rounding mode from one ordinary value. Include a tie, a negative value, and a value near a binade boundary in a policy test.

Test positive and negative overflow

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);

julia> Multiply(Binary8p4se, RNE_SN, w, two)
Binary8p4se(Inf ≡ 0x7f)

julia> Multiply(Binary8p4se, RNE_SF, w, two)
Binary8p4se(224.0 ≡ 0x7e)
```

Repeat the check for a negative result and, if relevant, an unsigned result format. Saturation behavior depends on domain, signedness, direction, and the rounded classification; “overflow clamps” is not a sufficient policy description.

Keep policy local

Prefer explicit forms in libraries and experiments:

```
Multiply(F, RNE_SF, x, y)
```

For a scoped region of exploratory code, use a `with_default_*` combinator rather than mutating a default and remembering to restore it. Process-global setters are convenient at the REPL and shared by all tasks in the process.

Validate the complete boundary

For small formats, enumerate all code points and test the operation under each supported projection. At minimum assert:

- exact-grid inputs remain exact;
- ties land according to the named rule;
- positive and negative out-of-range results follow the selected saturation;
- NaN and infinity cases follow the operation's P3109 rows;
- scalar and array forms return the same code points.

What can go wrong

SatNone does not mean finite clamp

SatNone means that neither of the two simplifying saturation policies was selected. It applies the draft's full boundary rows and can produce an infinity, an extremal finite value, or NaN depending on the case.

A default is state, not a declaration

A default setter changes process-global state. Name the projection in code that must be reproducible or safe under concurrency.

Understand it

Rounding and Saturation and The Exact-Then-Project Contract.

Look it up

Projection Specifications.

Chapter 8

Run Operations over Arrays

8.1 Run Operations over Arrays

Use this workflow to apply the scalar operation contract to arrays, choose between named calls, `vmap!`, and an `Op` callable, and avoid measuring a cold table build as steady-state performance.

The two registers, again

Every scalar operation used on those pages came from one of two registers, and the same split carries over to arrays:

- **The spec-named register** exposes every draft operation under its draft name with an explicit result format and spec: `Op(fr, ρ, operands...)`. This is `Add`, `Multiply`, `Exp`, and the rest of the operation catalog, where you name ρ yourself.
- **The Base register** makes each format an ordinary Julia number under its *default* spec (`RNE_SN` unless you changed it): `+ - * /`, `exp`, `sqrt`, `min`, `max`, comparisons — same-format operands only, no silent cross-format promotion.

Both registers exist because they serve different code: library code that must be insensitive to the session default names ρ explicitly through the spec-named register; exploratory code reads naturally through the Base register's operators. Both extend to arrays without any new vocabulary.

Array calls: `Add(T, ρ, A, B)`, `Exp(T, ρ, A)`

Every registered operation has array methods with exactly the same argument shape as the scalar form, operands replaced by arrays:

```
julia> A = Binary8p4se.(randn(Xoshiro(7), 4) .* 2)
4-element Vector{Binary8p4se}:
 Binary8p4se(-0.875 ≡ 0xbe)
 Binary8p4se(4.0 ≡ 0x50)
 Binary8p4se(-3.25 ≡ 0xcd)
 Binary8p4se(2.25 ≡ 0x49)

julia> Exp(Binary8p4se, RNE_SN, A)
4-element Vector{Binary8p4se}:
 Binary8p4se(0.40625 ≡ 0x35)
 Binary8p4se(56.0 ≡ 0x6e)
 Binary8p4se(0.0390625 ≡ 0x1a)
 Binary8p4se(9.0 ≡ 0x59)
```

`Add(T, ρ, A, B)` reads the same way, elementwise, for a two-array call.

Understand cold and warm calls

The first deterministic array call for a new `(operation, formats, projection)` specialization may build a result table. Later calls reuse it. Warm the exact specialization before measuring steady-state performance, and use `table_bytes()` or `empty_tables!()` only when you need to inspect or reset the cache.

This does not create another numerical path. Every table entry is produced by the scalar operation, so scalar and array forms have the same code-point contract. Stochastic operations compute per element because each projection consumes its own draw.

The cache design and ternary tier policy belong to Function Tables and Array Kernels; they are not required to use the array API correctly.

`vmap` and `vmap!`

`vmap`/`vmap!` are the generic, table-aware entry points behind the named array calls above. `vmap!` is the in-place form and is what you want when you already have a destination array to fill rather than a fresh one to allocate:

```
C = vmap(:Add, T, ρ, A, B)
vmap!(C, Val(:Add), T, ρ, A, B)
```

Reach for `vmap!` in a loop that reuses a buffer; reach for the named form (`Add(T, ρ, A, B)`) everywhere else — they run the same kernel.

The `Op` callable, with `map` and broadcast

`Op(T, ρ, A)` above already reads like `map`. When you want Julia's own array verbs — `map`, broadcast — rather than the named array call, bind the operation, result format, and projection into an `Op` value and use it as a function:

```
julia> A = Binary8p4se.([1.5, 0.25]); B = Binary8p4se.([0.25, 1.5]);

julia> map(Op(:Add, Binary8p4se, RNE_SN), A, B)
2-element Vector{Binary8p4se}:
 Binary8p4se(1.75 ≡ 0x46)
 Binary8p4se(1.75 ≡ 0x46)

julia> Op(:Exp, Binary8p4se, RNE_SN).(A)
2-element Vector{Binary8p4se}:
 Binary8p4se(4.5 ≡ 0x51)
 Binary8p4se(1.25 ≡ 0x42)
```

`Op` is a singleton type — the operation, format, and projection all live in its type parameters — so a call specializes exactly as `Add(Binary8p4se, RNE_SN, x, y)` does and allocates nothing beyond the array `map` or broadcast itself builds.

Sorting

Sorting is special-cased for every `Binary` format: values compare through integer order keys, and vectors of `Binary` sort with an O(n) **counting sort** installed as the default algorithm. `sort(A)` follows P3109's total order, with the single NaN first (last under `rev=true`):

```
decode.(sort(Binary4p2se.(0x00:0x0f)))      # broadcast the code-point constructor
```

```
[NaN, -Inf, -2.0, -1.5, -1.0, -0.75, -0.5, -0.25,
  0.0, 0.25, 0.5, 0.75, 1.0, 1.5, 2.0, Inf]
```

`TotalOrder(x, y)` exposes the draft's total order directly if you need the comparison itself rather than a sorted array. This NaN placement differs from `Float64` sorting and is intentional.

Try it

Given `A = Binary8p4se.([1.5, 0.25])` and `B = Binary8p4se.([0.25, 1.5])` from above, write the `Op`-callable version of `Add(Binary8p4se, RNE_SN, A, B)` using broadcast rather than `map`.

<details> <summary>Answer</summary>

```
Op(:Add, Binary8p4se, RNE_SN).(A, B)
```

This broadcasts the same bound callable used with `map` above and returns the same two-element vector, `[1.75, 1.75]` in `Binary8p4se`.

</details>

Next steps

Use Blocks for Dynamic Range, Operation Catalog, Arrays, Blocks & Packed Storage.

Chapter 9

Use Blocks for Dynamic Range

9.1 Use Blocks for Dynamic Range

Use this workflow when groups of values need more dynamic range than a narrow element format can provide. A `Block` pairs one shared scale with a group of elements, following the MX-style `scale × element` model.

Scale × element

Small formats have small ranges. A `Block{B,FS,FE}` fights that by sharing one scale, in format `FS`, across `B` elements in format `FE`; the value each element *represents* is `scale × element`, not the element alone. Construct one directly from a scale and its elements:

```
julia> b = Block(Binary8p1uf(4.0), Binary8p4se(1.5), Binary8p4se(-0.75),
                 Binary8p4se(2.0), Binary8p4se(0.5))
Block{4, Binary8p1uf, Binary8p4se}(Binary8p1uf(4.0 ≡ 0x82), (…))
```

Read the type parameters: block size `4`, scale format `Binary8p1uf` (unsigned — scales are non-negative — with the huge range that precision `1` buys), element format `Binary8p4se`. The represented values are `4.0 × 1.5 = 6.0`, `4.0 × -0.75 = -3.0`, and so on.

`ConvertToBlockMaxAbsFinite`

Building a block by hand from a scale you already chose is one path; the more common one is to quantize a tuple of values and let the draft's algorithm pick the scale for you, from the maximum finite `|element|`, then project each element against it:

```
julia> ConvertToBlockMaxAbsFinite(Binary8p1uf, Binary8p4se, RNE_SN, RNE_SN,
           (Binary8p4se(100.0), Binary8p4se(-12.0), Binary8p4se(0.5), Binary8p4se(3.0)))
Block{4, Binary8p1uf, Binary8p4se}(Binary8p1uf(64.0 ≡ 0x86), (…))
```

Read the arguments in order: scale format, element format, a projection spec for the scale, a projection spec for the elements, then the values. The algorithm picked scale 64 from the largest-magnitude element (100) and projected each of the four elements against that scale.

`ConvertFromBlock`

Read a block back out as plain values in a format of your choosing: each `scale × element` is decoded exactly and projected once.

```
julia> ConvertFromBlock(Binary8p3se, RNE_SN, b)   # decode scale×elem, project
(Binary8p3se(6.0 ≡ 0x4a), Binary8p3se(-3.0 ≡ 0xc6), Binary8p3se(8.0 ≡ 0x4c), Binary8p3se(2.0 ≡ 0x44))
```

The stage-wide pitfall

`ConvertToBlockMaxAbsFinite` takes already-`Binary` elements as input. If you stage your raw `Float64` data through the narrow *element* format before quantizing into a block, `SatFinite` clamps the large values **before** the block scale ever gets a chance to absorb them — and the block buys you nothing. This is the single most consequential mistake with this API, so watch it happen on real data before you write code that makes it.

Each row of `W` here lives at a different scale, spanning roughly 2^16 across the matrix — exactly the shape MX-style blocking exists for:

```
using SmallFloats, Random, Statistics

rng = Xoshiro(3)
W = randn(rng, 64, 64) .* 2 .* (2.0 .^ (collect(0:63) ./ 4))   # per-row scales

function mx_rows(W, ::Type{FST}, ::Type{FS}, ::Type{FE}, ::Val{B}) where {FST,FS,FE,B}
    n, m = size(W)
    [begin
        # stage through a WIDE-RANGE format so nothing clamps before scaling
        seg = ntuple(k -> Convert(FST, RNE_SF, W[i, (j-1)*B + k]), Val(B))
        ConvertToBlockMaxAbsFinite(FS, FE, RNE_SN, RNE_SN, seg)
    end for i in 1:n, j in 1:m ÷ B]
end

blocks = mx_rows(W, Binary8p2se, Binary8p1uf, Binary8p4se, Val(32))
recon = [let b = blocks[i, (j-1) ÷ 32 + 1]
             decode(b.s) * decode(b.x[(j-1) % 32 + 1])
         end for i in 1:64, j in 1:64]
plain = [decode(Convert(Binary8p4se, RNE_SF, w)) for w in W]

relerr(A) = sqrt(mean(((W .- A) ./ max.(abs.(W), 1e-9)).^2))
relerr(plain), relerr(recon)
```

```
(0.616, 0.109)      # plain 8p4se clamps the large rows; MX tracks them
```

The plain, unblocked `Binary8p4se` conversion clamps every row whose scale exceeds what `Binary8p4se` alone can hold — relative error 0.616. Staging through the wide-range `Binary8p2se` first, then letting the per-row block scale absorb the range, brings that down to 0.109 — the residual is now just the staging format's own precision.

Stage wide, then block-quantize

`ConvertToBlockMaxAbsFinite` takes already-`Binary` elements. If you stage your `Float64` data through the *element* format first, `SatFinite` clamps the large values **before** the block scale can absorb them, and blocks buy you nothing — we measured exactly that (0.617 vs 0.616) with `Binary8p4se` staging. Stage through a wide-range format (`Binary8p2se` above); the residual 0.109 here is the staging format's own precision, which bounds what any downstream scheme can keep.

At `B = 32` with an 8-bit scale the storage cost is 8.25 bits/value.

`BlockDotProduct`: one rounding, at the end

`BlockDotProduct` computes every lane product and the accumulation *exactly*, in a wide carrier, and projects only once, at the very end — there is no hidden intermediate rounding lane by lane:

```
rng = Xoshiro(9)
a64, b64 = randn(rng, 32), randn(rng, 32)
qb(v) = ConvertToBlockMaxAbsFinite(Binary8p1uf, Binary8p4se, RNE_SN, RNE_SN,
            ntuple(i -> Convert(Binary8p4se, RNE_SF, v[i]), Val(32)))
dq = BlockDotProduct(Binary8p4se, RNE_SN, qb(a64), qb(b64))
(a64'b64, decode(dq))
```

```
(5.0634, 4.5)       # difference is input quantization only, never accumulation error
```

The gap between `5.0634` and `4.5` here is entirely the up-front quantization of a64 and b64 into 4-bit-precision elements — accumulation itself contributes nothing, because the lane products and their sum are carried exactly until the single final projection. Reductions besides the dot product follow the same shape: `BlockReduceAdd`, `BlockReduceMultiply`.

`BlockVector`

For many blocks of the same shape, `BlockVector` stores them in a structure-of-arrays layout rather than as an array of individually boxed `Block` values — a memory-bound storage form for blocks, the same way packed code-point storage exists for bare values.

Try it

You have raw `Float64` data with widely varying per-row magnitude, and you want to quantize it into `Block{32,Binary8p1uf,Binary8p4se}`. Name the staging mistake to avoid, and the one-line fix.

<details> <summary>Answer</summary>

The mistake is converting straight to `Binary8p4se`, the narrow element format.

Passing those narrowed elements to `ConvertToBlockMaxAbsFinite` lets `SatFinite` clamp large rows before the block scale can absorb them. Stage through a wide-range format first, such as `Convert(Binary8p2se, RNE_SF, w)`, and pass those staged elements onward—the `seg = ntuple(k -> Convert(FST, RNE_SF, ...), ...)` line in the demo above.

</details>

Next steps

Stochastic Rounding, Arrays, Blocks & Packed Storage.

Chapter 10

Make Stochastic Work Reproducible

10.1 Make Stochastic Work Reproducible

Make stochastic-rounding code reproducible — across runs, and inside test suites — by fixing the entropy source.

Ingredients

- An explicit `AbstractRNG` (e.g. `Xoshiro(seed)`) passed as `rng`.
- An explicit R (the raw stochastic draw) for tests that must be bit-exact.
- `DefaultRNG!` if you want the implicit (no-rng-argument) forms to be reproducible too.
- The `projection` keyword on scalar `rand`/`randn` forms.

Recipe

1. Prefer an explicit rng stream over the task-local default. Passing any `AbstractRNG` first gives a stream independent of global state:

```
julia> rand(Xoshiro(1), Binary8p4se)
Binary8p4se(0.0703125 ≡ 0x21)

julia> rand(Xoshiro(8), Binary8p4se, 4)
4-element Vector{Binary8p4se}:
 Binary8p4se(0.40625 ≡ 0x35)
 Binary8p4se(0.021484375 ≡ 0x13)
 Binary8p4se(0.75 ≡ 0x3c)
 Binary8p4se(0.28125 ≡ 0x31)
```

If you rely on the implicit task-local generator instead, `Random.seed!` controls it exactly as for any other type:

```
julia> Random.seed!(1234); rand(Binary8p4se)
Binary8p4se(0.3125 ≡ 0x32)

julia> Random.seed!(1234); rand(Binary8p4se)      # same seed, same draw
Binary8p4se(0.3125 ≡ 0x32)
```

2. A stochastic projection draws from the same rng as the value draw. One underlying `Float64` draw, landed on the grid four different ways — the seed fixes the draw, the projection decides the landing:

```
julia> rand(Xoshiro(2), Float64)                # the underlying uniform draw
0.0022505868897625403

julia> rand(Xoshiro(2), Binary8p4se)            # default: floor (stays in [0,1))
Binary8p4se(0.001953125 ≡ 0x02)

julia> rand(Xoshiro(2), Binary8p4se; projection = RTP_SN)   # ceiling
Binary8p4se(0.0029296875 ≡ 0x03)

julia> rand(Xoshiro(2), Binary8p4se; projection = RNE_SN)   # nearest
Binary8p4se(0.001953125 ≡ 0x02)

julia> rand(Xoshiro(2), Binary8p4se; projection = RSA_SN(8))  # stochastic
Binary8p4se(0.001953125 ≡ 0x02)
```

Because the stochastic projection draws its random bits from the *same* rng as the uniform draw, seeded streams stay reproducible end to end — you do not need a second seed for the rounding decision.

3. For a test, fix the raw draw `R` instead of an rng. This makes a single projection exactly reproducible without depending on an RNG's internal algorithm at all — the right tool when you want to test both edges of a stochastic rounding decision deterministically:

```
julia> σ = RSA_SN();                 # ≡ ProjSpec(StochasticA{8}(), SatNone())

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125); rng = Xoshiro(1))
Binary8p4se(2.0 ≡ 0x48)

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125); R = 0)
Binary8p4se(2.0 ≡ 0x48)      # smallest draw: rounds down

julia> Add(Binary8p4se, σ, Binary8p4se(2.0), Binary8p4se(0.03125); R = 255)
Binary8p4se(2.25 ≡ 0x49)     # largest draw: rounds up
```

4. Set `DefaultRNG!` if you want implicit calls to be reproducible too. `DefaultRNG` is one of the six session defaults (initial value: the `Xoshiro` type); set it once at the top of a script or test file if you want every call that doesn't name an rng to still be controlled by your seed.

What can go wrong

Array and `¡` forms always use the defaults

`rand(T, dims)`, `randn!(A)`, and friends always use the default projection (floor for `rand`, nearest + SatFinite for `randn`) — there is no `projection` keyword on the array forms. For arrays under another projection, draw scalars explicitly: `[rand(rng, T; projection = ρ) for _ in 1:n]`.

`R` must lie in `0:2^N-1`

For an N-bit stochastic mode, an out-of-range explicit `R` is a contract violation, not a silently wrapped value — pick `R` from the mode's actual bit budget (`RSA_SN(8)` takes `R ∈ 0:255`).

Nearest/ceiling can return exactly 1.0 from ‘rand‘

The default floor projection is the contract-keeper for `rand`: it guarantees results stay in `[0, 1)`. Opting into `RTP_SN` or `RNE_SN` can return exactly `1.0` (mass near the top rounds up) — only opt out of the default if your code can tolerate that.

Next steps

Choose a Format, Quantize a Tensor or Model.

Chapter 11

Pack Values for Storage

11.1 Pack Values for Storage

Store a vector of small-format values at its true bit width instead of one byte (or two) per element.

Ingredients

- `PackedVector(v)` built from an existing `Vector{F}`.
- Standard indexing, `collect`, iteration — `PackedVector` is a full `AbstractVector{F}`.
- `vmap` for computing directly on packed data (it unpacks tiles internally).
- The sizing rule: `bitwidth(F)` bits per element, rounded up to whole storage words.

Recipe

1. Build a packed vector from ordinary values. `PackedVector{F}` stores each code point in exactly `bitwidth(F)` bits:

```
julia> v = Binary5p2se.(rand(Xoshiro(3), 6) .* 4);

julia> pv = PackedVector(v); sizeof(pv.data)
8                    # 6 × 5 bits = 30 bits → one 64-bit word

julia> pv[3] == v[3]
true
```

Six values at 5 bits each is 30 bits, which rounds up to one 64-bit backing word — `sizeof(pv.data)` reports 8 bytes.

2. Index, collect, and iterate like any other vector. `PackedVector` is a full `AbstractVector{F}`:

```
pv[2]
pv[2] = Binary5p2se(0.75)
collect(pv)
```

3. Compute with `vmap` directly on the packed vector. `vmap` accepts a `PackedVector` argument directly, unpacking cache-friendly tiles internally — you do not need to `collect` first:

```
out = vmap(:Exp, Binary5p2se, ρ, pv)
```

4. Size larger vectors the same way. The packing ratio is exactly `bitwidth(F) / 8` against a plain byte-per-element vector (for K ≤ 8 formats). At a million elements of a 5-bit format:

```
n = 1_000_000
model = [rawvalue(Binary5p2se, UInt8(rand(0:31))) for _ in 1:n]
pv = PackedVector(model)
(sizeof(model), sizeof(pv.data))
```

```
(1000000, 625000)   # 8 bits → 5 bits per value; indexing and vmap work directly on pv
```

1,000,000 bytes of plain storage becomes 625,000 bytes packed — `n × K / 8` bytes, in general, rounded to whole words.

What can go wrong

There is deliberately no in-place packed arithmetic

The rule is *store packed, compute unpacked*. Don't reach for an in-place packed +=-style operation — it doesn't exist. Compute with `vmap` (which unpacks internally and returns a fresh result) or unpack with `collect` first, compute, then re-pack with `PackedVector` if you want to keep the result compressed at rest.

Packing helps storage bandwidth, not per-element compute speed

Reach for `PackedVector` when memory footprint or bandwidth is the bottleneck (e.g. a large quantized model kept resident). If you are only computing, not storing, an ordinary `Vector{F}` (one byte per element at K ≤ 8) is simpler and does not pay any unpacking cost.

Next steps

Quantize a Tensor or Model, Choose a Format.

Chapter 12

Interoperate with Float16 and BFloat16

12.1 Interoperate with Float16 and BFloat16

Move data between SmallFloats formats and IEEE `Float16`/`BFloat16` without silently changing its value by a factor of two.

Ingredients

- `Convert(T, ρ, x)` — the only safe path between formats with different exponent bias.
- `datumsexact` to check whether the two datum sets permit an exact round trip.
- Never `reinterpret` across formats with different bias — see the warning below.

Recipe

1. Understand the mismatch before you touch either format. The K ≤ 16 grid contains two formats that look like `Float16` and `BFloat16` but are not — the difference is the kind that produces correct-looking numbers rather than errors:

	`Binary16p11se`	`Float16`	`Binary16p8se`	`BFloat16`
precision P	11	11	8	8
exponent bias	**16**	**15**	**128**	**127**
largest finite	**65472**	**65504**	**3.3762e38**	**3.3895e38**
NaN encodings	**1**	**2046**	**1**	**2046**
negative zero	**no**	yes	**no**	yes
domain is a parameter	**yes** (`se`/`sf`)	no	**yes**	no

P3109 defines the bias so the exponent range is symmetric about the format's midpoint; IEEE-754 defines it as `2^(E-1) − 1`. The two differ by one, so **every datum in the format is a factor of two away from its IEEE counterpart** — `Binary16p11se` and `Float16` share a significand layout and disagree on the value of every code point.

2. Always convert — never reinterpret. `Convert` is a real conversion: `Float16(1.5)` becomes `Binary16p11se(1.5)`, unchanged in value:

```
julia> using SmallFloats.Formats

julia> Convert(Binary16p11se, RNE_SF, Float16(1.5))    # a conversion: 1.5 ↦ 1.5
Binary16p11se(1.5 ≡ 0x4200)

julia> reinterpret(Binary16p11se, Float16(1.5))        # the SAME BITS: 1.5 ↦ 0.75
Binary16p11se(0.75 ≡ 0x3e00)
```

`Float16(1.5)` has the bit pattern `0x3e00`; read as a `Binary16p11se`, those same bits denote **0.75**, because the exponent bias differs by one. Nothing errors, nothing warns, and the answer is off by a factor of two.

3. Check `datumsexact` when you need to know if the round trip is lossless. The package converts to and from both `Float16` and `BFloat16` exactly, wherever the datum sets permit — `datumsexact` tells you where that holds so you don't have to assume.

4. Remember the other differences are draft decisions, not bugs. A single NaN encoding means there is no payload to propagate and no quiet/signalling distinction — the 2045 code points IEEE spends on NaN payloads are ordinary datums here. No negative zero means `-0.0` and `0.0` are the same code point. The domain parameter (`e` for extended with ±Inf, `f` for finite) has no IEEE analogue: `Binary16p11sf` spends the two infinity code points on finite values instead. None of these need "fixing" before converting — `Convert` already accounts for all of them.

What can go wrong

'reinterpret' silently returns the wrong value, off by 2×

`reinterpret(Binary16p11se, Float16(1.5))` gives `0.75`, not `1.5` — the same bit pattern, read under a different exponent bias. This is the single most dangerous operation in cross-format interop: it never errors, so a pipeline that accidentally reinterprets instead of converting produces plausible-looking numbers that are systematically wrong by a factor of two. Always use `Convert`, never `reinterpret`, when moving between a SmallFloats format and `Float16`/`BFloat16`.

If you want IEEE semantics, use Float16/BFloat16 directly

`Binary16p11se` and `Binary16p8se` are not drop-in replacements for `Float16`/`BFloat16` — they exist for the P3109 draft's own reasons (symmetric exponent range, single NaN, no negative zero, parametric domain). If your downstream consumer specifically needs IEEE bit patterns, keep the data in `Float16`/`BFloat16` and convert only at the SmallFloats boundary.

Next steps

Choose a Format, Read & Export the Conformance Declaration.

Chapter 13

Produce a Conformance Declaration

13.1 Read and Export Conformance

Produce a machine-readable statement of exactly which formats, operations, and approximations a session used, and attach it to an experiment's artifacts.

Ingredients

- `conformance()` — the live declaration as a Julia value.
- `conformance_report()` — a printed, human-readable form.
- `conformance_dict()` — a plain nested `Dict{String,Any}` for JSON/TOML serialization.
- `draft_revision()` — the P3109 draft revision the package implements.
- Any JSON/TOML writer your project already depends on.

Recipe

1. Inspect the live declaration. `conformance()` returns the formats, operations, mode vocabulary, the table specializations actually instantiated this session, and every registered approximation. `conformance_report()` prints the same information for a human to read at the REPL.

2. Export it as a plain `Dict` for serialization.

```
d = conformance_dict()          # plain nested Dict{String,Any}
(d["package"], length(d["formats"]), length(d["operations"]),
 [s["op"] for s in d["cached_specializations"]])
```

Serialize `d` with any JSON/TOML writer to attach a machine-readable conformance statement to experiment artifacts — for example, alongside a saved model checkpoint or a benchmark result, so a later reader can confirm exactly which formats and approximations produced it without re-running the session.

3. Record the draft revision alongside it. `draft_revision()` reports which revision of the P3109 draft this package implements — include it next to the conformance dict so an artifact remains interpretable even if the draft itself changes in a later package version.

4. Confirm registered approximations show up. Any approximation registered with `register_approx!` appears in the conformance declaration automatically — `conformance_report()` lists it alongside the exact operation catalog, so the declaration always reflects the true provenance of a session's results, exact and approximate alike.

What can go wrong

The declaration is live, not static

`conformance()` and `conformance_dict()` reflect the *current* session — including which table specializations have actually been built so far. Call them at the point you want to snapshot (typically right before saving results), not at the top of a script, or the declaration may under-report what the run actually exercised.

A conformance dict does not travel with unregistered approximations

If you registered an approximation in one session and load results from it in a fresh session, `conformance_dict()` in the new session will not know about that approximation unless you re-register it. Save the exported dict itself with the artifact — do not rely on regenerating it later from a differently configured session.

JSON/TOML serialization is your responsibility, not the package's

`conformance_dict()` returns a plain nested `Dict`; the package does not ship a JSON or TOML writer. Use whatever serialization library your project already depends on — the dict is deliberately unopinionated about the output format.

Next steps

Register & Measure an Approximation (κ), Choose a Format.

Chapter 14

Register and Measure an Approximation

14.1 Register an Approximation

Register a faster, inexact kernel in place of the bit-exact default, with an honest, measured error bound.

Ingredients

- `measure_kappa(fn, opname, resultformat, operandformats, ρ)` to measure the worst-case code-point deviation exhaustively.
- `register_approx!(name, opname, resultformat, operandformats, ρ, fn; κ)` to register the implementation under a declared bound.
- `approx`, `list_approx`, `kappa`, `kappa_measured` to retrieve and inspect registered approximations.
- `unregister_approx!` to remove one.

Recipe

1. Measure κ before declaring anything. `measure_kappa` runs your approximate function against the defined operation over every input and returns the worst code-point distance, exhaustively verified:

```
step2(x) = (r = Exp(Binary8p4se, RNE_SN, x);
            isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)

measure_kappa(step2, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN)
```

```
(2.0, true)                # κ = 2 code points, verified over all 256 inputs
```

2. Register with the measured (or a larger) κ. `register_approx!` takes the same signature plus the implementation and a declared κ:

```
register_approx!(:cheater, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN, step2; κ=1)
```

```
ERROR: ArgumentError: declared κ = 1.0 understates measured κ = 2.0 — registration rejected
```

Declaring κ = 1 when the measured deviation is 2 is rejected outright — the registry measures your honesty at registration time; you cannot understate the bound.

3. Register with a truthful κ and retrieve it. Using κ=1 failed above; declaring the true value (or higher) succeeds. The same shape applied to a hardware-style approximation (`hardtanh` standing in for Tanh) shows the full round trip:

```
one4 = Binary8p4se(1.0)
hardtanh(x) = Clamp(Binary8p4se, RNE_SN, x, Negate(one4), one4)

κ, exhaustive = measure_kappa(hardtanh, :Tanh, Binary8p4se, (Binary8p4se,), RNE_SN)
(κ, exhaustive)
```

```
(4.0, true)                # worst deviation: 4 code points, verified on all 256 inputs
```

```
impl = register_approx!(:hardtanh_act, :Tanh, Binary8p4se, (Binary8p4se,),
                        RNE_SN, hardtanh; κ=4)
(kappa(:hardtanh_act), kappa_measured(impl), :hardtanh_act in list_approx())
```

```
(4.0, 4.0, true)           # declared = measured; conformance_report() now lists it
```

4. Remove a registration when you're done with it.

```
unregister_approx!(:hardtanh_act)
```

What can go wrong

Understating κ is not silently clamped — it throws

You cannot register `:cheater` with κ=1 "just to see" and get a warning; `register_approx!` throws `ArgumentError` and refuses the registration entirely. There is no partial-registration state to clean up.

NaN-mismatching implementations need κ = NaN, not a number

If your approximate kernel disagrees with the defined operation on whether a NaN is produced, you must acknowledge this explicitly by declaring `κ = NaN` — a numeric κ asserts NaN behavior matches.

Approximate implementations never enter the default API

Registering `:my_fast_exp` or `:hardtanh_act` does not change what Exp or Tanh return by default — the bit-exact path is always the one Exp, +, and friends use. Retrieve an approximation explicitly with `approx(:name)` and call it yourself; `list_approx()` enumerates what's registered, and `conformance_report()` lists them alongside the exact catalog.

Next steps

Read & Export the Conformance Declaration.

Part IV

Concepts

Chapter 15

P3109 in One Chapter

15.1 P3109 in One Chapter

The IEEE P3109 draft — *Arithmetic Formats for Machine Learning* — as this package sees it. Fifteen minutes here buys you the vocabulary every other page assumes. Claims are tagged with the draft clause they come from; the full transliteration is in the repository at `docs/other/IEEE_D1.md` for when you want the normative text itself.

The one law everything hangs on

Every numeric operation in the draft is three steps (§4.3.2):

```
decode the operands  →  compute exactly over the extended reals  →  project once
```

There is no arithmetic "in the format." Operands are decoded to exact values, the operation is performed exactly, and a single projection — rounding plus saturation — writes the result back to a code point. The comparisons and predicates stop after decoding (they return Booleans); everything else follows the law. SmallFloats.jl is built as a direct implementation of this sentence.

Formats: four parameters, 504 instances

A format is `Binary{K, P, Σ, Δ}` (§3.1):

parameter	meaning	range
K	total bitwidth	3 – 16
P	significand precision, implicit bit included	signed: `0 < P < K`; unsigned: `0 < P ≤ K`
Σ	signedness: signed or unsigned	
Δ	domain: **extended** (has ±Inf) or **finite**	

The short name reads off the parameters: `Binary8p4se` = K 8, p 4, **s**igned, **e**xtended. The draft's exponent bias is *derived*, not chosen (§3.1, §4.14):

```
B = 2^(K−P−1)   (signed)        B = 2^(K−P)   (unsigned)
```

This symmetric-about-the-midpoint bias is the single most consequential formula in the draft — see "Differences from IEEE 754" below.

Datums and code points

A format's *datum set* is a finite subset of the closed extended reals ($\mathbb{R}$ plus ±Inf and NaN), in bijection with the code points `0 … 2^K − 1` (§3, Annex B). Every finite datum has a canonical form

```
(−1)^sign · S · 2^(1−P) · 2^E        E > −B
```

with significand S classifying it as normal (`2^(P−1) ≤ S < 2^P`) or subnormal (`0 < S < 2^(P−1)`).

The specials are where the draft parts company with IEEE-754 (Annex B, Table 3):

- **Exactly one zero.** There is no −0; a signed format has an odd number of finite datums.
- **Exactly one NaN**, sitting at the code point IEEE-754 gives to −0 in signed formats. No payloads, no quiet/signalling distinction — the thousands of code points IEEE spends on NaN encodings are datums here.
- **±Inf only in extended formats**, adjacent to the finite extremes. A finite format spends those code points on more finite values.

Projection: rounding × saturation

A projection specification ρ is the pair *(rounding mode, saturation mode)* (§4.2, §4.7.3–4.7.5).

Six deterministic rounding modes — `NearestTiesToEven`, `NearestTiesToAway`, `TowardPositive`, `TowardNegative`, `TowardZero`, `ToOdd` — plus three stochastic families `StochasticA/B/C`, which consume `N` random bits as an unsigned draw `R ∈ 0 … 2^N − 1` (§4.7.4; the draft deliberately leaves the bit source unspecified).

Three saturation modes decide what out-of-range results become (§4.7.5):

mode	finite overflow	±Inf input
`SatFinite`	clamp to the finite extremes	clamp
`SatPropagate`	clamp	preserve representable infinities, else clamp
`SatNone`	the draft's rows: to ±Inf, finite extreme, or NaN depending on rounding direction, signedness, and domain	preserve, else NaN

One ordering fact the whole clause turns on (§4.7.3 NOTE 1): **rounding precedes saturation**, but the rounding mode is passed *into* saturation so a directed mode can force a finite result. Saturation itself never rounds.

The operation catalog

§4.9–§4.16 define the catalog: `Convert`; sign and basic arithmetic (`Abs` through `Divide`, fused `FMA` and `FAA`); the elementary functions (roots, exp/log families, trig, hyperbolic, π-scaled trig, `Softplus`, `Hypot`, `ArcTan2`); the extrema families (`Minimum`/`Maximum` and their NaN-quieting, magnitude, and finite variants, plus `Clamp`); comparisons, predicates, and `TotalOrder`; format-level queries (`BitwidthOf`, `MaxFiniteOf`, …); and the neighbour operations `NextGreaterThan`/`NextLessThan`.

Some results are visible only in the pattern tables, and they surprise:

- `Divide(x, 0) = NaN` for every x — including ±Inf (§4.10.5 NOTE 2). There is no IEEE-style ±Inf quotient.
- `Recip(±Inf) = 0` and `Recip(0) = NaN` (§4.10.8).
- `TotalOrder` puts NaN first, below −Inf — deterministic tie-breaking for search and sort (§4.12.1), and the opposite end from Julia's float-sorting convention.
- `NextGreaterThan(+Inf) = NaN`, unlike IEEE-754 `nextUp` (§4.16 NOTE 2).
- FAA is associative by construction: `Add(Add(X,Y),Z) = Add(X,Add(Y,Z))` computed exactly (§4.10.7).

Blocks and scaled operations

Clause 5 adds the MX-style scheme: a *block* is a scale factor in one format plus B elements in another, representing `scale × element` per lane. Block operations decode lanes exactly, apply the operation, and project against a result scale; the reductions (`BlockReduceAdd`, `BlockDotProduct`) accumulate exactly and project once. `Scaled0p` is the block-size-1 form. This is how the draft tracks dynamic range that a bare element format cannot hold.

Conformance and κ

The draft's conformance machinery (§4.4–4.6) has two levels. A *conforming implementation* produces the defined result for every operation, format, and mode. A *κ-approximate implementation* may deviate, but must declare its maximum code-point distance κ from the defined result — measured along the total order, over inputs with finite defined results. Approximation is thus quantified and declared, never silent; SmallFloats implements both levels and measures κ by exhaustive enumeration at registration.

Differences from IEEE 754

`Binary16p11se` looks like `Float16`; `Binary16p8se` looks like `BFloat16`. They are not (Annex F compares them explicitly):

	`Binary16p11se`	`Float16`	`Binary16p8se`	`BFloat16`
exponent bias	**16**	**15**	**128**	**127**
largest finite	**65472**	**65504**	**3.3762e38**	**3.3895e38**
NaN encodings	**1**	**2046**	**1**	**2046**
negative zero	**no**	yes	**no**	yes

P3109's bias formula makes the exponent range symmetric about the midpoint; IEEE-754's is `2^(E−1) − 1`. The two differ by one, so **every code point denotes a value a factor of two away from its IEEE look-alike**. The same bits read as the other type silently halve or double every value. Converting between them is a conversion, never a reinterpretation.

Continue

Core Model, The Standard and this Design, First Session.

Chapter 16

Format Anatomy

16.1 Format Anatomy

Every value in SmallFloats.jl belongs to a *format*, and a format is a Julia type. This page explains what that type actually is, why the package draws a hard line between the abstract idea of a format and the concrete type your code uses, and why that line matters for both correctness and speed.

The parametric type

Every format is an instance of the same four-parameter type:

```
Binary{K,P,SGN,EXT}
```

K is the bitwidth, $3 \le K \le 16$. P is the precision — significand bits, including the implicit bit. SGN is a `Bool`: signed or unsigned. EXT is a `Bool`: extended (the domain includes ±Inf) or finite (it does not). Every legal combination of these four parameters is a distinct format, and the package ships all of them: **504 formats** in total.

That is the whole parameter space. There is no fifth axis, and — this is the detail that trips people up when they start comparing formats — K is not an independent budget. It is spent entirely by the other three: $K = P + E + SGN$ where E is the exponent width implied by K, P, and SGN. Raising precision by one bit necessarily lowers exponent range, because there is nowhere else in the word for the extra bit to come from.

<:, not ===

`Binary{K,P,SGN,EXT}` is an abstract type. It is not what a value's type actually is, and it is not what you should write as an array element type or a constructor target. The concrete type is a *representation* — a draft-named alias like `Binary8p4se` — related to the abstract format by subtyping, not equality:

```
julia> Binary8p4se <: Binary{8,4,true,true}      # `<:`, not `===` — see below
true

julia> formatname(Binary{6,3,true,false})
:Binary6p3sf

julia> SmallFloats.Binary16p6se                  # always reachable, unexported
Binary16p6se

julia> format(16, 6, true, true)                 # programmatic, runtime params
Binary16p6se

julia> using SmallFloats.Formats                 # …or bring all 504 into scope

julia> Binary16p6se
Binary16p6se
```

`Binary8p4se` is a concrete subtype of `Binary{8,4,true,true}`, but the two type objects are distinct. The distinction is not an implementation accident; it comes from the draft's own vocabulary. A *format* is a datum set and an encoding — an abstract description of which values exist and how they are named. A *representation* is a specific choice of machine word that carries a code point: how many bits, what storage unit, what bit layout. That choice sits below the level of description the standard cares about. `Binary` names the format; `Binary8p4se` is the representation you actually compute with.

The suffix on an alias reads directly: p4 is precision 4, s/u is signed/unsigned, e/f is extended/finite. `Binary8p4se` is 8-bit, precision 4, signed, extended — read the name and you have the four parameters without consulting a table.

Why 504 formats, and why only 120 are exported

Every legal `(K, P, SGN, EXT)` combination at `K ∈ 3:16` gets an alias, because the whole point of the package is that formats are cheap to have — each one is 256 (or fewer) code points and a handful of type-parameter queries, not a hand-written implementation. But 504 identifiers is more than any one script needs in scope at once, so `using SmallFloats` exports only the **120 aliases at K ≤ 8** — the sizes people actually reach for most: sub-byte and byte-sized formats for quantization, embeddings, and packed storage.

The other 384 (K from 9 to 16) are one opt-in away. You never need to define anything to reach them — they already exist as bindings, just not exported ones:

- `SmallFloats.Binary16p6se` — fully qualified, always available regardless of what's exported.
- `using SmallFloats.Formats` — brings every one of the 504 aliases into scope at once.
- `format(K, P, SGN, EXT)` — the programmatic route, for when the parameters are runtime values rather than something you can spell as a literal type name.

Format introspection

Every format answers a fixed set of queries about its own structure, drawn from the draft's Group M, in both a Julia-flavored spelling and the draft-named form:

```
julia> bitwidth(Binary8p4se), precision(Binary8p4se), expbias(Binary8p4se)
(8, 4, 8)

julia> MaxFiniteOf(Binary8p4se)
Binary8p4se(224.0 ≡ 0x7e)

julia> MinPositiveOf(Binary8p4se)
Binary8p4se(0.0009765625 ≡ 0x01)
```

`MinFiniteOf`, `MinNormalOf`, `MaxSubnormalOf`, `expbitwidth`, `trailingsigbits`, `issigned`, `isextended`, and their draft-named counterparts (`BitwidthOf`, `PrecisionOf`, `ExponentBiasOf`, ...) round out the set.

The important property of every one of these queries is that it is a pure function of the type parameters — nothing about a particular value changes the answer — so it works identically whether you hand it a type or a value:

```
julia> x = Binary8p4se(1.6);

julia> BitwidthOf(x), PrecisionOf(x), SignednessOf(x), DomainOf(x)
(8, 4, true, true)

julia> MaxFiniteOf(x)
Binary8p4se(224.0 ≡ 0x7e)
```

`bitwidth(x)` and `bitwidth(typeof(x))` are the same query, and both fold to a compile-time literal — there is no runtime cost to asking a value what kind of format it lives in.

The abstract format as an array element type

Because `Binary{K,P,Σ,Δ}` is abstract, declaring an array with it as the element type gives you a non-`isbits` element type, and every element is boxed:

```
julia> isbitstype(eltype(Vector{Binary{8,4,true,true}}(undef, 3)))
false

julia> isbitstype(eltype(Vector{Binary8p4se}(undef, 3)))
true
```

Nothing about this errors. Every operation still returns the right answer, because the package normalizes the abstract format at every constructor and entry point it can see. What you silently lose is the entire performance story: table gathers, zero-allocation scalar calls, specialized dispatch — all of it depends on the element type being concrete. This is the sharper edge of the type distinction above: comparing the concrete alias with the abstract format makes their different roles visible, while the abstract-array mistake costs you real performance and says nothing at all.

`similar(a, T)` normalizes the request for you, but `Vector{T}(undef, n)` cannot be intercepted at the call site. Always spell the alias, or use `format(K, P, Σ, Δ)` when the parameters are runtime values.

Continue

Values, Code Points, and Conversion explains how a value stores a code point, how decode recovers its exact datum, and how construction differs from raw code-point access.

Chapter 17

Values, Code Points, and Conversion

17.1 Values, Code Points, and Conversion

This page develops the identity of a `Binary` value: how it is constructed, read, stepped, classified, and converted. Read First Session first if the distinction between datum and code point is new.

Four ways in, two ways out

A `Binary` value is an immutable wrapper around a code point. There are four constructors and two accessors:

```
julia> x = Binary8p4se(1.6)              # construct from a Real: projects (rounds)
Binary8p4se(1.625 ≡ 0x45)

julia> Binary8p4se(0x45)                 # construct from a UInt8 CODE POINT (validated)
Binary8p4se(1.625 ≡ 0x45)

julia> rawvalue(Binary8p4se, 0x45)       # same, unchecked — the kernel-internal route
Binary8p4se(1.625 ≡ 0x45)

julia> Convert(Binary8p4se, RNE_SN, 3)   # explicit conversion, any mode
Binary8p4se(3.0 ≡ 0x4c)

julia> decode(x)                          # the exact datum, on the format's carrier
1.625

julia> codepoint(x)                       # the code point (extends Base.codepoint)
0x45
```

`decode` never rounds — it is a lookup from name to meaning. Construction from a `Real` is the only place rounding happens on the way in.

Important: `Unsigned` means code point, at every width

The First Session already showed you `Binary8p4se(0x02)` and `Binary8p4se(2)` landing on different values. The rule generalizes to every integer width and every format, and it is worth stating as a standalone hazard because it never raises an error — it just silently gives you the wrong datum if you meant a number:

```
julia> Binary8p4se(0x02), Binary8p4se(2)
(Binary8p4se(0.001953125 ≡ 0x02), Binary8p4se(2.0 ≡ 0x48))
```

'Unsigned' means code point — at every width

`T(0x02)` constructs code point `0x02`; `T(2)` projects the numeric value two. Signedness of the integer type is what distinguishes them, so the rule holds for `UInt8`, `UInt16` and any other `Unsigned`: `Binary16p6se(0x0002)` is code point 2, not the value 2.0. Use `Convert` when intent should be unmistakable.

Out-of-range codes throw for K < 8 (`Binary5p3sf(0x20)` is an error); the range check costs nothing measurable — 2.1 ns, identical to unchecked `rawvalue`. Round-tripping is `T(codepoint(x)) === x`.

`Convert` versus `convert`

`Convert` (capitalized, from the spec-named register) is the explicit projection you saw above: `Convert(T, ρ, value)`, numeric for every integer. Base's lowercase `convert` is a different, narrower operation that Julia's promotion machinery calls, and it is value-preserving on `Unsigned` input rather than code-point reading:

```
julia> Binary8p4se(0x02)
Binary8p4se(0.001953125 ≡ 0x02)
```

Keep the two apart: `T(x::Unsigned)` reads a code point; `Convert(T, ρ, x)` always projects a numeric value, whatever its type. When you want projection and the argument might be an `Unsigned`, reach for `Convert`, not the constructor.

The decode / codepoint round-trip

`decode` and `codepoint` are inverses by construction — that round-trip is a format's identity:

```
julia> x = Binary8p4se(1.6)
Binary8p4se(1.625 ≡ 0x45)

julia> codepoint(x)
0x45

julia> decode(x)
1.625
```

`decode` is **always exact**, but the type it returns is a property of the format, not of the package: `Float64` for the 432 formats whose exponent range it holds, `Float128` or an exact dyadic carrier for the other 72. `Float64(x)` always returns a `Float64` and rounds where the datum does not fit.

Enumerating a whole format

Small formats are small enough to look at in full. `Binary4p2se` has 16 code points, sorted here by the draft's total order — which places the single NaN **first**, below −Inf (§4.12.1), not last as `Float64` sorting does:

```
decode.(sort(Binary4p2se.(0x00:0x0f)))      # broadcast the code-point constructor
```

```
[NaN, -Inf, -2.0, -1.5, -1.0, -0.75, -0.5, -0.25, 0.0,
  0.25, 0.5, 0.75, 1.0, 1.5, 2.0, Inf]
```

That listing *is* the format: one NaN, no negative zero, subnormal spacing near zero, binades doubling outward. This is a good way to build intuition for a format before committing to it.

Stepping and classification

Stepping moves to the adjacent code point; classification names what kind of datum you're holding:

```
julia> NextGreaterThan(Binary8p4se(1.0))
Binary8p4se(1.125 ≡ 0x41)

julia> Class(Binary8p4se(0.01))           # draft classification
ClassPosNormal::FPClass = 0x06
```

`NextGreaterThan`/`NextLessThan` are the draft's Next operations; `nextfloat` and `prevfloat` alias them, so both spellings work. Standard predicates apply too: `isnan`, `isinf`, `isfinite`, `iszero`, `signbit`, `issubnormal`:

```
julia> Binary8p4se(1e9)                   # overflow under the default spec → +Inf
Binary8p4se(Inf ≡ 0x7f)

julia> Binary8p4se(-0.0)                  # no −0: projects to the single zero
Binary8p4se(0.0 ≡ 0x00)

julia> isnan(Binary8p4se(NaN)), isfinite(Binary8p4se(2.0))
(true, true)
```

`zero`, `one`, `eps`, `typemax`

The usual numeric-type vocabulary works on every format too — `zero`, `one`, `eps`, `typemin`, `typemax` — alongside the predicates and stepping operations above; they answer from the format's own grid, not from any IEEE assumption about it.

Try it

Using only what this tutorial covered: what does `decode(Binary8p4se(0x01))` return, and how is it different from `decode(Binary8p4se(1))`? Work it out before checking.

<details> <summary>Answer</summary>

`Binary8p4se(0x01)` reads code point `1` — the smallest positive subnormal for this format, `0.0009765625` (shown as `MinPositiveOf(Binary8p4se)` in the User Guide). `Binary8p4se(1)` projects the numeric value one onto the nearest datum, `1.0`. Same digit, two different constructors, two very different answers — the pitfall from this tutorial in miniature.

</details>

Continue

Rounding and Saturation and Formats and Value Queries.

Chapter 18

The Exact-Then-Project Contract

18.1 The Exact-Then-Project Contract

Every number that enters a `Binary` format goes through the same pipeline, regardless of whether it arrives via a constructor, an arithmetic operator, or an explicit conversion. This page explains what that pipeline promises, what "exact" means at each stage, and the handful of cases where the contract refuses rather than bends.

One write path

There is exactly one way a code point gets produced in SmallFloats.jl:

```
exact result  →  RoundToPrecision  →  Saturate  →  code point
```

The operation is computed *exactly* first — as exact real arithmetic, not as a simulation of some intermediate low-precision format — and only then is the result rounded to the target precision and, if it falls outside the representable range, saturated according to the chosen saturation mode. This holds for a scalar `Add`, for a fused `FMA` (one rounding, not two), and for a full `BlockDotProduct` (every lane product and the accumulation exact, projected once at the end). There is no second path that skips a step or rounds twice; every code point the package ever produces came out of this same pipeline.

The consequence worth sitting with: there is no accumulated rounding error *inside* an operation. Rounding error exists only at the single boundary where the exact result crosses into a code point — never before it.

What "exact" means, per carrier

"Exact math" has to happen somewhere concrete, and the carrier it happens on depends on the format. `decode(x)` always returns the format's exact datum, never a rounded approximation of it — but the *type* it returns is a property of the format, not a package-wide constant.

For 432 of the 504 formats, the datum's exponent range fits inside a `Float64`, so `decode` returns a `Float64` and the exact arithmetic runs there. For the other 72 — formats whose exponent range or precision a `Float64` cannot hold without rounding — `decode` returns `Float128` or an exact dyadic carrier instead, so no precision is lost representing the datum in the first place. `datumcarrier(T)` names which carrier a given format uses, and it is this carrier, not `Float64` universally, that the exact-math half of the pipeline runs on.

This distinction matters because `Float64(x)` always returns a `Float64` regardless of the format's own carrier, and rounds wherever the datum does not fit — it is a display/interop convenience, not the internal computation path. At the user level, `promotecarrier(T)` names the *promotion* carrier — `Float64` for the 432 formats at the ordinary rung, `BigFloat` for the other 72 — which is the type an operation between a `Binary` value and an ordinary number lands in.

Symbolic sticky: projecting "just below" a number

Some values are not just numbers — they are the limit of a sequence approaching a number from one side, and a directed rounding mode needs to know which side to land the projection on. `tanh` approaching 1 as its argument grows is the canonical case: the exact mathematical limit is 1, but `tanh(x) < 1` for every finite `x`, so a correct projection under `TowardNegative` must round *down from* 1, not treat 1 as an ordinary carrier value sitting exactly on a code point.

The engine handles this with a `sticky` argument, `sticky ∈ {-1, 0, +1}`, meaning "the true value is the carrier value plus an infinitesimal of this sign." This lets `project` land exactly on the correct neighbor for enclosure endpoints and asymptotes without inventing a fake carrier value that isn't quite 1:

```
using SmallFloats: project
project(Binary8p4se, ProjSpec(TowardNegative(), SatNone()), 1.0; sticky=-1)
```

```
Binary8p4se(0.9375 ≡ 0x3f)     # == NextLessThan(1.0): the engine crossed the binade
```

Read this carefully: the carrier value passed in is `1.0`, exactly representable in `Binary8p4se`. Without `sticky`, a directed-toward-negative projection of an exactly-representable value is the identity — it would return `1.0` unchanged. `sticky=-1` tells the engine that the true value sits an infinitesimal *below* 1.0, so `TowardNegative` must move to the next lower code point instead of stopping at 1.0 itself — and it does, crossing the binade boundary down to `0.9375`, which is exactly `NextLessThan(1.0)`. This is how the package gets asymptotic behavior like `tanh → 1⁻` exactly right under every rounding mode, rather than approximately right under most of them.

Why `Rational` inputs are refused, not double-rounded

`Convert` accepts `Binary` values of any format, `Float16`/`Float32`/`Float64`, `Float128`, any `Integer`, and `BigFloat` — every type whose values the package can project *exactly* onto the carrier it needs. `Float128` in particular projects directly, preserving all 113 significand bits, which matters when a value sits just above a rounding midpoint: staging through `Float64` first would round it onto the midpoint and then break the tie the wrong way, a genuine double rounding that changes the answer.

`Rational` is conspicuously missing from that list, and that is deliberate rather than an oversight. A `Rational{Int}` can represent a value that no supported carrier holds exactly (a denominator that is not a power of two, for instance), so converting it would require rounding once to reach a carrier and again to reach the format — silently, with no record of it having happened. That would put a second, invisible rounding step inside what looks like a single conversion, which is exactly the guarantee the one-write-path contract exists to prevent.

So `Convert` refuses `Rational` inputs outright, with an error that tells you what to do about it: convert explicitly to a supported carrier first (`Binary8p4se(Float64(π))` for irrational values, or `Binary8p4se(Float64(r))` for a `Rational` `r`) and thereby own the double rounding as a choice you made, not a default the package made for you.

Why the contract is a single, named function

All of this — exact math, rounding, saturation, and the sticky adjustment for asymptotic limits — lives behind one function, `project`, rather than being reimplemented at each call site that needs a code point. That centralization is what makes the "one write path" claim checkable rather than aspirational: every operation in the package, from a scalar `Add` to a table build to a block reduction, is required to produce its code point by calling through `project`, and the test suite is able to assert this because there is exactly one place the assertion needs to look. A second hand-rolled rounding step anywhere in the codebase would be a bug by definition, not a matter of style.

This is also why the table-gather performance strategy described elsewhere in the documentation costs nothing in correctness. A cached table entry and a fresh scalar computation are guaranteed identical precisely because both routes go through `project` — the table is not a separate, hopefully-equivalent fast path, it is the *same* path, memoized. Nothing about caching results for speed introduces a second contract to keep in sync with the first.

The same reasoning extends to approximate kernels. When a registered approximation replaces the default path for one operation, it is measured against exactly this contract's output — the exact-math-then-project result — rather than against some looser notion of "close enough." The projection contract is not just how the default path stays correct; it is also the fixed reference every deliberate approximation in the package is honestly scored against.

That is what makes a declared κ a fact about the *difference* between two functions, rather than a vague impression of "close enough" — both sides of the comparison ultimately trace back to the same contract this page describes.

Continue

Rounding and Saturation develops the two policy axes. Control Rounding and Overflow applies them to midpoint and boundary cases.

Chapter 19

Rounding and Saturation

19.1 Rounding and Saturation

A projection answers two independent questions. Rounding chooses a neighbor when an exact value lies between datums. Saturation chooses the permitted result when the rounded value lies outside the format's range or belongs to a special boundary case.

Projection is a product of policies

```
exact result → rounding decision → saturation decision → code point
```

SmallFloats represents the pair as `ProjSpec(rounding, saturation)`. Names such as `RNE_SN` are convenient values, not fused algorithms. The independence is important: changing saturation must not silently change how in-range values round, and changing rounding can alter which side of a boundary is reached.

Deterministic rounding modes

The deterministic modes answer different questions at the same pair of neighboring datums:

Family	Decision
nearest, ties to even	nearest datum; a tie selects the even code lattice choice
nearest, ties away	nearest datum; a tie increases magnitude
toward positive	least representable datum not below the exact value
toward negative	greatest representable datum not above the exact value
toward zero	neighbor no farther from zero
to odd	select the odd endpoint when discarded information is nonzero

Directed rounding describes order, not simply “up” or “down” in magnitude. For a negative exact value, toward positive moves toward zero while toward negative moves away from zero.

Stochastic rounding modes

Stochastic modes make the landing a function of both the exact fractional position and a finite random draw. They are defined distributions. With an explicit draw `R`, a stochastic projection is deterministic and can be tested exhaustively.

The mode's random-bit budget determines the draw range. An `N`-bit mode accepts `R ∈ 0:(2^N - 1)`; out-of-range draws are contract violations rather than silently wrapped values.

Saturation modes

`SatFinite` clamps results to the finite range. `SatPropagate` preserves representable infinities under its contract and handles other overflow by its policy. `SatNone` applies the draft's full domain-, sign-, direction-, and operation-dependent rows.

The name `SatNone` is easy to misread. It does not mean “do nothing” and it does not mean “never saturate.” It means neither simplifying policy was selected.

Rounding and range interact

Suppose an exact positive result lies just beyond the largest finite datum. Toward zero may land back on that finite datum before saturation classifies the result, while nearest may cross into overflow. Consequently two specs with the same saturation mode can produce different boundary outcomes.

This is why the package projects once through a combined spec rather than rounding in one call and clamping in another. Separating them operationally would introduce an extra representable intermediate and could change the defined result.

Construction, conversion, and operations share projection

`T(x)`, `Convert(T, ρ, x)`, and `Op(T, ρ, operands...)` all end at the same projection engine. Their difference is how the exact input to projection is obtained and whether the policy is implicit or explicit.

That shared path supports a useful reasoning rule: once you know the exact mathematical value and the `ProjSpec`, you can reason about construction, conversion, scalar operations, and array table entries in the same way.

Consequences for users

- Name ρ in reusable numerical code.
- Test ties and both signs when selecting a rounding policy.
- Test both range boundaries and special values when selecting saturation.
- Record stochastic bit budget and entropy source.
- Do not emulate a spec with a sequence of Base rounding and clamp calls.

Use it

Control Rounding and Overflow and Make Stochastic Work Reproducible.

Look it up

Projection Specifications.

Chapter 20

Julia's Numeric Model

20.1 Julia Numeric Behavior

Mixing two `Binary` formats in an arithmetic expression fails. Mixing a `Binary` value with an ordinary Julia number succeeds, but the result is not a `Binary` value. Both behaviors are deliberate, and this page explains the reasoning behind each, along with why the obvious third option — a `promote_rule` that produces a wider `Binary` format automatically — was considered and rejected.

Mixed `Binary` formats: deliberate failure, not silent widening

Adding two values from different formats does not silently pick a format wide enough to hold both and round into it. It fails, with an error that names the problem:

```
julia> x = Binary8p4se(1.5);

julia> x + Convert(Binary8p4se, RNE_SN, Binary5p3sf(1.0))
Binary8p4se(2.5 ≡ 0x4a)
```

The line above only works because the `Binary5p3sf` value was converted *explicitly* first. Without that conversion, `x + y` for `x::Binary8p4se` and `y::Binary5p3sf` raises a `promotion ... failed to change` error rather than silently choosing some third format to compute in. This is not a gap in the promotion machinery — it's the package's stated position that mixing formats is a decision the caller makes explicitly, with `Convert`, never one the package makes for you by picking a resolution.

`Binary` with ordinary numbers: promotes to the promotion carrier

Combining a `Binary` value with an ordinary Julia number — an `Int`, a `Float64` — does not fail, but it does not produce a `Binary` value either. It promotes to the format's **promotion carrier**, an ordinary float type, and the result is that ordinary float:

```
julia> x + 2.0
3.5
```

For 432 of the 504 formats — the ones whose exponent range and precision fit inside a `Float64` without rounding — that carrier is `Float64`, which is what the example above returns. For the other 72, the carrier is `BigFloat`: a `Float64` cannot hold a `Binary16p1uf` datum exactly, and promoting into `Float64` anyway would silently turn a perfectly finite value into `±Inf` with no P3109 operation anywhere in sight. `promotecarrier(T)` names the target carrier for any format, so the promotion is queryable rather than a fact you have to remember.

If you actually want the result back in the original format, project it back explicitly — the package will not do this step for you, for the same reason it won't guess which format to promote two mixed `Binary` values into:

```
Convert(Binary8p4se, RNE_SN, decode(x) + 2.0)
```

Why widen is not another format

It's tempting to think the fix for mixed-format arithmetic is obvious: define a `promote_rule` between two `Binary` types that picks the smallest format containing both operands' datum sets, the way `promote_type(Int, Float64)` picks `Float64`. A companion internal study worked this out in detail, and the answer is that this rule cannot be built soundly. Formats trade precision for range through the identity K = P + E + σ — raising precision by one bit necessarily lowers exponent range by the same amount — so two formats at the same bitwidth are usually *incomparable*: neither contains the other's datum set. Measured over the full 504-format grid, a "smallest format containing both" answer exists for only about 60% of format pairs; the other 40% have no common format at all within the K ≤ 16 grid, so any `Binary`-valued rule would have to fall back to something else for those pairs anyway.

Worse, even where the smallest containing format does exist, picking it pairwise is not associative — because Julia's `promote_type(a, b, c)` reduces pairwise, a non-associative rule would make the *result format* depend on the order the operands happened to appear in a + chain, which is exactly the kind of instability the package refuses to ship. And even in the cases where the rule is well-defined, it is often surprising on its own terms: two formats that agree on everything except whether they carry infinities do not promote to either input's own bitwidth — they need one bit more, because the wider format has to hold both the finite format's largest value *and* an infinity code point, and at the narrower width there is no code point left for both. A silent promotion landing in a format the user never named, at a width neither operand has, is the wrong kind of convenience for a package whose entire premise is that you choose your result format. The full argument, with the exact percentages and the associativity counterexample, is worked out in the promotion-rules study (docs/other/promoterules.md in the repository).

Why this differs from `Int`/`Float64` promotion

It helps to be precise about what makes format promotion a different problem from ordinary numeric promotion. `promote_type(Int, Float64)` works because the target, `Float64`, is fixed in advance and holds every `Int` exactly (up to `2^53`) — the rule never has to ask "which format contains both operands" because there is only one plausible answer and it never runs out of room. Two `Binary` formats have no such fixed universal target: the format space is a lattice of finite datum sets, not a single always-sufficient supertype, and which format contains which shifts with every combination of precision, range, signedness, and domain. The question `promote_type` answers for ordinary numbers is trivial; the same question for two arbitrary `Binary` formats is a genuine combinatorial problem, and the promotion-rules study exists because that problem was worked out in full rather than assumed away.

Explicit `Convert` as the answer

Given that no automatic rule is both sound and predictable, the package's answer is to make the choice explicit every time: `Convert(T, ρ, x)` moves any value into the format T you name, under the projection ρ you choose. There is no ambiguity about which format the computation happens in, no order-dependence, and no case where the rule silently falls back to a carrier for some inputs and a `Binary` format for others. The lattice of which format contains which is real and occasionally useful to know explicitly, but it belongs in a query you call when you want the answer, not in an operator that decides for you every time two formats meet.

This is the same philosophy that governs every other write path in the package: one explicit projection, chosen by the caller, rather than a rule that infers intent from context. A `promote_rule` between two `Binary` formats would be exactly the kind of implicit rounding decision the rest of the design goes out of its way to avoid — it would round somewhere other than inside `project`, which is the one invariant the whole package is built not to break. Treating cross-format arithmetic as a decision you make explicitly, rather than a convenience the library infers, is not a limitation bolted on around an otherwise-automatic feature; it is the same rule that governs rounding modes, saturation modes, and result formats applied consistently to one more place where an implicit choice could have hidden.

What this buys you as a caller

The practical upshot is that a `Binary` expression never surprises you about which format it computed in. If every operand in an expression shares a format, the result is that format, under whatever projection was in play. If you mix in an ordinary number, the result predictably drops to the promotion carrier, and you can query `promotecarrier(T)` in advance to know which carrier that will be. And if you try to mix two different `Binary` formats without converting, the error fires immediately rather than letting a silently-widened result propagate three functions deeper before something notices the format changed. All three outcomes are things you can predict from the types alone, before running anything — which is the actual goal a promotion rule is supposed to serve, achieved here by refusing the one case that could not serve it honestly.

Continue

Values, Code Points, and Conversion gives the constructor and `Convert` rules. Julia Compatibility Register lists the complete Base surface.

Chapter 21

Session State and Reproducibility

21.1 Session State and Reproducibility

Six values control what "the default" means for any convenience call in SmallFloats.jl — the format `T(2.1)` produces, the projection `x + y` follows, the RNG `rand(T)` draws from. This page explains what those six defaults are, how they stay consistent with each other, why they are dangerous to set casually, and the safe pattern for consuming them.

The six defaults

Six session-wide defaults are readable as `DefaultX()` and settable as `DefaultX!(v)`:

default	initial value	setter accepts
`DefaultType`	`Binary8p2se`	any fully-parameterized `Binary` type
`DefaultReturnType`	`Binary8p2se`	any fully-parameterized `Binary` type
`DefaultRoundingMode`	`NearestTiesToEven()`	mode instance or type
`DefaultSaturationMode`	`SatNone()`	mode instance or type
`DefaultProjection`	`RNE_SN`	a `ProjSpec`, or `(mode, sat)`
`DefaultRNG`	the Xoshiro type	RNG type or instance
`DefaultRbits`	8	`Int` in `1:60`

Coherence in both directions

`DefaultProjection` and its two component defaults, `DefaultRoundingMode` and `DefaultSaturationMode`, are kept coherent no matter which one you change. Setting `DefaultRoundingMode!` or `DefaultSaturationMode!` rebuilds `DefaultProjection` around the new component; setting `DefaultProjection!` directly decomposes it back into both components. The invariant `DefaultProjection() === ProjSpec(DefaultRoundingMode(), DefaultSaturationMode())` holds always, regardless of which of the three setters you used last:

```
julia> DefaultRoundingMode!(TowardZero())
TowardZero()

julia> DefaultProjection()               # followed the component
(TowardZero, SatNone)

julia> DefaultProjection!(RNA_SF)
(NearestTiesToAway, SatFinite)

julia> DefaultRoundingMode(), DefaultSaturationMode()   # followed the projection
(NearestTiesToAway(), SatFinite())

julia> DefaultProjection!(RNE_SN)        # restore: these setters are PROCESS-wide
(NearestTiesToEven, SatNone)
```

The defaults are process-global

The session defaults are process-global and they stay set

`DefaultProjection!` and its siblings mutate state shared by every module in the process. Nothing scopes them for you: set one in a script, a notebook cell, or a REPL session and *every* later `T(x::Real)`, `a + b`, and `Exp(x)` follows it until something sets it back.

Prefer `with_default_projection` (and `with_default_type`, `with_default_returntype`), which restore on exit even if the body throws:

```
with_default_projection(RTZ_SF) do
    Binary8p4se(1e9)          # clamps, inside this block only
end
```

Every example in the manual assumes the pristine `(NearestTiesToEven, SatNone)`, which is why each demonstration that changes a default restores it explicitly afterward.

`DefaultProjection` is not merely advisory — `default_projspec` reads it, so the same-format convenience methods (`a + b`, `Exp(x)`, ...), the Base-register operators, and `T(x::Real)` construction all follow it:

```
julia> x, y = Binary8p4se(200.0), Binary8p4se(2.0);

julia> x * y                              # RNE_SN: overflow → +Inf
Binary8p4se(Inf ≡ 0x7f)

julia> DefaultProjection!(RTZ_SF);

julia> x * y                              # now clamps to MaxFinite
Binary8p4se(224.0 ≡ 0x7e)

julia> DefaultProjection!(RNE_SN)         # restore before moving on — see below
(NearestTiesToEven, SatNone)

julia> Multiply(Binary8p4se, RNE_SN, x, y)   # explicit ρ is unaffected
Binary8p4se(Inf ≡ 0x7f)
```

Changing the default is a global semantic change

Every caller of the convenience forms — including code in other packages — sees it. Library code that needs a specific projection should name it explicitly rather than rely on the session default.

The `with_default_*` combinators

To *consume* a default in your own code without paying dynamic-dispatch costs, go through the `with_default_*` combinators — `with_default_type`, `with_default_returntype`, `with_default_projection` — which call `f(default, args...)`:

```
julia> with_default_type((T, x) -> T(x), 1.5)
Binary8p2se(1.5 ≡ 0x41)
```

The speculation-guard cost model

Following the default costs nothing while it holds its initial value: the convenience forms consume it through a speculation guard, the same mechanism the `with_default_*` combinators use, so they compile against the constant and stay allocation-free with concretely inferred results — this is pinned in the test suite, not an incidental optimization. Once you change the default, they cross a function barrier instead: one dynamic dispatch per call, with everything inside that barrier still fully specialized. The explicit forms, `Add(T, ρ, x, y)`, are unaffected either way — they never read a default in the first place.

Allocation contract

When `f`'s result type does not depend on the default — `with_default_projection` with the formats fixed by the caller is the normal shape — the call is zero-allocation with a concretely inferred result, pinned in the test suite. When the result's type *is* the default (`with_default_type` used as a constructor, where the returned value's type is whatever `DefaultType` currently names), the value is still computed on the specialized path, but it boxes once where it escapes the function barrier — the irreducible cost of returning a runtime-chosen type.

There is no accumulator default

There is no accumulator default

`DefaultAccumulatorType` existed through v0.1.0 and was removed. The wide-precision accumulator used inside a reduction is not a matter of preference: the span filter measures the operands and picks the cheapest carrier that is *provably exact* for them, so the reduction returns the defined result regardless of which carrier that turns out to be. A knob overriding that choice would make `BlockDotProduct` inexact from the default API — a much larger cost than losing a configuration option. To trade speed for nothing else, set `ENV["SmallFloats_Float128"] = "disable"`, which is bit-identical by construction and only affects build/oracle speed.

Continue

Rounding and Saturation explains the policy stored by `DefaultProjection`. Make Stochastic Work Reproducible shows when to replace implicit state with an explicit RNG or draw.

Chapter 22

Performance Model

22.1 Performance Model

Performance advice for SmallFloats.jl is scattered across the manual, the cheat sheet, and the technical examples because it applies at every layer — construction, arithmetic, arrays, storage. This page pulls all of it into one spine: the handful of decisions that determine whether your code runs at scalar speed (tens of nanoseconds) or falls off a cliff into microsecond dynamic dispatch.

Pass formats statically

Every scalar entry point in the package — `Add`, `project`, a constructor — fully specializes when the format type is known at compile time: through a `const` binding, a type parameter, or an ordinary function argument. Under that condition, a scalar `Add` runs at roughly 18–26 ns and `project` at roughly 13 ns, with zero allocations.

The moment a format type is read from a **non-`const` global**, Julia can no longer resolve the method at compile time, and every call pays for dynamic dispatch instead — roughly 1 µs for a keyword call, a difference of nearly two orders of magnitude for the identical computation:

The 60× benchmarking slowdown

The same benchmark, run with the format type `T` read from a non-`const` global instead of passed as a type parameter, measures around 1 µs — Julia's dynamic keyword dispatch overhead, not this package's arithmetic. Two real project post-mortems trace their "SmallFloats is slow" reports to exactly this mistake. The shipped `benchmarking/benchmarking.jl` asserts zero warm-path allocation before it believes any number it produces, and your own benchmarks should do the same:

```
using Chairmarks, SmallFloats, Random
using Statistics: median

function bench_add(::Type{T}) where {T<:Binary}          # T: type parameter, not a global
    pool = [rawvalue(T, rand(UInt8)) for _ in 1:4096]
    @be (rand(pool), rand(pool)) (t -> Add(T, RNE_SN, t[1], t[2]))(_)
end

b = bench_add(Binary8p4se)
(round(median(b).time * 1e9; digits=1), median(b).allocs)
```

```
(16.3, 0.0)          # ns per full scalar Add, zero allocations
```

The fix, when a format genuinely has to come from a runtime value, is one function barrier: write `f(::Type{T}, …) where {T}` and call into it once the type is known, and everything inside specializes at full speed again.

Format operands entering `Chairmarks`-style benchmarks specifically must come from the untimed `setup` phase, and if you retrieve an operation function reflectively (`getfield(SmallFloats, op)`), pass it through an argument barrier so it specializes too — the same discipline that makes the benchmark trustworthy also makes ordinary calling code fast.

Convenience forms are free at the initial default

`x + y`, `Exp(x)`, and `T(2.1)` all read the session default projection through a speculation guard rather than a dynamic lookup. While the default holds its initial value, the guard lets these calls compile against a constant, so they are allocation-free with concretely inferred results — a property pinned in the test suite, not an incidental benchmark result. The moment you change the default, these same calls cross a function barrier instead: one dynamic dispatch per call, with everything inside that barrier still fully specialized. Code that must stay insensitive to whatever the session default happens to be should name its projection explicitly — `Add(T, ρ, x, y)` with a `const ρ` — rather than rely on the convenience spelling in a hot loop.

Bulk work belongs in array calls

The table-gather kernels that back array operations run roughly 50× faster than the scalar path per element, because after the first call for a given `(op, formats, ρ)` specialization, every later element is a single table lookup rather than a full evaluation. That first call pays a one-time table build: roughly 0.4 ms for a unary table, tens of milliseconds for an 8×8 binary table, up to a few milliseconds for a 2 MiB ternary table. Benchmark warm calls separately from the first cold call, or the build cost will dominate a measurement that is supposed to be about steady-state throughput. Once warm, measured throughput is around 0.27 ns/element for unary gathers and 0.5 ns/element for binary gathers.

Ternary tiers scale with bitwidth

Ternary operations (`FMA`, `FAA`, `Clamp`) ride the same table-gather mechanism whenever the combined operand bitwidth keeps the resulting table affordable, and the package picks the tier automatically — no action needed on your part:

- **Eager** (up to 256 KiB; every all-K ≤ 6 signature): built on the first array call, exactly like a unary or binary table.
- **Adaptive** (up to 2 MiB; the K = 7 band): built only once a signature has processed enough elements to earn its build, held in a byte-bounded, LRU-evicted cache — so a signature used once in passing doesn't pay for a table it will never reuse.
- **Compute** (K = 8; a 16 MiB table stops being a cache win): the scalar pipeline runs per element instead, optionally threaded for long arrays (roughly 4× at 4 threads).

Every table entry, eager or adaptive, is built through the exact scalar path, so a table lookup and the equivalent scalar call are bit-identical by construction — the speed comes with no accuracy trade at all.

Stochastic array calls are always per-element

Deterministic array operations use cached tables; stochastic ones cannot, because each element needs its own independent random draw. Stochastic array calls always run the scalar pipeline once per element, consuming one draw per projection. Pass an explicit `rng` when you need the result reproducible — the array and `!` forms always use whatever RNG you supply, so this is the one lever available for controlling a stochastic array call's output exactly.

Memory: `PackedVector` and `BlockVector`

When storage bandwidth matters more than direct byte-level access, `PackedVector{F}` stores code points at `bitwidth(F)` bits per element rather than rounding up to a whole byte or word — a `Binary5p2se` vector at 5 bits/element instead of 8. It behaves as a full `AbstractVector{F}` for indexing, `collect`, and iteration, and `vmap` accepts it directly, unpacking cache-friendly tiles internally rather than materializing the whole array. The governing rule is *store packed, compute unpacked*: there is deliberately no in-place packed arithmetic, because computing directly on sub-byte-packed bits would give up more in kernel complexity than it could ever save in bandwidth. `BlockVector` is the analogous structure for many same-shape `Block`s, holding them in a structure-of-arrays layout instead of an array of individually-boxed blocks.

`table_bytes()` reports the current cache footprint across unary, binary, and ternary tables alike; `empty_tables!()` resets it, which is useful both for isolating a benchmark from a previous session's warm tables and for freeing memory in a long-running process that has touched many format specializations.

Sorting: O(n) counting sort

Sorting is special-cased rather than falling through to Base's generic comparison sort. `Binary` values compare through integer order keys, and a vector of `Binary` values sorts with an **O(n) counting sort** installed as the default algorithm — roughly 8× the stock comparison sort at 64K elements, with `rev=true` supported at the same cost. `sort(A)` simply picks this path up automatically. P3109's total order places the single NaN first (last under `rev=true`), unlike Base's ordering for ordinary floating-point NaNs.

Do not use `@fastmath`

Every number in the performance model above depends on the one-write-path contract: exact math, then exactly one projection. `@fastmath` exists in Julia to relax IEEE semantics for speed — reordering operations, assuming no NaNs, dropping distinctions the standard is careful about — and every one of those relaxations is exactly the kind of silent rounding shortcut this package's whole design refuses to take. Applying `@fastmath` to code using `Binary` values does not make the arithmetic faster in any meaningful way, because the actual bottleneck the sections above address is dispatch and table-gather behavior, not floating-point instruction scheduling — but it can silently invalidate the exactness guarantee that makes the table-gather speedup safe to rely on in the first place. If a computation is slow, the fix is one of the items above: static format types, a function barrier, array calls instead of scalar loops, or `PackedVector` for memory pressure — never `@fastmath`.

Continue

> Continue with **Benchmarking Without Fooling Yourself** (Understanding track) for the full benchmark doctrine — Chairmarks setup discipline, argument barriers for reflectively-retrieved functions, and the warm/cold table-build measurement split.

Chapter 23

Defined, Stochastic, and Approximate Results

23.1 Defined, Stochastic, and Approximate Results

SmallFloats can produce results that vary for three very different reasons. Keeping them separate is essential: a selected rounding policy is not an implementation error, and a registered approximation is not a defined result.

Defined result

A defined result is the value required by the selected operation, result format, and projection specification. For a deterministic spec, the same inputs always produce the same code point.

```
julia> x, y = Binary8p4se(1.625), Binary8p4se(1.75);

julia> Add(Binary8p4se, RNE_SN, x, y)
Binary8p4se(3.5 ≡ 0x4e)
```

The exact sum is 3.375. `RNE_SN` determines the one permitted landing. The scalar path, array path, and any cached function table must agree on code point `0x4e`.

Stochastic defined result

A stochastic projection is still defined. Its additional input is a random draw. Once that draw is fixed, the result is deterministic and testable.

```
σ = RSA_SN(4)
x = 2.0 + 3/64
[codepoint(SmallFloats.project(Binary8p4se, σ, x; R)) for R in 0:15]
```

For this value, exactly 3 of the 16 draws round upward. The distribution is part of the stochastic mode's contract. Passing an explicit `R` tests one point in that contract; passing a seeded RNG reproduces a stream of draws.

The result varies because policy requests variation, not because the implementation is imprecise.

Approximate result

An approximate result comes from an implementation registered as intentionally different from the defined operation. It is reachable only by name through the approximation registry.

The registry records κ, the maximum distance in code points between the approximate and defined results over the measured input domain. Registration measures that distance and rejects a claimed bound that is too small.

```
κ, exhaustive = measure_kappa(candidate, :Tanh, Binary8p4se,
                              (Binary8p4se,), RNE_SN)
register_approx!(:fast_tanh, :Tanh, Binary8p4se,
                 (Binary8p4se,), RNE_SN, candidate; κ)
```

The pair `(κ, exhaustive)` matters. κ says how far the implementation moved; the Boolean says whether the measurement covered the complete finite input space.

The provenance test

When two runs differ, ask these questions in order:

1. Did the operation, result format, or projection specification change?
2. Is the projection stochastic, and did the draw or RNG stream change?
3. Did the call explicitly select a registered approximation?
4. Did process-global defaults change between calls?

If all answers are no, a changed code point is a defect. This decision rule is more useful than treating every difference as "floating-point noise."

Consequences for experiments

- Record the result format and projection spec with an experiment.
- Record the RNG algorithm and seed for stochastic work.
- Record approximation name, declared κ, measured κ, and whether measurement was exhaustive.
- Export `conformance_dict()` with artifacts that need an auditable numerical contract.
- Compare code points when verifying conformance; compare decoded application metrics when evaluating usefulness. They answer different questions.

Use it

Make Stochastic Work Reproducible, Register an Approximation, and Read and Export Conformance.

Look it up

Projection Specifications and Conformance and Approximation Registry.

Chapter 24

Code-Point Algebra

24.1 Formal Code-Point Algebra

Everything the package does to a `Binary{K,P,S,D}` value is integer arithmetic on its code point. This page states that arithmetic as identities — where the distinguished code points are, how a code decodes to a value, how the two directions invert each other, and how ordering is recovered.

It is the layer *below* Formats, Aliases & Representations. Read that page for what a format is and how to name one; read this page when you need to compute with code points, prove something about them, or check the package against an independent derivation.

Verified, not transcribed

Every identity below was checked against the package over **all 504 formats** — the distinguished codes for each, and the decoding formula at **every positive finite code point** of every format. The check is the script in Verification Sessions; it is the same kind of exhaustive enumeration the test suite runs, and it reported no disagreement.

Source for the derivations and proofs: *Code-point forms for `Binary{K,P,S,D}`* (IEEE P3109 working material). This page carries the results and the verification; the proofs are there.

Parameters

A format is four numbers. The package writes signedness and domain as `Bool`; the algebra writes them as `0`/`1`, which is what makes the identities one-liners rather than case analyses.

symbol	meaning	value	package spelling
`K`	bitwidth — bits in the code	given, $3 \le K \le 16$	`bitwidth(T)`
`P`	precision — significand digits *including* the hidden bit	given, $1 \le P \le K-S$	`precision(T)`
`S`	signedness	`0` unsigned, `1` signed	`issigned(T)`
`D`	domain	`0` finite, `1` extended	`isextended(T)`

Derived quantities

symbol	meaning	formula	package spelling
Q	total code points	2^K	—
H	sign-bit weight; signed-region offset	2^(K−1)	signmask(T)
L	trailing-significand patterns per exponent block	2^(P−1)	—
Ebits	exponent-field bitwidth	K−P+1−S	expbitwidth(T)
B	exponent bias	2^(K−P−S) = 2^(Ebits−1)	expbias(T)
η	minimum positive datum; the universal value quantum	2^(2−P−B)	MinPositiveOf(T)
ν	minimum positive normal	2^(1−B) = Lη	MinNormalOf(T)
σ	maximum positive subnormal	(L−1)η = ν−η (when P > 1)	MaxSubnormalOf(T)
δ(j)	ulp of exponent block j	2^(j−B−P+1) = 2^(j−1)η	—

The bias is not the IEEE-754 one

P3109 uses B = 2^(Ebits−1), where IEEE-754 uses 2^(Ebits−1) − 1. The difference of one is exactly why Binary16p11se is not Float16 — every code point denotes a different value. See Why Binary16p11se is not Float16.

The NaN anchor

This is the structural keystone. The NaN code is

```
N = Q − 1 − S(H − 1)
```

which specializes to N = Q−1 (unsigned: all ones) and N = H (signed: sign bit only, nothing else). Both signed and unsigned formats place the positive exceptional endpoint immediately *below* N, so anchoring at N collapses what would otherwise be a signed/unsigned case split into two one-line identities:

```
C(+∞)         = N − 1         (when D = 1)
C(MaxFinite) = N − 1 − D     always
```

Worked for Binary8p4se — K=8, P=4, S=1, D=1, so Q=256, H=128, L=8, B=8:

quantity	formula	value	confirms
N	255 − 1·127	128 = 0x80	Binary8p4se(NaN) ≡ 0x80
C(+∞)	N − 1	127 = 0x7f	Binary8p4se(Inf) ≡ 0x7f
M = C(MaxFinite)	N − 1 − D	126 = 0x7e	MaxFiniteOf(Binary8p4se) ≡ 0x7e

Decoding: code to value

Split a positive finite code by Euclidean division on the block size L:

```
j = c ÷ L        the biased exponent (unbiased: e = j − B)
t = c mod L      the trailing significand
```

Then the whole positive finite range decodes with **one** formula in each regime, and the two agree at the boundary:

```
V(c) = c · η                                  1 ≤ c < L      (subnormal)
V(c) = (L + t) · 2^(j−1) · η                  L ≤ c ≤ M      (normal)
```

The normal form is more familiar written as V(c) = (1 + t/L)·2^(j−B), which is the usual hidden-bit reading; the η-unit form above is the one to compute with, because it is integer arithmetic times a single power of two.

Compute the exponent once

`(L + t) · 2^(j−1) · η` and `(L + t) · 2^(j−1+2−P−B)` are the same number, but the first overflows `Float64` at the wide formats while the second does not. This is not hypothetical — it is exactly the mistake the verification script for this page made on its first run, and it reported 1405 false "defects" in the package before the arithmetic was corrected. Combine the exponents, then `ldexp` once.

Integer image

For positive finite `c`, `I(c) = V(c)/η` is an integer — the value counted in quanta. Working in `I` turns questions about values into questions about integers, which is what makes exhaustive proofs about small formats tractable.

Within a binade

Inside one exponent block, code distance *is* ulp distance. For $c_1 \le c_2$ with $c_1 \div L = c_2 \div L = j$:

$$V(c_2) - V(c_1) = (c_2 - c_1) \cdot \delta(j)$$

Two consequences the package relies on:

- **Midpoints are exact.** The value halfway between two same-binade neighbours is representable in the next precision up, which is what makes tie-breaking a decidable integer question rather than a floating-point comparison.
- **`codedistance` is meaningful.** The κ registry measures approximation error in *code points* precisely because within a binade that unit is proportional to the value error, and across binades it is the natural relative unit.

Ordering

Raw code comparison is **not** a numerical order for signed formats: negative values occupy the upper half with `C(x) = H + C(|x|)`, so more-negative values have *larger* codes. The fix is a monotone key. For signed numerical codes, excluding NaN:

```
OrderKey(c) =
    Q − c,      H < c < Q       negative values, folded below H
    H,          c = 0           zero
    H + c,      0 < c < H       positive values, above H
```

with $V(c_1) < V(c_2)$ iff $\mathtt{OrderKey}(c_1) < \mathtt{OrderKey}(c_2)$.

The algebra deliberately says nothing about where NaN goes — "*exceptional codes must be handled according to their specified ordering or unordered semantics*". That is the draft's call, and P3109 §4.12.1 places the single NaN **below** −Inf. The package therefore reserves key `0` for NaN and shifts every datum key up by one, so:

```
order_key(NaN) = 0  <  order_key(−Inf)  <  …  <  order_key(+Inf)
```

Which is why `sort` on a `Binary` vector puts NaN **first**, not last as `Float64` sorting does. See Julia Compatibility for what that means for generic code.

Sign, magnitude, negation

For signed formats and $c \notin \{0, N\}$:

```
MagCode(c) = c mod H        code of |V(c)|
NegCode(c) = c xor H        code of −V(c)
```

Negation is an involution with **one exception**: zero. There is a single zero code (`c = 0`), and `0 xor H = H`, which is the NaN code `N` in a signed format — not a negative zero. P3109 has no negative zero, and this is where that shows up in the algebra.

Parameter recovery

The format's parameters are recoverable from its own code lattice, which is the basis of the round-trip validation the test suite performs: precision from the block structure, bias from where the normal range begins, signedness from whether the upper half mirrors the lower, and domain from whether `N−1` decodes to an infinity or to a finite datum.

```
S = [the upper half mirrors the lower]
D = [V(N−1) is infinite]
M = N − 1 − D
```

Checking your own derivation

The formulas above are worth re-deriving if you are implementing against the draft. To check a derivation against this package:

```
using SmallFloats
using SmallFloats: nan_code, posinf_code, expbias, MaxFiniteOf

T = Binary8p4se
K, P = bitwidth(T), precision(T)
S, D = issigned(T) ? 1 : 0, isextended(T) ? 1 : 0
Q, H, L, B = 1 << K, 1 << (K - 1), 1 << (P - 1), 1 << (K - P - S)
N = Q - 1 - S * (H - 1)

(N == Int(nan_code(T)),
 N - 1 == Int(posinf_code(T)),
 N - 1 - D == Int(codepoint(MaxFiniteOf(T))),
 B == expbias(T))
```

```
(true, true, true, true)
```

The exhaustive form of that check — every distinguished code of every one of the 504 formats, plus the decoding formula at every positive finite code point — is in Verification Sessions.

Continue

- Formats, Aliases & Representations — what a format is, and the type-vs-alias distinction.
- Encoding and Decoding — how the package walks this algebra at speed.
- Why `Binary16p11se` is not `Float16` — the bias difference, stated in these terms.
- Format Queries — the package's accessors for every symbol named here.

Part V

Reference

Chapter 25

Formats and Values

25.1 Formats and Value Queries

This page is the lookup contract for format names, representation types, and format queries. For the conceptual model, read Format Anatomy; for a worked introduction, use First Session.

Naming grammar

```
Binary K p P (s|u) (e|f)
       │   │  │     └─ extended (Inf) or finite domain
       │   │  └─────── signed or unsigned
       │   └────────── significand precision, including the implicit bit
       └──────────────  total bitwidth, 3 through 16
```

Field	Values	Meaning
K	3–16	total bitwidth
P	1–K	significand precision, including the implicit bit
s|u	signed / unsigned	sign field present or absent
e|f	extended / finite	domain includes ±Inf, or spends those code points on finite datums

There are 504 legal `(K,P,SGN,EXT)` combinations, each with a draft-named alias. 120 aliases at $K \leq 8$ are exported by `using SmallFloats`.

Example alias	K	P	Signed	Domain
`Binary8p4se`	8	4	signed	extended
`Binary8p4sf`	8	4	signed	finite
`Binary6p3ue`	6	3	unsigned	extended
`Binary5p5uf`	5	5	unsigned	finite
`Binary16p6se`	16	6	signed	extended (opt-in export)
`Binary16p1uf`	16	1	unsigned	finite

`<:` versus `===`

- `Binary` is abstract; `Binary8p4se` is its concrete representation.
- `Binary8p4se <: Binary{8,4,true,true}` is `true`.
- `Binary8p4se === Binary{8,4,true,true}` is `false`. Use the alias, or `format(K, P, Σ, Δ)` for runtime parameters. See Values, Code Points, and Conversion.

`format()` and opt-in access

Form	Scope	Result
`using SmallFloats`	exports 120 aliases, $K \leq 8$	alias directly in scope
`using SmallFloats.Formats`	brings all 504 aliases into scope	alias directly in scope
`SmallFloats.Binary16p6se`	unexported, always reachable	alias by qualified path
`format(K, P, Σ, Δ)`	programmatic, runtime parameters	the alias for those parameters
`formatname(Binary{K,P,Σ,Δ})`	programmatic	`Symbol` of the alias name

Format query table

Every query accepts either a Type or a value; `bitwidth(x) ≡ bitwidth(typeof(x))`, and both fold to a literal constant.

Query	Returns	Draft-named form	Related guide
`bitwidth(T)`	K	`BitwidthOf(T)`	Cheat Sheet
`precision(T)`	P	`PrecisionOf(T)`	Cheat Sheet
`issigned(T)`	`Bool`	`SignednessOf(T)`	Cheat Sheet
`isextended(T)`	`Bool`	`DomainOf(T)`	Cheat Sheet
`expbias(T)`	exponent bias	`ExponentBiasOf(T)`	Cheat Sheet
`expbitwidth(T)`	exponent field width	—	Cheat Sheet
`trailingsigbits(T)`	trailing significand bits	—	Cheat Sheet
`MaxFiniteOf(T)`	largest finite value, typed	—	Cheat Sheet
`MinFiniteOf(T)`	smallest-magnitude finite value, typed	—	Cheat Sheet
`MinPositiveOf(T)`	smallest positive value, typed	—	Cheat Sheet
`MinNormalOf(T)`	smallest positive normal value, typed	—	Cheat Sheet
`MaxSubnormalOf(T)`	largest subnormal value, typed	—	Cheat Sheet

Draft-named forms exist for every query in Group M, not only the four listed above (`BitwidthOf`, `PrecisionOf`, `SignednessOf`, `DomainOf`, `ExponentBiasOf`, and their neighbors); Julia-style and draft-named forms are interchangeable.

Value-and-type call forms

```
bitwidth(Binary8p4se)        # type form
x = Binary8p4se(1.6)
bitwidth(x)                  # value form, same answer, folds to 8
MaxFiniteOf(Binary8p4se)     # Binary8p4se(224.0 ≡ 0x7e)
MaxFiniteOf(x)               # same result
```

Related contracts

Core Model, Values, Code Points, and Conversion, Choose a Format.

Chapter 26

Projection Specifications

26.1 Projection Specifications

This page is the lookup contract for `ProjSpec`, the predefined deterministic grid, stochastic constructor families, and projection accessors. For the conceptual model, read Rounding and Saturation.

`ProjSpec` constructor

```
ProjSpec(rounding_mode, saturation_mode)
```

A `ProjSpec` is the pair `(rounding mode, saturation mode)`. Both components are zero-size singleton types; a `ProjSpec` costs nothing at runtime and fully specializes every call that reaches it.

Deterministic predefined specs (6 × 3)

Named R<mode>_Sat<mode>. Every cell below is an exported constant.

Rounding	SatFinite	SatPropagate	SatNone	Related guide
NearestTiesToEven	RNE_SF	RNE_SP	RNE_SN	Rounding and Saturation
NearestTiesToAway	RNA_SF	RNA_SP	RNA_SN	Rounding and Saturation
TowardPositive	RTP_SF	RTP_SP	RTP_SN	Rounding and Saturation
TowardNegative	RTN_SF	RTN_SP	RTN_SN	Rounding and Saturation
TowardZero	RTZ_SF	RTZ_SP	RTZ_SN	Rounding and Saturation
ToOdd	RTO_SF	RTO_SP	RTO_SN	Rounding and Saturation

RNE_SN is the package-wide initial default (DefaultProjection()).

Stochastic families

Constructors, not constants — the random-bit budget N is a type parameter. N ∈ 1:60; omitting N uses the default N = 8 (SmallFloats.DEFAULT_RBITS).

Variant	SatFinite	SatPropagate	SatNone	Related guide
StochasticA	RSA_SF(N)	RSA_SP(N)	RSA_SN(N)	Rounding and Saturation
StochasticB	RSB_SF(N)	RSB_SP(N)	RSB_SN(N)	Rounding and Saturation
StochasticC	RSC_SF(N)	RSC_SP(N)	RSC_SN(N)	Rounding and Saturation

```
RSA_SN()      # StochasticA, N = 8, SatNone — the default form
RSA_SN(16)    # StochasticA, N = 16, SatNone
RSC_SF(16) === ProjSpec(StochasticC{16}(), SatFinite())   # true
```

Accessors

Accessor	Returns	Related guide
roundingmode(ρ)	the rounding-mode instance	Rounding and Saturation
saturationmode(ρ)	the saturation-mode instance	Rounding and Saturation
isstochastic(ρ)	Bool	Rounding and Saturation
nrandbits(ρ)	N for a stochastic spec	Rounding and Saturation

Saturation-mode meaning

Mode	Out-of-range behavior
SatFinite	clamp everything to the finite range
SatPropagate	preserve representable infinities; clamp other overflow
SatNone	apply the draft's domain-, signedness-, and direction-dependent rows (nearest modes overflow to ±Inf or NaN for finite formats; directed modes pointing away from the overflow clamp to the extremal finite value)

Explicit R range rule

For an N-bit stochastic mode, an explicit draw R passed as a keyword must satisfy 0 ≤ R ≤ 2^N - 1. Passing R outside that range is an error.

```
Add(T, σ, x, y; rng = Xoshiro(1))   # reproducible stream
Add(T, σ, x, y; R = 17)             # exact draw, ideal for tests
```

Related contracts

Rounding and Saturation, Session Defaults.

Chapter 27

Operation Catalog

27.1 Operation Catalog

This is the canonical catalog of 52 operations: 30 unary, 18 binary, 3 ternary, and `Convert`. The tables distinguish P3109 operation names from Julia Base spellings. Use the named form when result format and projection must be explicit; use Julia Compatibility Register for the complete Base mapping.

Unary (30)

Operation	Arity	Base spelling	Notes	Related guide
`Abs`	unary	`abs`		Cheat Sheet
`Negate`	unary	`unary -`		Cheat Sheet
`Sqrt`	unary	`sqrt`		Cheat Sheet
`RSqrt`	unary	—	no Base counterpart; call by draft name	Julia Compatibility Register
`Recip`	unary	`inv`	`Recip(±Inf) = 0`	Cheat Sheet
`Exp`	unary	`exp`		Cheat Sheet
`Exp2`	unary	`exp2`		Cheat Sheet
`ExpMinusOne`	unary	`expm1`		Cheat Sheet
`Log`	unary	`log`		Cheat Sheet
`Log2`	unary	`log2`		Cheat Sheet
`LogOnePlus`	unary	`log1p`		Cheat Sheet
`Softplus`	unary	—	no Base counterpart; call by draft name	Julia Compatibility Register
`Sin`	unary	`sin`		Cheat Sheet
`Cos`	unary	`cos`		Cheat Sheet
`Tan`	unary	`tan`		Cheat Sheet
`ArcSin`	unary	`asin`		Cheat Sheet
`ArcCos`	unary	`acos`		Cheat Sheet
`ArcTan`	unary	`atan`	single-argument form; see `ArcTan2` for two-argument	Cheat Sheet
`Sinh`	unary	`sinh`		Cheat Sheet
`Cosh`	unary	`cosh`		Cheat Sheet
`Tanh`	unary	`tanh`		Cheat Sheet
`ArcSinh`	unary	`asinh`		Julia Compatibility Register
`ArcCosh`	unary	`acosh`		Julia Compatibility Register
`ArcTanh`	unary	`atanh`		Julia Compatibility Register
`SinPi`	unary	`sinpi`	reduces exactly mod 2 first	Cheat Sheet
`CosPi`	unary	`cospi`	reduces exactly mod 2 first	Cheat Sheet
`TanPi`	unary	`tanpi`	reduces exactly mod 2 first	Cheat Sheet
`ArcSinPi`	unary	—	no Base counterpart; call by draft name	Julia Compatibility Register
`ArcCosPi`	unary	—	no Base counterpart; call by draft name	Julia Compatibility Register
`ArcTanPi`	unary	—	no Base counterpart; call by draft name	Julia Compatibility Register

Trig of ±Inf is NaN for all rows above.

Operation	Arity	Base spelling	Notes	Related guide
`CopySign`	binary	`copysign`		Julia Compatibility Register
`Add`	binary	`+`		Cheat Sheet
`Subtract`	binary	`-`		Cheat Sheet
`Multiply`	binary	`*`		Cheat Sheet
`Divide`	binary	`/`	`x / 0` → NaN for every x, including ±Inf	Cheat Sheet
`Hypot`	binary	`hypot`		Julia Compatibility Register
`ArcTan2`	binary	`atan(y, x)`	Base's `(y, x)` argument order	Cheat Sheet
`ArcTan2Pi`	binary	—	no Base counterpart; call by draft name	Julia Compatibility Register
`Maximum`	binary	`max`	NaN-propagating, exactly Base's float semantics	Cheat Sheet
`Minimum`	binary	`min`	NaN-propagating, exactly Base's float semantics	Cheat Sheet
`MaximumNumber`	binary	—	NaN-ignoring extremum; no Base counterpart	Julia Compatibility Register
`MinimumNumber`	binary	—	NaN-ignoring extremum; no Base counterpart	Julia Compatibility Register
`MaximumMagnitude`	binary	—	magnitude extremum; no Base counterpart	Julia Compatibility Register
`MinimumMagnitude`	binary	—	magnitude extremum; no Base counterpart	Julia Compatibility Register
`MaximumMagnitudeNumber`	binary	—	NaN-ignoring magnitude extremum; no Base counterpart	Julia Compatibility Register
`MinimumMagnitudeNumber`	binary	—	NaN-ignoring magnitude extremum; no Base counterpart	Julia Compatibility Register
`MinimumFinite`	binary	—	finite-preferring extremum; no Base counterpart	Julia Compatibility Register
`MaximumFinite`	binary	—	finite-preferring extremum; no Base counterpart	Julia Compatibility Register

Binary (18)

`0 · ∞` → NaN. A NaN operand generally propagates, except the `*Number`/`*Finite` extremum variants, which prefer a non-NaN (and, for the `Finite` variants, finite) operand.

Ternary (3)

Operation	Arity	Base spelling	Notes	Related guide
`FMA`	ternary	`fma`, `muladd`	one rounding for both Base spellings	Cheat Sheet
`FAA`	ternary	—	fused add-add; no Base counterpart	Julia Compatibility Register
`Clamp`	ternary	`clamp`		Cheat Sheet

Conversion

Operation	Arity	Base spelling	Notes	Related guide
`Convert`	conversion	—	the one operation accepting non-`Binary` operands: `Binary` (any format), `Float16/32/64`, `Float128`, `Integer`, `BigFloat`	Values, Code Points, and Conversion

Catalog total: 30 unary + 18 binary + 3 ternary + 1 conversion = 52.

Base spellings that are componentwise composites

These Base functions are not single draft operations; each is a componentwise tuple of draft operations run under the session default.

Base spelling	Composite of
`sincos`	`(Sin(x), Cos(x))`
`sincospi`	`(SinPi(x), CosPi(x))`
`minmax`	`(Minimum(x, y), Maximum(x, y))`

Deliberately unmapped draft operations

No Base spelling exists for these; call them by draft name. Base has no counterpart for any row in this list.

Draft operation	Reason unmapped
`RSqrt`	Base has `inv(sqrt(x))` as two roundings, not one; no single-rounding Base spelling
`Softplus`	no Base equivalent
`ArcSinPi`	Base has only the `sinpi` family (no inverse)
`ArcCosPi`	Base has only the `sinpi` family (no inverse)
`ArcTanPi`	Base has only the `sinpi` family (no inverse)
`ArcTan2Pi`	Base has only the `sinpi` family (no inverse)
`MaximumNumber`, `MinimumNumber`	Base has no NaN-ignoring extremum pair
`MaximumMagnitude`, `MinimumMagnitude`	Base has no magnitude-extremum pair
`MaximumMagnitudeNumber`, `MinimumMagnitudeNumber`	Base has no NaN-ignoring magnitude-extremum pair
`MinimumFinite`, `MaximumFinite`	Base has no finite-preferring extremum pair
`FAA`	Base has no fused add-add

The mapping is a declarative partition over the full op list; the test suite asserts it is exhaustive — every operation is either mapped to a Base spelling or listed in `_NO_BASE_COUNTERPART`.

Call shapes

Explicit form, every operation:

```
Op(result_format, ρ, operands...; rng, R)
```

`rng` and R apply only to stochastic ρ.

Convenience form, same-format operands, `DefaultProjection()`:

```
Op(x...)
```

Related contracts

Cheat Sheet, Why No Cross-Format Promotion, Values, Code Points, and Conversion.

Chapter 28

Arrays, Blocks, and Packed Storage

28.1 Arrays, Blocks, and Packed Storage

This page collects signatures and contracts for bulk operations, block-scaled values, and packed storage. For procedures, use Run Operations over Arrays, Use Blocks for Dynamic Range, or Pack Values for Storage.

Array operation forms

Form	Signature	Related guide
Unary array	`Op(fr, ρ, A)`	Cheat Sheet
Binary array	`Op(fr, ρ, A, B)`	Cheat Sheet
Ternary array	`Op(fr, ρ, A, B, C)`	Cheat Sheet
Generic map	`vmap(:Op, fr, ρ, operands...)`	Cheat Sheet
Generic map, in-place	`vmap!(C, Val(:Op), fr, ρ, operands...)`	Cheat Sheet
Callable operation object	`Op(:Op, fr, ρ)` then `.(A)` or `map(Op(:Op, fr, ρ), A, B)`	Run Operations over Arrays

Operands and destination must have matching axes.

Table cache introspection

Function	Signature	Notes	Related guide
`table_bytes`	`table_bytes()`	current cache footprint, bytes	Cheat Sheet
`empty_tables!`	`empty_tables!()`	reset the table cache	Cheat Sheet

Deterministic unary/binary array calls use cached result tables (256 B unary, 64 KiB binary). Ternary tables tier by combined operand bitwidth: eager (≤ 18 bits total), adaptive with LRU eviction (≤ 21 bits total), compute per element at $K = 8$. Stochastic operations always compute per element and consume one draw per projection.

Block functions

Function	Signature	Related guide
`blocksize`	`blocksize(b)`	Cheat Sheet
`scaleformat`	`scaleformat(b)`	Cheat Sheet
`elemformat`	`elemformat(b)`	Cheat Sheet
`ConvertFromBlock`	`ConvertFromBlock(fr, ρ, b)`	Cheat Sheet
`ConvertToBlock`	`ConvertToBlock(fs, fr, ρ, values_tuple, scale)`	Cheat Sheet
`ConvertToBlockMaxAbsFinite`	`ConvertToBlockMaxAbsFinite(fs, fr, scale_ρ, element_ρ, values_tuple)`	Cheat Sheet
`BlockReduceAdd`	`BlockReduceAdd(fr, ρ, b)`	Cheat Sheet
`BlockReduceMultiply`	`BlockReduceMultiply(fr, ρ, b)`	Cheat Sheet
`BlockDotProduct`	`BlockDotProduct(fr, ρ, bx, by)`	Cheat Sheet

`BlockReduceAdd`, `BlockReduceMultiply`, and `BlockDotProduct` perform their lane products and accumulation exactly, projecting once at the end — no hidden intermediate rounding.

Generated `BlockOp` / `ScaledOp` pattern

Every scalar operation except `Convert` has a generated block form and a generated scaled form.

Pattern	Signature
`BlockOp`	`BlockOp(fr, ρ, b1, b2, result_scale)` (arity follows the underlying `Op`)
`ScaledOp`	`ScaledOp(fr, ρ, scale1, x1, scale2, x2, ...)` (block size 1; arity follows the underlying `Op`)

Examples:

```
BlockAdd(T, ρ, b1, b2, result_scale)
BlockExp(T, ρ, b, result_scale)

ScaledAdd(T, ρ, scale1, x1, scale2, x2)
ScaledExp(T, ρ, scale, x)
```

`BlockVector`

Stores many equal-shape blocks in a structure-of-arrays layout. No additional scalar API beyond the `Block` operations above; construct from a collection of same-shape `Blocks`.

`PackedVector`

Operation	Signature	Notes
Construct	`PackedVector(v)`	`v::AbstractVector{F}`
Index (read)	`pv[i]`	returns an `F`
Index (write)	`pv[i] = x`	`x::F`
Collect	`collect(pv)`	materializes a `Vector{F}`
`sizeof` rule	`sizeof(pv.data)`	`ceil(length(pv) * bitwidth(F) / 8)` bytes, rounded to the storage word
`vmap` acceptance	`vmap(:0p, F, ρ, pv)`	accepted directly; tiles unpack internally

`PackedVector` stores each code point in exactly `bitwidth(F)` bits. Computation unpacks tiles internally; packed arithmetic is deliberately not in-place — store packed, compute unpacked.

Related contracts

Cheat Sheet, Run Operations over Arrays, Use Blocks to Track Dynamic Range.

Chapter 29

Defaults, Randomness, and Display

29.1 Defaults, Randomness, and Display

This page collects the contracts for process defaults, random sources, and display policy. For their scope and concurrency consequences, read Session State and Reproducibility.

Getter / setter table

Read with `DefaultX()`, set with `DefaultX!(v)`.

Default	Initial value	Setter accepts	Related guide
`DefaultType`	`Binary8p2se`	any fully-parameterized `Binary` type	Cheat Sheet
`DefaultReturnType`	`Binary8p2se`	any fully-parameterized `Binary` type	Cheat Sheet
`DefaultRoundingMode`	`NearestTiesToEven()`	mode instance or type	Cheat Sheet
`DefaultSaturationMode`	`SatNone()`	mode instance or type	Cheat Sheet
`DefaultProjection`	`RNE_SN`	a `ProjSpec`, or `(mode, sat)`	Cheat Sheet
`DefaultRNG`	the `Xoshiro` type	RNG type or instance	Cheat Sheet
`DefaultRbits`	`8`	`Int` in `1:60`	Cheat Sheet

```
DefaultType()             # Binary8p2se     DefaultType!(Binary8p4se)
DefaultReturnType()       # Binary8p2se     DefaultReturnType!(Binary8p3se)
DefaultRoundingMode()     # NearestTiesToEven()
DefaultSaturationMode()   # SatNone()
DefaultProjection()       # RNE_SN
DefaultRNG()              # Xoshiro         DefaultRNG!(Xoshiro(42))
DefaultRbits()            # 8               DefaultRbits!(16)
```

Coherence invariant

```
DefaultProjection() === ProjSpec(DefaultRoundingMode(), DefaultSaturationMode())
```

This holds at all times, in both directions:

Action	Effect
`DefaultRoundingMode!(mode)`	rebuilds `DefaultProjection` around the new mode, holding saturation fixed
`DefaultSaturationMode!(mode)`	rebuilds `DefaultProjection` around the new mode, holding rounding fixed
`DefaultProjection!(ρ)`	decomposes ρ back into both components

There is no `DefaultAccumulatorType`: removed after v0.1.0. Reduction accumulator carriers are picked by the span filter (provably exact for the operands), not by a session preference.

`with_default_*` combinators

Combinator	Signature	Related guide
`with_default_type`	`with_default_type(f, args...)` calls `f(DefaultType(), args...)`	Cheat Sheet
`with_default_returntype`	`with_default_returntype(f, args...)` calls `f(DefaultReturnType(), args...)`	Cheat Sheet
`with_default_projection`	`with_default_projection(f, args...)` calls `f(DefaultProjection(), args...)`	Cheat Sheet

```
with_default_type((T, x) -> T(x), 1.5)          # Binary8p2se(1.5 ≡ 0x41)
with_default_projection((ρ, x, y) -> Add(T, ρ, x, y), x, y)
with_default_returntype(f, args...)              # f(DefaultReturnType(), args...)
```

Each combinator restores nothing itself — it only *reads* the default at call time via `f(default, args...)`. For scoped mutate-and-restore, the same names also support a do-block form that restores the prior value on exit, even if the body throws.

Cost model

- **Speculation guard.** Convenience forms (`x + y`, `Exp(x)`, `T(2.1)`) and the `with_default_*` combinators read the default through a speculation guard: allocation-free and concretely inferred while the default holds its initial value.
- **One barrier dispatch after a change.** Once a default is changed, each call crosses a function barrier: one dynamic dispatch at entry, everything inside still fully specialized.
- **Explicit forms are unaffected.** `Add(T, ρ, x, y)` with a `const` or argument-bound ρ never consults a session default and never pays this cost either way.
- **Boxing at escape.** When the combinator's result type *is* the default itself (`with_default_type` used as a constructor), the value computes on the specialized path but boxes once where it escapes the call — the irreducible cost of a runtime-chosen type. When the result type does not depend on the default, the call is zero-allocation with a concretely inferred result.

Randomness

Form	Entropy source	Projection control
`rand(T)`	task-local RNG	default uniform projection
`rand(rng, T)`	explicit `rng`	default uniform projection
`rand(rng, T; projection = ρ)`	explicit `rng`	explicit `ρ`
`randn(rng, T)`	explicit `rng`	default normal projection
stochastic operation with `rng = rng`	explicit stream	selected stochastic `ProjSpec`
stochastic operation with `R = r`	exact raw draw	selected stochastic `ProjSpec`

An explicit `R` must lie in `0:(2^N - 1)` for an `N`-bit stochastic mode. Array `rand`/`randn` forms use their documented defaults; construct arrays from scalar calls when each element must use another explicit projection.

Display

`get_show_style()` returns the process-wide style and `set_show_style!(style)` sets it. `VALID_SHOW_STYLES` contains four values:

Style	Example rendering
`:value`	`1.625`
`:codepoint`	`0x45`
`:datum`	`(1.625 ≡ 0x45)`
`:typed`	`Binary8p4se(1.625 ≡ 0x45)`

An `IOContext` property, `:binary_show_style`, overrides the process default for one stream. Prefer that form in concurrent code and in libraries that should not change another caller's display.

Related contracts

First Session, Julia Numeric Behavior, Projection Specifications.

Chapter 30

Julia Compatibility

30.1 Julia Compatibility Register

`Binary` values speak ordinary Julia. The **Base register** (`src/juliacompat.jl`) maps every draft operation that has a Base counterpart onto its Base name, so generic Julia code — and your fingers — can use the spellings they already know. Every method there is exactly one same-format spec-register call under the session default projection; there is no third semantics: `x + y` *is* `Add(x, y)`, which *is* `Add(T, DefaultProjection(), x, y)`.

Using the overloads

```
julia> x, y, z = Binary8p4se(1.6), Binary8p4se(0.25), Binary8p4se(2.0);

julia> x + y                        # Add
Binary8p4se(1.875 ≡ 0x47)

julia> exp(y)                       # Exp — a 256-byte table lookup once warm
Binary8p4se(1.25 ≡ 0x42)

julia> atan(y, x)                   # ArcTan2, in Base's (y, x) argument order
Binary8p4se(0.15625 ≡ 0x2a)

julia> fma(x, y, z)                 # FMA — one rounding; muladd is the same op
Binary8p4se(2.5 ≡ 0x4a)

julia> sincos(y)                    # composite: (Sin(y), Cos(y)) componentwise
(Binary8p4se(0.25 ≡ 0x30), Binary8p4se(1.0 ≡ 0x40))
```

Because the veneers follow the session default projection, changing it changes them (see Session Defaults):

```
julia> Binary8p4se(200.0) * Binary8p4se(2.0)     # RNE_SN: overflow → Inf
Binary8p4se(Inf ≡ 0x7f)

julia> DefaultProjection!(RTZ_SF);

julia> Binary8p4se(200.0) * Binary8p4se(2.0)     # now clamps to MaxFinite
Binary8p4se(224.0 ≡ 0x7e)

julia> DefaultProjection!(RNE_SN)                # restore: this setter is PROCESS-wide
(NearestTiesToEven, SatNone)
```

Code that must be insensitive to the session default names its projection: `Multiply(T, RNE_SN, x, y)`.

Restore what you set

`DefaultProjection!` mutates state shared by every module in the process, and nothing scopes it for you — it changes `T(x::Real)` construction too, so a later `Binary8p4se(1.1)` would truncate rather than round to nearest. Prefer `with_default_projection`, which restores on exit even if the body throws. Every example after this point assumes the pristine `(NearestTiesToEven, SatNone)`, which is why the demonstration above puts it back.

The mapping

Base spelling	draft operation
`+ - * /`, unary `-`	`Add`, `Subtract`, `Multiply`, `Divide`, `Negate`
`abs`, `inv`, `sqrt`	`Abs`, `Recip`, `Sqrt`
`exp`, `exp2`, `expm1`, `log`, `log2`, `log1p`	`Exp`, `Exp2`, `ExpMinusOne`, `Log`, `Log2`, `LogOnePlus`
`sin`, `cos`, `tan`, `asin`, `acos`, `atan`	`Sin`, `Cos`, `Tan`, `ArcSin`, `ArcCos`, `ArcTan`
`sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`	`Sinh`, `Cosh`, `Tanh`, `ArcSinh`, `ArcCosh`, `ArcTanh`
`sinpi`, `cospi`, `tanpi`	`SinPi`, `CosPi`, `TanPi`
`atan(y, x)`	`ArcTan2` (Base's argument order)
`copysign`, `hypot`	`CopySign`, `Hypot`
`min`, `max`	`Minimum`, `Maximum` (NaN-propagating, exactly Base's float semantics)
`fma`, `muladd`	`FMA` (both: one rounding)
`clamp`	`Clamp`
`sincos`, `sincospi`, `minmax`	componentwise composites of the draft ops

The mapping is a declarative partition over the op lists, and the test suite asserts it is exhaustive: every operation is either mapped above or listed in `_NO_BASE_COUNTERPART`. Adding an operation to the registry forces an explicit decision here.

What is *not* mapped, and why

Draft operations with no Base spelling — call them by their draft names: `RSqrt`, `Softplus`, `ArcSinPi`/`ArcCosPi`/`ArcTanPi`/`ArcTan2Pi` (Base has only the `sinpi` family), the NaN-ignoring/magnitude/finite extremum families (`MinimumNumber`, `MaximumMagnitude`, `MinimumFinite`, ...; Base has no NaN-ignoring pair), and `FAA` (no Base fused add-add).

```
julia> min(Binary8p4se(NaN), x)          # Base semantics: NaN propagates
Binary8p4se(NaN ≡ 0x80)

julia> MinimumNumber(Binary8p4se(NaN), x)   # the NaN-ignoring draft op
Binary8p4se(1.625 ≡ 0x45)
```

Mixed `Binary` formats — deliberately a `promotion ... failed to` change error, never a silent widening. Mixing formats is an explicit `Convert`:

```
julia> x + Convert(Binary8p4se, RNE_SN, Binary5p3sf(1.0))
Binary8p4se(2.5 ≡ 0x4a)
```

`Binary` with ordinary numbers — promotes to the format's **promotion carrier** through the rules in `formats.jl`, so the result is an ordinary float, not a re-projected `Binary`.

For the 432 formats at rung 1 that carrier is `Float64`, which is what the example below shows. The 64 rung-2 formats promote to `Float128`; the 8 rung-3 formats promote to `BigFloat`. `Float64` cannot hold a `Binary16p1uf` datum, and promoting into it would return `±Inf` from a perfectly finite value with no P3109 operation involved. `promotecarrier(T)` names the target.

```
julia> x + 2.0
3.625
```

Project it back explicitly when that is what you mean: `Convert(Binary8p4se, RNE_SN, decode(x) + 2.0)`.

The rest of the Base surface

Beyond arithmetic, `Binary` integrates where the other layers provide it: predicates and constants (`isnan`, `isfinite`, `signbit`, `zero`, `one`, `eps`, `typemin`/`typemax`, `floatmin`/`floatmax`), comparisons and `isless` on integer order keys, `sort` via an O(n) counting sort (which places the single NaN **first**, below −Inf, per the draft's total order — not last as `Float64` sorting does), `nextfloat`/`prevfloat` as the draft Next operations, `codepoint`, and `rand`/`randn` (see Random Values). All of it goes through the same projection engine and decode tables as the draft-named API.

The `AbstractFloat` contract

`Binary <: AbstractFloat`, and generic Julia code reaches for that interface without asking. The routinely used numeric contracts are implemented, while operations with no P3109 meaning refuse explicitly:

verb	behaviour
`hash`, `Base.decompose`	exact — a datum is `S · 2^Q`, so `Dict` keys and `Set` elements work
`round`, `floor`, `ceil`, `trunc`	**one** rounding: the integer value is formed exactly on the carrier and projected once, under the session default's saturation
`round(x, r::RoundingMode)`	`RoundNearest`, `RoundNearestTiesAway`, `RoundUp`, `RoundDown`, `RoundToZero`
`exponent`, `significand`, `frexp`	exact; `DomainError` on zero and the non-finites, as in Base
`widen`	the format's promotion carrier — see No Automatic Promotion for why it is not another format
`reinterpret`	both directions, and the incoming one is **checked** against the representation invariant

```
julia> hash(Binary8p4se(1.5)) == hash(Binary8p4se(1.5))
true

julia> Base.decompose(Binary8p4se(1.5))          # 12 · 2^-3 = 1.5, exactly
(12, -3, 1)

julia> floor(Binary8p4se(1.6)), ceil(Binary8p4se(1.1))
(Binary8p4se(1.0 ≡ 0x40), Binary8p4se(2.0 ≡ 0x48))

julia> reinterpret(Binary8p4se, 0x44)            # checked, unlike `rawvalue`
Binary8p4se(1.5 ≡ 0x44)
```

Refusals name themselves. Where a Base verb has no draft counterpart, the error says which and why rather than surfacing as a bare `MethodError` that cannot be told apart from an oversight:

```
julia> rem(Binary8p4se(1.5), Binary8p4se(2.0))
ERROR: ArgumentError: rem is not defined for Binary8p4se: the draft defines no remainder; …
```

That covers `rem`/`mod`, `round` under `RoundFromZero` and `RoundNearestTiesUp` (no draft μ), and `significand` on the one format whose binade `[1, 2)` is truncated by its `Inf` encoding.

Chapter 31

Conformance and Approximation

31.1 Conformance and Approximation Registry

This page is the contract for exporting the live conformance declaration and for measuring and registering optional approximate implementations. The default operation API never substitutes a registered approximation.

Conformance functions

Function	Signature	Returns	Related guide
`conformance`	`conformance()`	the live declaration: formats, operations, mode vocabulary, table specializations instantiated this session, registered approximations	Read and Export Conformance
`conformance_dict`	`conformance_dict()`	a plain nested `Dict`, for JSON/TOML serialization	Read and Export Conformance
`conformance_report`	`conformance_report()`	prints the declaration	Read and Export Conformance
`draft_revision`	`draft_revision()`	the P3109 draft revision this package conforms to	Read and Export Conformance

κ registry

Function	Signature	Returns	Related guide
`measure_kappa`	`measure_kappa(fn, op, fr, (argformats...), ρ)`	`(κ, exhaustive)` — measured max code-point deviation and whether the measurement was exhaustive	Register an Approximation
`register_approx!`	`register_approx!(name, op, fr, args, ρ, fn; κ)`	registers `fn` as approximation `name` for `op`, declaring bound `κ`	Register an Approximation
`approx`	`approx(name)`	the registered implementation object	Register an Approximation
`kappa`	`kappa(impl)`	the declared `κ` bound	Register an Approximation
`kappa_measured`	`kappa_measured(impl)`	the measured `κ` from registration	Register an Approximation
`list_approx`	`list_approx()`	all registered approximation names	Register an Approximation
`unregister_approx!`	`unregister_approx!(name)`	removes a registered approximation	Register an Approximation

```
κ, exhaustive = measure_kappa(fn, :Exp, T, (T,), ρ)
register_approx!(:fast_exp, :Exp, T, (T,), ρ, fn; κ)
impl = approx(:fast_exp)
kappa(impl)
kappa_measured(impl)
list_approx()
unregister_approx!(:fast_exp)
```

Understatement-rejection rule

`κ` is measured exhaustively at registration time whenever the domain permits exhaustive enumeration. Declaring a `κ` smaller than the measured deviation throws — an implementation cannot register itself as more accurate than it measures.

κ = NaN acknowledgment rule

An implementation whose NaN behavior differs from the default API's (a NaN mismatch against the exhaustive reference) must be registered with an explicit `κ = NaN`. This is an acknowledgment, not a numeric bound: it cannot be produced by measurement alone and must be supplied by the caller of `register_approx!`.

Approximate implementations registered through this path are never substituted into the default API; the default API is bit-exact, always.

Related contracts

Read and Export Conformance, Register an Approximation, and Verification Strategy.

Chapter 32

Public API Index

32.1 Public API Index

This index renders source docstrings for the public surface. Begin with the curated family table: it states where each contract is maintained. The 120 format aliases at $K \le 8$ are exported by `using SmallFloats`; the other 384 are opt-in through `SmallFloats.Formats` or available by qualification.

The index below groups the exported surface by family and points each family at the descriptive reference page that documents its full table. The `@autodocs` block that follows renders the docstrings themselves, alphabetically, as generated from source.

Index

Family	Scope	Descriptive reference page
Arrays, blocks, packed storage	`Op(fr,ρ,A,...)`, `vmap`/`vmap!`, `Block`, `BlockVector`, `PackedVector`, `table_bytes`, `empty_tables!`	Arrays, Blocks, and Packed Storage
Conformance and approximation registry	`conformance`, `conformance_dict`, `conformance_report`, `draft_revision`, `measure_kappa`, `register_approx!`, `approx`, `kappa`, `kappa_measured`, `list_approx`, `unregister_approx!`	Conformance and Approximation Registry
Formats	504 aliases (120 exported at $K \leq 8$), `format`, `formatname`, and format-query accessors	Formats and Value Queries
Operations	30 unary, 18 binary, 3 ternary, and `Convert`	Operation Catalog
Projection specs	`ProjSpec`, deterministic grid, stochastic constructors, and accessors	Projection Specifications
Session state	defaults, RNG control, display styles, and scoped combinators	Defaults, Randomness, and Display
Julia integration	Base overloads, promotion carriers, sorting, and conversion behavior	Julia Compatibility Register

`SmallFloats.SmallFloats` – Module.

```
SmallFloats
```

A conforming, performance-oriented Julia implementation of the IEEE P3109 draft standard for arithmetic formats for machine learning (bitwidths 3–16).

Bit-exact defined results on every default path; the projection engine (`RoundToPrecision → Saturate → Encode`) is the single write path into a code point; approximate fast paths exist only behind the explicit κ registry (`register_approx! / approx`), never substituted silently. See `conformance()` for the live declaration and `SmallFloats.draft_revision()` for the implemented draft.

Performance note

Pass format types through `const` bindings, type parameters, or function arguments. All exported entry points fully specialize when the format type is statically known (measured: a complete scalar `Add` ≈ 26 ns, `project` ≈ 13 ns, zero allocations). Calling through a **non-`const` global** format type forces dynamic dispatch on every call — measured at ~1 µs per scalar keyword call — which is Julia semantics, not package overhead; a single function barrier (`f(::Type{T}, args...) where {T}`) restores full speed. The test suite pins the specialization properties (concrete return types, zero warm-path allocation) as deterministic regressions.

`SmallFloats.RNA_SF` – Constant.

(NearestTiesToAway, SatFinite).

`SmallFloats.RNA_SN` – Constant.

(NearestTiesToAway, SatNone).

`SmallFloats.RNA_SP` – Constant.

(NearestTiesToAway, SatPropagate).

`SmallFloats.RNE_SF` – Constant.

(NearestTiesToEven, SatFinite).

`SmallFloats.RNE_SN` – Constant.

(NearestTiesToEven, SatNone) — the package-wide default ρ.

`SmallFloats.RNE_SP` – Constant.

(NearestTiesToEven, SatPropagate).

`SmallFloats.RTN_SF` – Constant.

(TowardNegative, SatFinite).

`SmallFloats.RTN_SN` – Constant.

(TowardNegative, SatNone).

`SmallFloats.RTN_SP` – Constant.

(TowardNegative, SatPropagate).

`SmallFloats.RTO_SF` – Constant.

(ToOdd, SatFinite).

`SmallFloats.RTO_SN` – Constant.

(ToOdd, SatNone).

`SmallFloats.RTO_SP` – Constant.

(ToOdd, SatPropagate).

`SmallFloats.RTP_SF` – Constant.

(TowardPositive, SatFinite).

`SmallFloats.RTP_SN` – Constant.

(TowardPositive, SatNone).

`SmallFloats.RTP_SP` – Constant.

(TowardPositive, SatPropagate).

`SmallFloats.RTZ_SF` – Constant.

(TowardZero, SatFinite).

`SmallFloats.RTZ_SN` – Constant.

(TowardZero, SatNone).

`SmallFloats.RTZ_SP` – Constant.

(TowardZero, SatPropagate).

`SmallFloats.ApproxImpl` – Type.

A registered κ-approximate implementation of one operation specialization.

SmallFloats.Binary – Type.

```
Binary{K,P,SGN,EXT} <: AbstractFloat
```

A P3109 **format**: bitwidth `K ∈ 3:16`, precision `P`, signedness `SGN::Bool` (`true` = Signed), domain `EXT::Bool` (`true` = Extended, i.e. the datum set includes infinities).

`Binary` is **abstract**, and that is the draft's own distinction: a format is "a datum set and an encoding" (D1 §3.1), while the width of the machine word carrying a code point is a representation choice below the standard's level of description. `Code8` is that representation. The parameterized name is the *format*; the draft-named alias is its *representation*:

```
Binary{8,4,true,true}   # the format  — abstract
Binary8p4se             # Code8{8,4,true,true}, its representation — concrete
```

Consequently `Binary8p4se === Binary{8,4,true,true}` is **`false`** and `Binary8p4se <: Binary{8,4,true,true}` is **`true`**. Use the alias, or `format(K, P, Σ, Δ)`, wherever a concrete type is needed — an array element type, a `similar`, a constructor target.

The code point occupies the low `K` bits of the storage unit; the high bits are maintained zero as a representation invariant. Construct raw code points with `T(c::Unsigned)` (validated code-point construction), `rawvalue(T, c)` (unchecked kernel route), or `Code8{...}(Val(:code), x)`; construct from numeric values with `T(x::Real)` (default projection spec) or `Convert`. `Unsigned` is the one argument-type class meaning *code point*, at **every** width — `T(0x02)` is code point 2 whether `T`'s unit is a `UInt8` or a `UInt16`; all other `Real`s mean *value*.

There are `4K − 2` formats at each `K` (`P ∈ 1:K` × signed/unsigned × finite/extended, less the two `P == K` signed combinations, which have no exponent bits), so `KMIN:KMAX` = 504 in total.

SmallFloats.Block – Type.

```
Block{B, FS<:Binary, FE<:Binary}
```

A P3109 block `(s, [x₁ … x_B])`: scale factor in format `FS`, `B ≥ 1` elements in format `FE` (draft §5). `isbits`; blocks live in registers/stack.

SmallFloats.BlockVector – Type.

```
BlockVector{B,FS,FE} <: AbstractVector{Block{B,FS,FE}}
```

Structure-of-arrays storage: `scales::Vector{FS}`, `elems::Matrix{FE}` (`B × n`, column-major so each block's elements are one contiguous column).

SmallFloats.FPClass – Type.

Eight-way datum classification returned by `Class` (draft §4.13.1).

SmallFloats.NearestTiesToAway – Type.

Round to nearest; ties away from zero (draft §4.7.1).

SmallFloats.NearestTiesToEven – Type.

Round to nearest; ties to the even code point (IEEE default; draft §4.7.1).

`SmallFloats.Op` – Type.

```
Op(name::Symbol, FR::Type{<:Binary}, ρ::ProjSpec) -> Op
```

A registry operation bound to a result format and a projection, as a **callable** — so the idiomatic array verbs work directly:

```
map(Op(:Add, Binary8p4se, RNE_SN), A, B)
Op(:Exp, Binary8p4se, RNE_SN).(A)
```

`Op` is a singleton type: the operation, format and projection all live in the type parameters, so calls specialize exactly as `Add(Binary8p4se, RNE_SN, x, y)` does. Use `vmap!` when you have a destination array to fill — it is the in-place, table-aware kernel and this type is a thin front for it.

`SmallFloats.PackedVector` – Type.

```
PackedVector{F<:Binary} <: AbstractVector{F}
```

Bit-packed storage of `F` code points at `bitwidth(F)` bits per element. Construct with `PackedVector(A::AbstractVector{F})`; recover with `Vector(pv)` or `collect(pv)`. Memory: `⌈n·K/64⌉` words vs `n·sizeof(codeunit_type(F))` unpacked.

It saves nothing when `K == 8·sizeof(codeunit_type(F))` — K = 8 and K = 16 — where the packed layout is bit-for-bit the unpacked one and every access pays the shift/mask/splice for no return. `packing_saves(F)` says so; see its docstring for why that is a query rather than an error.

`SmallFloats.ProjSpec` – Type.

```
ProjSpec{R<:RoundingMode3109, S<:SaturationMode}
```

A projection specification ρ = (rounding mode, saturation mode), draft §4.2. Zero-size; construct as `ProjSpec(NearestTiesToEven(), SatNone())` or via the exported constants. Kernels specialize on its type.

`SmallFloats.StochasticA` – Type.

Stochastic rounding, variant A of draft §4.7.4: RoundAway $\Longleftrightarrow$ ⌊ν·2^N⌋ + R ≥ 2^N.

`SmallFloats.StochasticB` – Type.

Stochastic rounding, variant B: RoundAway $\Longleftrightarrow$ ⌊ν·2^(N+1)⌋ + (2R+1) ≥ 2^(N+1).

`SmallFloats.StochasticC` – Type.

Stochastic rounding, variant C: RoundAway $\Longleftrightarrow$ RNITE(ν·2^N) + R ≥ 2^N.

`SmallFloats.ToOdd` – Type.

Round inexact results to the nearest *odd* code point (draft §4.7.3; sticky-friendly).

`SmallFloats.TowardNegative` – Type.

Directed rounding toward −∞ (draft §4.7.2).

`SmallFloats.TowardPositive` – Type.

Directed rounding toward +∞ (draft §4.7.2).

`SmallFloats.TowardZero` – Type.

Directed rounding toward zero — truncation (draft §4.7.2).

`SmallFloats.BlockDotProduct` – Method.

BlockDotProduct (draft §5.3.2). Lane products can carry 64 significant bits, so the products and their sum are formed in the exact accumulator, with the ∞/NaN fold algebra resolved on the Float64 classifications first.

`SmallFloats.BlockReduceAdd` – Method.

BlockReduceAdd (draft §5.3.1): project(reduce(ωAdd, [0, X...])).

`SmallFloats.BlockReduceMultiply` – Method.

BlockReduceMultiply (draft §5.3.1): project(reduce(ωMultiply, [1, X...])).

`SmallFloats.Class` – Method.

Class(v) -> FPClass (draft §4.13.1): classify v as NaN, ±Inf, ±normal, ±subnormal, or zero.

`SmallFloats.Convert` – Method.

```
Convert(fr, ρ, A::AbstractArray{<:Union{Float16,Float32,Float64,BFloat16}}; rng) -> Array{fr}
```

Bulk ingestion: exact widening per element, then projection under ρ. Unlike the Binary-array Convert (a Shape-A table gather), external inputs are not enumerable, so this is a Shape-B loop; stochastic ρ draws per element.

`SmallFloats.Convert` – Method.

```
Convert(fr, ρ, x) -> fr
```

Draft §4.9 Convert(f*x*, *f*r, ρ). Accepts `Binary`, IEEE binary16/32/64 floats (widened exactly to the Float64 carrier), `Float128` (projected directly), `Integer` (exact via a sufficiently wide BigFloat), and `BigFloat` (projected directly; the caller warrants the value is exact).

`SmallFloats.ConvertFromBlock` – Method.

ConvertFromBlock (§5.2.1): blockdecode, then project each element (no scale division).

`SmallFloats.ConvertToBlock` – Method.

ConvertToBlock (§5.2.2): decode elements, BlockProject against the supplied scale.

`SmallFloats.ConvertToBlockMaxAbsFinite` – Method.

ConvertToBlockMaxAbsFinite (§5.2.3): S = reduce(ωMaximumFinite, [NaN, $|X_1|$...]), project S under ρs, then BlockProject the elements under ρ.

`SmallFloats.DefaultProjection` – Method.

```
DefaultProjection() -> ProjSpec
DefaultProjection!(ρ::ProjSpec) -> ρ
DefaultProjection!(m, s) -> ProjSpec
```

The session's default projection specification. Initialized to `(DefaultRoundingMode(), DefaultSaturationMode()) = RNE_SN`. Setting it directly decomposes ρ into `DefaultRoundingMode` and `DefaultSaturationMode`, so the three stay coherent in both directions.

`SmallFloats.DefaultRNG` – Method.

```
DefaultRNG() -> Type{<:AbstractRNG} | AbstractRNG
DefaultRNG!(rng) -> rng
```

The session's default random-number generator for stochastic rounding. Initialized to the `Xoshiro` *type* (a fresh generator per use); may be set to an `AbstractRNG` instance for a reproducible stream.

`SmallFloats.DefaultRbits` – Method.

```
DefaultRbits() -> Int
DefaultRbits!(n::Int) -> n
```

The session's default random-bit budget N for the stochastic rounding families (`StochasticA{N}`, `StochasticB{N}`, `StochasticC{N}`). Initialized to 8; must satisfy $1 \le N \le 60$.

`SmallFloats.DefaultReturnType` – Method.

```
DefaultReturnType() -> Type{<:Binary}
DefaultReturnType!(T::Type{<:Binary}) -> T
```

The session's default *result* format — the format an operation projects into when the caller does not name one. Initialized to `Binary8p2se`. Independent of `DefaultType`, which is the default *operand* format.

`SmallFloats.DefaultRoundingMode` – Method.

```
DefaultRoundingMode() -> RoundingMode3109
DefaultRoundingMode!(m) -> m
```

The session's default rounding mode. Initialized to `NearestTiesToEven()`. Setting it rebuilds `DefaultProjection` from the new mode and the current `DefaultSaturationMode`. Accepts an instance or a fully-parameterized type (`NearestTiesToAway`, `StochasticA{8}`, ...).

`SmallFloats.DefaultSaturationMode` – Method.

```
DefaultSaturationMode() -> SaturationMode
DefaultSaturationMode!(s) -> s
```

The session's default saturation mode. Initialized to `SatNone()`. Setting it rebuilds `DefaultProjection` from the current `DefaultRoundingMode` and the new mode. Accepts an instance or a type.

`SmallFloats.NextGreaterThan` – Method.

NextGreaterThan(v) (draft §4.16): the least datum greater than v in the total order — one step up the code lattice, with NaN → NaN and MaxFinite/+Inf → NaN at the top. `Base.nextfloat` on `Binary` is this operation.

`SmallFloats.NextLessThan` – Method.

NextLessThan(v) (draft §4.16): the greatest datum less than v in the total order — one step down the code lattice, with NaN → NaN and MinFinite/−Inf → NaN at the bottom. `Base.prevfloat` on `Binary` is this operation.

`SmallFloats.RoundOf` – Function.

Draft-named accessor: RoundOf(ρ) — the rounding mode of ρ (§4.2).

`SmallFloats.SatOf` – Function.

Draft-named accessor: SatOf(ρ) — the saturation mode of ρ (§4.2).

`SmallFloats.TotalOrder` – Method.

TotalOrder(fx,fy) (draft §4.12.1): x ≤ y in the total order (single NaN smallest — NaN ≼ −Inf ≼ … ≼ +Inf). Same-format comparisons run on the integer order key (bitops plan K1) — proven equivalent to the decode order exhaustively over all pairs; cross-format keys are not comparable, so mixed formats keep the decode path.

`SmallFloats.approx` – Method.

Retrieve a registered approximate implementation (callable via `.fn`).

`SmallFloats.bitwidth` – Method.

Format bitwidth K (3–8).

`SmallFloats.codedistance` – Method.

Code-point distance along the total order between two values of one format.

`SmallFloats.codeunit_type` – Method.

The storage unit of a format's code point: `UInt8` for K ≤ 8, `UInt16` above.

`SmallFloats.conformance` – Method.

```
conformance() -> ConformanceDeclaration
```

The package's draft-§4.6-style conformance declaration, derived live from the operation registry, the table cache (the specializations actually instantiated), and the approximate-implementation registry. Serialize with `conformance_dict` or render with `conformance_report`.

`SmallFloats.conformance_dict` – Function.

Nested `Dict{String,Any}` form of the declaration — serializable by any JSON/TOML writer.

`SmallFloats.conformance_report` – Function.

Human-readable conformance report.

`SmallFloats.decode!` – Method.

```
decode!(dest::AbstractArray{Float32}, A::AbstractArray{<:Binary}) -> dest
decode!(dest::AbstractArray{Float64}, A::AbstractArray{<:Binary}) -> dest
```

Exact bulk ωDecode into an external float array.

Exactness is the whole contract here — it is what distinguishes `decode!` from `Float32.(A)`, which rounds like any other Julia conversion — so the destination element type is **gated on the format**, not assumed. `datumsexact(X, F)` decides it, and a format whose datums do not all fit X raises rather than silently rounding a bulk array. Every K ≤ 8 format passes both gates, so this is a new refusal only for formats that did not exist before the K ≤ 16 extension.

Where the gate passes and K ≤ KSPLIT, the implementation is unchanged: a Shape-A gather of the (narrowed) constant decode table.

SmallFloats.decode – Method.

```
decode(v::Binary) -> datumcarrier(typeof(v))
```

ωDecode (draft §4.7.2). **Exact for every datum of every format**, which is why the return type is the format's carrier rather than a fixed one: Float64 where the bias allows (432 of the 504 formats), Float128 to B = 8192, BigFloat above. Float64(v) is the way to ask for a Float64 and accept the rounding.

Two implementations, selected by decodepolicy — dispatch on the representation, not a branch on K (invariant 9, §2 R-F):

- TableDecode (K ≤ KSPLIT): a 2^K-entry constant-tuple lookup (bitops plan K2), generated once from _decode_compute, so the two are correct by construction and asserted equivalent exhaustively; constant inputs fold.
- ComputeDecode (K > KSPLIT): _decode_compute directly. No table — 65 536 entries is what invariant 10 exists to forbid.

SmallFloats.empty_tables! – Method.

Drop every cached table and adaptive counter (they rebuild lazily on next use).

SmallFloats.expbias – Method.

Exponent bias: 2^(K−P−1) signed, 2^(K−P) unsigned.

SmallFloats.expbitwidth – Method.

Width of the exponent field in bits: (K − signbit) − (P − 1).

SmallFloats.f32_exact – Method.

```
f32_exact(op, f1, f2) -> Bool
```

True iff op ∈ (:Add, :Subtract, :Multiply) on Float32-decoded operands is exact for every finite pair of (f1, f2) datums — checked once by exhaustive enumeration against a 300-bit BigFloat oracle, then cached. When true, a Float32 intermediate is an exact carrier under every projection mode.

SmallFloats.f32_impl – Method.

```
f32_impl(op, fr, ρ) -> fn
```

The decode32 → guarded-op32 → project kernel for op(… → fr, ρ), shaped for register_approx!. Nearest-ties ρ only: directed modes after a nearest-rounded Float32 compute measure κ ≥ 1 (and κ = NaN at saturation boundaries), and stochastic ρ is rejected by κ measurement itself.

SmallFloats.format – Method.

```
format(K, P, Σ, Δ) -> Type
```

The concrete representation type for a format, from **runtime** parameters — format(8, 4, true, true) === Binary8p4se. The programmatic route to a type that can be an array element or a constructor target, and the supported replacement for spelling Binary{K,P,Σ,Δ} where a concrete type is meant.

Deliberately a Dict lookup: this path is reflection, it is not performance-sensitive, and it must not pretend to be. Where the parameters are static, the aliases and reptype are the zero-cost routes.

`SmallFloats.formatname` – Method.

`formatname(T)` — the draft §3.2 name of a format type.

`SmallFloats.ftz_variant` – Method.

```
ftz_variant(op, fr, f1, ρ) -> fn
```

Build the Annex-style approximate unary implementation: compute the defined result, then flush subnormal results to zero or `MinNormalOf(fr)`, whichever is nearer, ties toward zero (magnitude-symmetric for signed formats). Returned `fn` is suitable for `register_approx!`; for `fr` with precision P its κ is 2^(P-2) when P ≥ 2 (largest flushed-to-zero subnormal) and 0 when P = 1 (no subnormals exist).

`SmallFloats.get_show_style` – Method.

```
get_show_style()      -> Symbol
get_show_style(io::IO) -> Symbol
```

The style show will use: the process default, or — given an `IO` — the style that `io` resolves to, which is its `:binary_show_style` property when set and the process default otherwise.

The no-argument form previously read an undefined `io` and threw `UndefVarError` on every call.

`SmallFloats.isextended` – Method.

Whether the format's domain is Extended (datum set includes infinities).

`SmallFloats.issigned` – Method.

Whether the format is Signed (has a sign bit and negative datums).

`SmallFloats.list_approx` – Method.

Sorted names of all registered κ-approximate implementations.

`SmallFloats.measure_kappa` – Method.

```
measure_kappa(fn, op, fr, argformats, ρ; max_exhaustive=2^22, rng, samples=2^20)
    -> (κ::Float64, exhaustive::Bool)
```

Measure κ of `fn(x::argformats[1], …)::fr` against the defined results of draft operation `op` under ρ. Enumerates the full input cross-product when it has at most `max_exhaustive` points (always true for arity ≤ 2); otherwise measures on `samples` uniform draws and reports `exhaustive = false`.

`SmallFloats.nrandbits` – Method.

Number of random bits N consumed per projected element; 0 for pure modes.

`SmallFloats.projmode` – Method.

Translate a `Base.RoundingMode` to its `RoundingMode3109` counterpart (identity on the latter). Used only at API boundaries. Internals always carry RoundingMode3109.

`SmallFloats.rawvalue` – Method.

Unsafe raw constructor used by kernels after invariants are established.

Accepts an abstract format and normalizes through `reptype`; the type is passed *as an argument* to `_rawvalue` rather than assigned to a local, so it becomes a type parameter of the callee instead of a value the compiler has to fold (technique T6). Kernel call sites pass concrete types and pay nothing.

SmallFloats.register_approx! – Method.

```
register_approx!(name, op, fr, argformats, ρ, fn; κ=nothing, kwargs...) -> ApproxImpl
```

Register fn as a named κ-approximate implementation of op⟨argformats → fr, ρ⟩. κ is measured by enumeration (see measure_kappa); a declared κ smaller than the measured value is **rejected**. Omitting κ declares the measured value. NaN-κ implementations (non-finite mismatches) may be registered only by declaring κ=NaN.

SmallFloats.register_f32! – Method.

```
register_f32!(op, fr, argformats, ρ; κ=nothing) -> ApproxImpl
```

Build, measure, and register the Float32 kernel for op⟨argformats → fr, ρ⟩ under the canonical name f32/op/args→fr/rm_sm. Registration measures κ by exhaustive enumeration and fails loudly on understatement — that is the point. Measured baseline (2026-07-24): κ = 0 for all 120 same-format signatures × {Add, Subtract, Multiply, Divide, Sqrt} × RNE_{SatNone, SatFinite, SatPropagate}.

SmallFloats.reptype – Method.

```
reptype(F) -> Type
```

The concrete representation of a format. The identity on a concrete input, and the Binary{K,P,S,E} → Code8|Code16 map on an abstract one.

Julia will **not** exploit "this abstract type has exactly one subtype": there are no sealed types, inference cannot narrow an abstract element type to its unique subtype, and nothing stops a third party declaring another <: Binary. So reptype carries real weight rather than being belt-and-braces — it is how a method that holds only the format parameters reaches a constructible type.

The Val barrier is deliberate (invariant 9). The obvious spelling, K <= KSPLIT ? Code8{…} : Code16{…}, returns a Type, so its inferred return type is a small Union unless the compiler folds the comparison. It usually will; "usually" cannot underwrite a correctness-critical selection that also gates a zero-allocation guarantee. Behind a Val, each _rep method has exactly one concrete return type and there is nothing to fold. Gate G9 checks it per format.

SmallFloats.table_bytes – Method.

Total bytes currently held by every table cache.

SmallFloats.table_count – Method.

Number of cached unary/binary specializations, both code units.

SmallFloats.table_policy – Method.

```
table_policy(op, fr, f1[, f2[, f3]], ρ) -> (; shape, entries, bytes, reason)
```

Which shape an array call on this signature will take, and why — :A for the table gather, :B for the per-element scalar path.

§4 Stage 5 item 6: "this signature is running the compute kernel" should be *discoverable*, not inferred from a benchmark that came out slower than expected. Reads the same predicates the kernels do, so it cannot drift from them.

SmallFloats.ternary_count – Method.

Number of cached ternary specializations, both code units.

`SmallFloats.trailingsigbits` – Method.

Trailing-significand width P − 1 (the stored fraction bits).

`SmallFloats.unregister_approx!` – Method.

Remove a registered κ-approximate implementation (no-op if absent).

`SmallFloats.vmap!` – Function.

```
vmap!(dest, Val(op), fr, ρ, A[, B[, C]]; rng) -> dest
vmap(op, fr, ρ, A...; rng)
```

Elementwise draft operation over arrays of `Binary` values, projecting into `fr` under ρ. Pure ρ runs the Shape-A gather whenever a table exists or the ternary bitwidth policy grants one; stochastic specializations (and untabled ternary signatures) run the scalar path per element (each element drawing its own R).

`SmallFloats.vmap` – Method.

```
vmap(op, fr, ρ, pv::PackedVector; rng=nothing) -> Vector{fr}
```

Elementwise op over packed storage: unpack a 256-element tile into a byte scratch, run the ordinary byte kernel (`vmap!`), emit, repeat. Never computes on packed words directly — the deliberate compute-unpacked/store-packed boundary.

`SmallFloats.with_default_projection` – Function.

```
with_default_type(f, args...)          -> f(DefaultType(), args...)
with_default_returntype(f, args...)    -> f(DefaultReturnType(), args...)
with_default_projection(f, args...)    -> f(DefaultProjection(), args...)
```

Call `f` with the named session default as its first argument — the supported way to *consume* a default. While the default still holds its initial value (the overwhelmingly common case) the call is statically compiled against that constant: no dynamic dispatch, `f` fully specialized. After the default is changed, the call crosses a function barrier instead: one dynamic dispatch, everything inside fully specialized.

Allocation contract: the combinator's return type unions both paths. When `f`'s result type does not depend on the default — e.g. `with_default_projection` with the formats fixed by the caller — the union is concrete and the call is **zero-allocation with a concretely inferred result**. When the result's type *is* the default (`with_default_type((T, x) -> T(x), …)`), the value is computed on the specialized path but boxes once at escape — the irreducible cost of a runtime-chosen type.

```
julia> with_default_type((T, x) -> T(x), 1.5)
Binary8p2se(1.5 ≡ 0x41)

julia> with_default_projection((ρ, x, y) -> Add(Binary8p4se, ρ, x, y),
                               Binary8p4se(1.5), Binary8p4se(0.25))
Binary8p4se(1.75 ≡ 0x46)
```

`SmallFloats.with_default_returntype` – Function.

```
with_default_type(f, args...)          -> f(DefaultType(), args...)
with_default_returntype(f, args...)    -> f(DefaultReturnType(), args...)
with_default_projection(f, args...)    -> f(DefaultProjection(), args...)
```

Call `f` with the named session default as its first argument — the supported way to *consume* a default. While the default still holds its initial value (the overwhelmingly common case) the call is statically compiled against that constant: no dynamic dispatch, `f` fully specialized. After the default is changed, the call crosses a function barrier instead: one dynamic dispatch, everything inside fully specialized.

Allocation contract: the combinator's return type unions both paths. When `f`'s result type does not depend on the default — e.g. `with_default_projection` with the formats fixed by the caller — the union is concrete and the call is **zero-allocation with a concretely inferred result**. When the result's type *is* the default (`with_default_type((T, x) -> T(x), …)`), the value is computed on the specialized path but boxes once at escape — the irreducible cost of a runtime-chosen type.

```
julia> with_default_type((T, x) -> T(x), 1.5)
Binary8p2se(1.5 ≡ 0x41)

julia> with_default_projection((ρ, x, y) -> Add(Binary8p4se, ρ, x, y),
                               Binary8p4se(1.5), Binary8p4se(0.25))
Binary8p4se(1.75 ≡ 0x46)
```

`SmallFloats.with_default_type` – Method.

```
with_default_type(f, args...)          -> f(DefaultType(), args...)
with_default_returntype(f, args...)    -> f(DefaultReturnType(), args...)
with_default_projection(f, args...)    -> f(DefaultProjection(), args...)
```

Call `f` with the named session default as its first argument — the supported way to *consume* a default. While the default still holds its initial value (the overwhelmingly common case) the call is statically compiled against that constant: no dynamic dispatch, `f` fully specialized. After the default is changed, the call crosses a function barrier instead: one dynamic dispatch, everything inside fully specialized.

Allocation contract: the combinator's return type unions both paths. When `f`'s result type does not depend on the default — e.g. `with_default_projection` with the formats fixed by the caller — the union is concrete and the call is **zero-allocation with a concretely inferred result**. When the result's type *is* the default (`with_default_type((T, x) -> T(x), …)`), the value is computed on the specialized path but boxes once at escape — the irreducible cost of a runtime-chosen type.

```
julia> with_default_type((T, x) -> T(x), 1.5)
Binary8p2se(1.5 ≡ 0x41)

julia> with_default_projection((ρ, x, y) -> Add(Binary8p4se, ρ, x, y),
                               Binary8p4se(1.5), Binary8p4se(0.25))
Binary8p4se(1.75 ≡ 0x46)
```

Chapter 33

Internal API Index

33.1 Internal API Index

This index renders docstrings for unexported machinery used by the implementation and contributor guides. These names are not covered by the public compatibility contract. Begin with Architecture and Invariants and follow its source map before depending on an internal entry point.

Public contracts are collected in Public API Index.

`SmallFloats.DEFAULT_RBITS` – Constant.

Default random-bit budget used by no-argument stochastic projection constructors.

`SmallFloats.ExactExternalFloat` – Type.

External float element types whose widening to the Float64 carrier is exact.

`SmallFloats.KMAX` – Constant.

Highest bitwidth this package implements.

`SmallFloats.KMIN` – Constant.

Lowest bitwidth the draft defines (`K = 3`: sign, one exponent bit, one significand bit).

`SmallFloats.KSPLIT` – Constant.

Largest K whose code point fits a `UInt8` — the `Code8`/`Code16` seam.

`SmallFloats.MaybeR` – Type.

An explicit stochastic draw R, or `nothing` to draw one from the rng.

`SmallFloats.TABLE_EAGER_BITS` – Constant.

Largest table the package will *build*, as `log2(entries)`. 16 = 65 536 entries.

A bound on **time**, not memory: a table costs one scalar trip per entry, at a measured ~1 µs/entry, so this band caps first-call latency at ~0.1 s while 2^24 entries would be tens of seconds — regardless of how comfortably 32 MiB fits in RAM.

16 is not a guess — it is exactly the largest table the K ≤ 8 grid already builds (a binary 8×8 table, 65 536 entries), so **no K ≤ 8 decision changes** and G5 stays byte-identical by construction. It also admits the case that matters most at wide K: a unary table for a K = 16 format is 2^16 entries, the same count, now 128 KiB instead of 64 KiB because the entries are `UInt16`.

`SmallFloats.TABLE_MAX_BITS` – Constant.

Hard ceiling on a single table, as `log2(bytes)` — the byte budget invariant 10 requires. 25 = 32 MiB.

Compared **as bits**. Computing `1 << ΣK` in order to decide whether `1 << ΣK` is too large is exactly the bug the budget exists to prevent: at ΣK = 64 the shift wraps to zero and the impossible allocation looks free.

`SmallFloats.TERNARY_ADAPTIVE_BITS` – Constant.

K1+K2+K3 up to which a ternary table may be built adaptively once the signature has processed `TERNARY_BUILD_ELEMS[]` elements (default 21 bits = 2 MiB — the K=7 band). Above this, ternary ops always run the compute kernel.

`SmallFloats.TERNARY_BUILD_ELEMS` – Constant.

Cumulative element count at which an adaptive-band signature earns its table.

`SmallFloats.TERNARY_CACHE_BYTES` – Constant.

Byte budget for the ternary cache; least-recently-used tables evict first.

`SmallFloats.TERNARY_EAGER_BITS` – Constant.

K1+K2+K3 up to which a ternary table is built eagerly on first array call (default 18 bits = 256 KiB — covers every all-K≤6 signature).

`SmallFloats.THREADED_KERNELS` – Constant.

Master switch for threaded ternary compute loops.

`SmallFloats.THREAD_MIN_ELEMS` – Constant.

Minimum element count before the pure-ρ ternary compute loop threads.

`Core.Type` – Method.

```
(T::Type{<:Binary})(c::Unsigned) -> T
```

Construct a value from its **code point** (validated: `c` must satisfy the representation invariant for the format's K, or an `ArgumentError` is thrown; the check folds away when K equals the storage width and for constant codes). `Binary5p3sf(0x08) == Binary5p3sf(1.0)`.

`Unsigned` is the argument-type *class* with code-point semantics, at every width — every other `Real` (including signed `Integers`) constructs by projecting the numeric value. `Binary8p4se(0x02)` is code point 2 (the second-smallest subnormal), while `Binary8p4se(2)` is the number 2.0. Widening an `Unsigned` is lossless, so the only way a wrong-width argument can be wrong is by being out of code range — which throws. `rawvalue(T, c)` remains the unchecked kernel-internal route.

`SmallFloats.BigExactF` – Type.

Deferred exact result: `f()` returns the value as an exact `BigFloat`.

`SmallFloats.Code16` – Type.

```
Code16{K,P,SGN,EXT} <: Binary{K,P,SGN,EXT}
```

The two-byte representation — every format with `KSPLIT < K ≤ KMAX`. Differs from its `Code8` sibling only in the width of the word carrying the code point; all semantics live in the `Binary` parameters.

SmallFloats.Code8 – Type.

```
Code8{K,P,SGN,EXT} <: Binary{K,P,SGN,EXT}
```

The one-byte representation of a P3109 format — every format with K ≤ 8. Differs from its Code16 sibling only in the width of the word carrying the code point; all semantics live in the Binary parameters.

SmallFloats.ComputeDecode – Type.

2^K > 256: compute from the code point; a 65 536-element tuple literal is not a table, it is a compile-time hazard (invariant 10).

SmallFloats.DecodePolicy – Type.

Decode strategy for a format: a 2^K-entry constant tuple, or computed.

SmallFloats.Enclose128F – Type.

Correctly-rounded Float128 bracket (plan Site C, Class R): IEEE mandates correct rounding for Float128 `/`, `sqrt`, `fma`, so a nearest-CR result of a value known inexact brackets the truth in the *open* interval `(lo, hi) = (prevfloat(q), nextfloat(q))` with no envelope assumption. `f` is the MPFR fallback for (near-impossible at P ≤ 8) grid-straddling brackets.

SmallFloats.EncloseF – Type.

Deferred enclosure, resolved by `_finish` through up to three stages, cheapest first:

1. `yd` — an *eager* Float64 estimate (NaN ⇒ absent) whose libm evaluation is faithful (≤ 1 ulp); the true value is taken to lie within `|yd|·2^-45` (_F64_RELEXP), ≥ 2^7 slacker than any faithful-libm error.
2. `fq` — a zero-argument Float128 estimator (plan Site D, Class E; `nothing` ⇒ absent): true value within `|y|·2^-90` of `y = fq()` (_F128_RELEXP), a bound ≥ 2^18 slacker than any published libquadmath transcendental error, discharged empirically by the differential-build tests.
3. `f(prec)` — the rigorous MPFR ladder: directed `(lo, hi)::NTuple{2,BigFloat}` strictly bracketing the true value.

Each estimate stage returns only when the two-sided sticky projection of its envelope agrees on a single code point; any disagreement (or an absent/degenerate estimate) falls through to the next stage. Stage 3 always decides.

SmallFloats.Head – Type.

```
Head
```

Evaluation-carrier selector: a singleton tag, so carrier choice is a method lookup rather than a value the compiler must fold. Rungs are ordered `HeadF64 < HeadF128 < HeadExact` by carrier width.

SmallFloats.HeadExact – Type.

Rung 3 — unbounded exponent. `BigFloat` until Stage 7, `Dyadic` after; the tag is the seam that makes that swap a one-line change (gate G7 pins that the two agree).

SmallFloats.HeadF128 – Type.

Rung 2 — `Float128` arithmetic (the existing escalation machinery, promoted to entry point). Allocation-free: `Float128` is a bitstype.

`SmallFloats.HeadF64` – Type.

Rung 1 — `Float64` arithmetic. Every K ≤ 8 format and 432 of the 504.

`SmallFloats.Rounded` – Type.

Result of ωRoundToPrecision in canonical integer form: a kind tag plus `sign · S · 2^Q` for finite results (design §5.2). The saturation stage consumes this without ever re-touching the carrier value.

`SmallFloats.StickyF` – Type.

Sticky-head exact result (wide-spread tail, Float128 revision follow-on): the true value is `v + sgn·ε` for an infinitesimal `ε > 0` — `v` carries every bit the projection can consume and `sgn ∈ {-1,+1}` the direction of the neglected tail. Sound because `v` is either exactly on a rounding threshold of the target grid (where `sticky` decides, for every mode including stochastic sub-grids) or strictly farther from the nearest threshold than the tail magnitude; the emitting sites in oracle.jl discharge that bound (operand significands ≤ 17 bits, target grids ≤ P−1+N ≤ 67 fractional bits, spreads > the *DE** thresholds). Non-allocating: replaces the BigFloat escalation for FMA/FAA.

`SmallFloats.TableDecode` – Type.

`2^K ≤ 256`: the generated constant tuple folds and costs one load.

`SmallFloats.TableKey` – Type.

Value key for the table cache: op + format parameters + ρ parameters (all values, so the cache itself is type-stable and non-allocating to query).

`SmallFloats.TernaryKey` – Type.

Value key for a ternary table: op + result + three operand formats + ρ.

`Base.nextfloat` – Method.

NextGreaterThan(v) (draft §4.16): the least datum greater than v in the total order — one step up the code lattice, with NaN → NaN and MaxFinite/+Inf → NaN at the top. `Base.nextfloat` on `Binary` is this operation.

`Base.prevfloat` – Method.

NextLessThan(v) (draft §4.16): the greatest datum less than v in the total order — one step down the code lattice, with NaN → NaN and MinFinite/−Inf → NaN at the bottom. `Base.prevfloat` on `Binary` is this operation.

`Base.widen` – Method.

The public promotion target for T against external numeric types. Always a Julia `AbstractFloat`, never the internal exact carrier.

`SmallFloats._bp_element` – Method.

One element of ωBlockProject: the draft's S-special rows, then ωDivide + ωProject.

Fast-path cascade after the special rows, ordered cheapest-first by result kind: exact Float64 quotient → CR Float128 quotient (exact or one-ulp bracket) → CR-divided Enclose128F bracket → fq-composition envelope (2^-89) — each resolved by the two-sided sticky gate; any miss falls through to the rigorous MPFR interval at the bottom.

`SmallFloats._check_tabulable` – Method.

Stochastic ρ is a distribution over R, never a table (design §5.4).

`SmallFloats._comparable` – Method.

Numeric comparison is defined only when neither operand is NaN — every ==/</<= below returns `false` otherwise, per IEEE unorderedness.

`SmallFloats._extexprange` – Method.

(least subnormal exponent, greatest normal exponent) of an external float.

`SmallFloats._resolve_rng` – Method.

Resolve a caller-supplied rng, falling back to the task-local default. Call sites hoist this out of loops so array kernels resolve once per call, not per element.

`SmallFloats._rng_for` – Method.

The rng an operation under ρ will actually draw from — `nothing` for pure ρ, which must never touch RNG state.

`SmallFloats._rungindex` – Method.

Per-format rung index from the exponent bias. A pure `Int` function of the type parameters, so `Val(_rungindex(F))` is a compile-time constant.

Stated on 2B rather than B so it is visibly the `n = 2` case of `_rungindex_span` and not an independent constant pair. `B ≤ 512` ⟺ `2B ≤ 1024` and `B ≤ 8192` ⟺ `2B ≤ 16384`, so the 432/64/8 partition is unchanged — G9 enumerates it.

`SmallFloats._rungindex_span` – Method.

Rung index for a monomial whose factor biases sum to ΣB. This is the primitive form of the boundary rule stated above; the per-format answer is its `n = 2` same-format case.

`SmallFloats._ternary_table_for` – Method.

```
_ternary_table_for(op, fr, f1, f2, f3, ρ, nelems) -> Union{Nothing,Memory{UInt8}}
```

The kernel-facing policy gate, called once per array operation with the call's element count. Eager band (Σ K_i ≤ `TERNARY_EAGER_BITS[]`): fetch/build now. Adaptive band (≤ `TERNARY_ADAPTIVE_BITS[]`): return the cached table if present, otherwise accumulate `nelems` against the signature and build only once it has earned `TERNARY_BUILD_ELEMS[]`. Beyond the adaptive band (the K=8 territory): always `nothing` — the compute kernel is the right tradeoff there.

`SmallFloats._unitmask` – Method.

```
_unitmask(U, K) -> U
```

The low-K-bits mask of storage unit `U`, built by **complement** — shift down from `typemax`, never up from one.

Julia defines a shift at or beyond a type's width as zero, so `UInt8(1) << 8` and `UInt16(1) << 16` are both `0`, and `2^K` computed in a unit of width K is `0`. The format grid **always contains a format whose K equals its storage width** (K = 8 on `UInt8`, K = 16 on `UInt16`), so that case is structural, not an edge. Here the shift amount is `width − K ∈ [0, width−1]`, in which the pathological amount is unreachable *by construction*, at every K, for every unit — including a future `Code32`.

`SmallFloats._unsupported` – Method.

Refuse a Base verb with a reason. Absence and refusal must not look alike.

`SmallFloats.apply_op` – Method.

apply_op(Val(op), fr, ρ, R, xs...) — evaluate op's ω-semantics on decoded operands and project the result into `fr` under ρ (with random bits R).

The evaluation carrier comes from the **operands**, not from `fr`. That is `carriers.jl`'s stated rule — the carrier constrains what ωeval may form, never what may be projected into a format, because `round_to_precision` works in `(P, B)` integer space and is exact for any exact input. Stage 3 selected on `rung(fr)` instead, as a deliberate stand-in while decode still refused K ≥ 9; keeping that would have refused `Add(Binary16p2se, ρ, x8, y8)`, whose operands are ordinary `Float64` datums and whose exact sum needs no wide carrier at all.

Completeness of the operand-only rule rests on one premise, stated because Stage 3's version of it is what M28 was: the spec register's maximum factor count is **2**, and the rung boundaries are stated on 2B precisely so that a datum on carrier C guarantees its square is also on C. Beyond two factors — `ScaledMultiply`'s four — the monomial bound stops following from the operand types and `rung(op, Fs...)` must be used with the formats in hand. That is why `blocks.jl` calls the join and this does not.

`SmallFloats.bigprec_prod` – Method.

```
bigprec_prod(xs...) -> Int
```

bigprec for an expression whose leading term is the **product** $x_1 \cdot x_2$ and whose remaining terms are added to it — the FMA shape. The product's exponent is $e_1 + e_2$, which lies outside the range of the operands' own exponents, so the sum method would understate the spread. Its significand is $P_1 + P_2$ bits wide.

`SmallFloats.blockdecode` – Method.

ωBlockDecode (draft §5.1.1): per lane ωMultiply(decode(s), decode(x_i)), exact on the carrier the block's two formats jointly require.

The carrier is keyed on `rung(Val(:Multiply), FS, FE)` — **not** on either format's own rung, and that distinction is the whole content of this method. A block's scale and element formats are independent, and what has to fit is their *product*: `carriers.jl`'s rule is $\Sigma B_i \le$ emax, so two formats that are each rung 1 at $2B \le 1024$ can compose into a lane product needing `B_S + B_E` well past it. Asserting `carriertype(rung(FE))` would be wrong in the worst way available here — the overflow produces `±Inf`, a plausible special value, rather than an error.

The assertion stays because it is load-bearing for more than documentation: it is what keeps the returned `NTuple` concretely typed, and with it the reductions downstream allocation-free. It now names a carrier computed from both formats instead of a constant.

`SmallFloats.blockproject` – Method.

ωBlockProject (draft §5.1.2) over a tuple of result kinds, scale supplied as a value.

`SmallFloats.codemask` – Method.

The low-K-bits mask of a format's storage unit (representation invariant 3): the code point occupies the low K bits, the high bits are maintained zero.

`SmallFloats.datumcarrier` – Method.

The exact carrier for the *datums* of F: what `decode(v::F)` returns.

`SmallFloats.datumsexact` – Method.

```
datumsexact(X, F) -> Bool
```

`true` when **every** datum of format F is exactly representable in the external float type X.

Three conditions, one per way a conversion can lose information:

- `P ≤ precision(X)` — the significand fits;
- the largest finite datum's exponent is within X's normal range. That exponent is `B − 1` for a signed format and `0` for an unsigned one, which is not symmetry-breaking pedantry: an unsigned format spends its whole exponent field below 1, so `Binary16p1uf`'s datums run down to `2^-32769` while its largest is `0.5`;
- `2 − P − B ≥` X's least subnormal exponent — and the smallest datum being a *subnormal* of X is fine, because a datum is `sig · 2^(2−P−B)` with integer `sig`, so once the step is a multiple of X's subnormal step every datum lands on X's grid exactly.

`SmallFloats.decodepolicy` – Method.

Decode strategy: a `2^K ≤ 256` constant tuple, or computed from the code.

`SmallFloats.encode` – Method.

```
encode(T, sign, S, Q) -> UInt8   (private; design §3.3)
```

ωEncode from canonical integer form: value = sign · S · 2^Q, with S ∈ 0:2^P (S == 2^P is the next-binade carry, draft §4.7.4 NOTE 4). Precondition: the value is in the datum set of T (guaranteed by RoundToPrecision ∘ Saturate).

`SmallFloats.get_table` – Function.

```
get_table(op, fr, f1, [f2,] ρ) -> Memory{codeunit_type(fr)}
```

Fetch (building and caching on first use) the complete result table for the pure-ρ specialization `op(fr; f1[, f2])` under ρ: entry `c + 1` (unary) or `(c1 << K2) + c2 + 1` (binary) holds the result code point for those operand code points. Throws for stochastic ρ, and throws when the table would exceed the byte budget — its contract is to return a table, so it says so rather than returning `nothing`. Callers that can fall back want `table_for`.

`@noinline` by design: kernels call this once per array operation and index the returned table in their hot loop.

`SmallFloats.get_table` – Method.

```
get_table(op, fr, f1, f2, f3, ρ) -> Memory{UInt8}
```

Ternary form: entry `((c1 << K2 | c2) << K3) + c3 + 1` holds the result code point. Builds unconditionally (policy lives in `_ternary_table_for`); pure ρ only.

`SmallFloats.joinhead` – Method.

The wider of two heads. The carrier join is a max over this order, which is what lets block and scaled operations compose without a second rule.

`SmallFloats.nan_order_key` – Method.

```
nan_order_key(F)
```

The order key reserved for the single NaN — **below** every finite key and ±Inf.

Key `0` is free by construction: every datum's key is at least 1 (`order_key` adds 1 above the code arithmetic on both the signed and unsigned paths), so `zero(orderkeytype(F))` sits strictly under the whole datum lattice without shifting any other key.

Per format, not a constant, and the key type is `orderkeytype(F)` rather than `UInt16`. The reason is arithmetic, not tidiness: a format has 2^K code points mapping to keys 1 … 2^K, so `UInt16` runs out at exactly K = 16 — `UInt16(c) + UInt16(1)` for `c = 0xffff` wraps **silently to 0**, which is now precisely the NaN key: the largest datum would compare below NaN and below every other datum. `orderkeytype` returns `UInt32` for `Code16`, which has room for keys 1 … 2^16 above the NaN sentinel.

(§4 Stage 4 item 3, pulled forward into Stage 3 — §11 M16. The rest of Stage 3 makes wide formats fail loudly; a wraparound in the total order is the one K ≥ 9 defect that would have been silent, so it is closed here rather than carried for a commit.)

`SmallFloats.orderkeytype` – Method.

Unsigned type wide enough for a format's 2^K + 1 total-order keys.

`SmallFloats.packing_saves` – Method.

```
packing_saves(F) -> Bool
```

Whether `PackedVector{F}` is smaller than `Vector{F}` — `false` exactly when `bitwidth(F)` equals the storage unit's width, i.e. K = 8 and K = 16.

Why this is a query and not a refusal. §4 Stage 4 item 5 called for `PackedVector` to refuse loudly at K = 16, on the grounds that packing is the identity there and a silent identity invites benchmark confusion. The grounds are right and the boundary is not: packing is *equally* the identity at K = 8, which has shipped since before the extension and is enumerated by G5's packed section over all 120 formats. A rule that fires on one of two identical cases is not a rule, and the alternative — refusing both — would be a breaking change justified by tidiness.

So the fact is exposed instead of enforced. Benchmarks and the docs read it; `PackedVector` at those widths stays correct, stays supported, and stays pointless. (§11 M21.)

`SmallFloats.project` – Method.

```
project(T, ρ, X; R=0, sticky=0) -> T
```

Project a closed-extended-real carrier value into format T under ρ. `X::Float64` must be *exact* (the Float64 fast path); `X::BigFloat` likewise (or an interval endpoint with `sticky ∈ {-1,+1}`). R is the stochastic draw.

`SmallFloats.promotecarrier` – Method.

The public promotion target for F against external numeric types. Always a Julia `AbstractFloat`, never the internal exact carrier.

SmallFloats.rung – Method.

```
rung(op::Val, Fs::Type{<:Binary}...) -> Head
```

The carrier for evaluating op over operands of formats Fs. A **join of two lower bounds**, and both are necessary:

- the **monomial bound** — op's exact result is a sum of monomials in at most opfactors(op) datum factors, so it needs _rungindex_span(nf · maxB) (nf · maxB rather than ΣB_i over the actual operands: it dominates every monomial shape, including x^2 from Hypot, which ΣB_i does not, and it coincides with the tight bound in the same-format case that is virtually all traffic);
- the **operand bound** — decode(v::F_i) has already produced a value on datumcarrier(F_i), so the head can never be narrower than any operand's own rung regardless of what the monomial needs.

Omitting the second is the subtle error. Abs has one factor, so the monomial bound alone would pick rung 1 for a B = 1024 format — whose datums arrive as Float128, for which ωeval(::HeadF64, …) has no row. The join makes lift's missing narrowing method a consequence rather than a hazard: the head dominates every operand, so nothing ever needs narrowing.

For the same format at two factors this agrees with rung(F) exactly, which is what keeps the 432/64/8 partition and every K ≤ 8 result unchanged.

SmallFloats.rung – Method.

```
rung(F) -> Head
```

The starting carrier for evaluating on datums of format F alone. For an operation over several formats use rung(op, Fs...), which additionally accounts for the operation's factor count — a ScaledMultiply of two Group A formats can need a wider carrier than either operand does.

SmallFloats.rungindex – Method.

Rung index 1/2/3 of a head — for ordering and for the carrier join.

SmallFloats.table_for – Method.

```
table_for(op, fr, f1[, f2], ρ) -> Union{Nothing,Memory{codeunit_type(fr)}}
```

The kernel-facing gate for unary and binary specializations: the table if policy grants one, nothing if the caller should run the scalar path per element.

Separate from get_table on purpose — **one name per question.** get_table's contract is "return the table", so it throws when it cannot honour that, and that is what the suite, conformance and the precompile workload want. table_for asks the different question "should there be a table at all", and answers nothing without prejudice. Its Union return costs nothing because kernels call it once per array operation and branch outside the loop.

SmallFloats.table_keys – Method.

Keys of every cached unary/binary specialization, in a deterministic order.

SmallFloats.tablebits – Method.

log2 of the bytes a table over Fs with results in fr would occupy.

SmallFloats.tablecache – Method.

The unary/binary table cache holding results of format fr.

`SmallFloats.ternarycache` – Method.

The ternary table cache holding results of format `fr`.

`SmallFloats.unpack_tile!` – Method.

Unpack `pv[first:first+len-1]` into `buf` (byte scratch); tile-granular kernel entry.

`SmallFloats.within_byte_budget` – Method.

Would this table fit the byte budget? (Memory, not build time.)

Part VI

Examples and Evidence

Chapter 34

Gallery

34.1 Example Gallery

Every worked example in the documentation, in one list. Each title links to the example itself. All are runnable as shown; outputs were captured from real sessions and seeds are fixed, so you can reproduce them exactly.

The examples live on two pages, split by the level they work at rather than by subject:

- **Applied** — the public API only. Format choice, quantization, blocks, stochastic rounding, packing.
- **Internals** — unexported machinery. Pipeline introspection, exhaustive self-verification, κ measurement, benchmarking. Stable enough to learn from, not covered by semver.

Basic

example	what it shows
One value under several rounding modes	deterministic policies compared on one between-grid value
Enumerating a whole format	a 16-point format printed in full, in total order
Saturation in one line each	`SatNone` / `SatFinite` / `SatPropagate` on the same overflow
Random values under a chosen projection	`rand`/`randn` land on code points through the engine
Watching RoundToPrecision work	the exact `(sign, S, Q)` decomposition before saturation sees it
Symbolic sticky	how directed modes land exactly on asymptotes
Tables are the scalar path, memoized	table entry ≡ scalar result, by construction
Order keys	the sign–magnitude fold that comparisons and the counting sort run on

General AI

example	what it shows
Log-odds belief updating	the running belief drifts far from exact while the threshold decision stays correct — order survives quantization when magnitude does not
Fuzzy inference in an unsigned format	Gödel t-norm/s-norm and product t-norm as bit-exact registry ops, so a rule base reproduces to the code point across machines
Search heuristics keep the ordering	200 scores collapse onto 78 code points; argmax and top-k survive
Exhaustive monotonicity audit	proving by enumeration that a format conversion never inverts the total order
κ-safe decision margins	two results more than 2κ code points apart cannot have their comparison inverted — and the exhaustive check of that rule

Machine Learning

example	what it shows
Quantizing a weight tensor	MSE, max error, overflow fraction — the three-number check for any quantization
Picking a format: precision vs range	RMSE and MaxFinite across four $K = 8$ formats on the same data
Stochastic rounding is unbiased	where nearest accumulates a systematic drift and stochastic does not
Verifying a quantizer exhaustively	your own code checked against an independent reference over every input
The κ workflow, including the refusal	registration rejecting an understated κ — the registry measures, it does not trust
Exporting the conformance declaration	what you hand a reviewer

Deep Learning

example	what it shows
MX-style block quantization	shared-scale blocks, and the staging mistake that silently costs accuracy
Quantized dot products, one rounding	all lane products accumulated exactly, projected once at the end
The swamping demo	why a training loop stalls under nearest and does not under stochastic
Activations are 256-byte tables	a whole unary op *is* its table, at a fraction of a nanosecond per element
Packing a quantized model	bit-packed storage at K not a multiple of 8
Proving a fused kernel exact	`BlockDotProduct` against 512-bit truth
Stochastic rounding, audited	the full-R sweep: every random draw, not a sample
Hard-tanh κ measurement	an accelerator-style activation, measured rather than assumed
Benchmarking without measuring the dispatcher	the two measurement post-mortems this package learned from

Verification

example	what it shows
Checking the code-point algebra	every distinguished code of all 504 formats, and the decoding formula at every positive finite code point

Use it

Task-shaped instructions, as opposed to worked demonstrations, begin with Choose a Format. A workflow tells you what to do; an example shows a complete session and interprets what its output means.

Chapter 35

Applied Sessions

35.1 Applied Sessions

Runnable examples spanning basic use, general AI, machine learning, and deep learning. Every output shown was captured from a real session; seeds are fixed so you can reproduce them exactly.

Basic

Two foundational examples live with their explanations

One value, every rounding mode is in Rounding and Saturation, and *Enumerating a whole format* is in Values, Code Points, and Conversion. Those pages explain the ideas the examples expose, so this collection links to them rather than repeating them.

Saturation in one line each

```
julia> w, two = Binary8p4se(200.0), Binary8p4se(2.0);   # MaxFinite is 224

julia> Multiply(Binary8p4se, RNE_SN, w, two)
Binary8p4se(Inf ≡ 0x7f)

julia> Multiply(Binary8p4se, RNE_SF, w, two)
Binary8p4se(224.0 ≡ 0x7e)
```

Random values under a chosen projection

`rand` and `randn` take a `projection` keyword on their scalar `::Type` forms. One underlying Float64 draw, landed on the format's grid four different ways — the seed fixes the draw, the projection decides the landing:

```
julia> rand(Xoshiro(2), Float64)                # the underlying uniform draw
0.0022505868897625403

julia> rand(Xoshiro(2), Binary8p4se)            # default: floor (stays in [0,1))
Binary8p4se(0.001953125 ≡ 0x02)

julia> rand(Xoshiro(2), Binary8p4se; projection = RTP_SN)   # ceiling
Binary8p4se(0.0029296875 ≡ 0x03)

julia> rand(Xoshiro(2), Binary8p4se; projection = RNE_SN)   # nearest
Binary8p4se(0.001953125 ≡ 0x02)

julia> rand(Xoshiro(2), Binary8p4se; projection = RSA_SN(8))  # stochastic
Binary8p4se(0.001953125 ≡ 0x02)
```

A stochastic projection draws its random bits from the *same* rng as the uniform draw, so seeded streams stay reproducible. Note the default is the contract-keeper: a nearest or upward projection can return exactly `1.0` (mass near the top rounds up), which is why floor is the default.

The same keyword on `randn` — here the draw is `-1.9757…`:

```
julia> randn(Xoshiro(6), Binary8p4se)           # default: nearest + SatFinite
Binary8p4se(-2.0 ≡ 0xc8)

julia> randn(Xoshiro(6), Binary8p4se; projection = RTZ_SF)  # toward zero
Binary8p4se(-1.875 ≡ 0xc7)
```

Where the saturation half of the default earns its keep: `Binary3p1se` has `MaxFinite = 1.0`, so normal tail draws overflow constantly. Seed 360 draws `z = 3.326…`:

```
julia> randn(Xoshiro(360), Binary3p1se)         # SatFinite clamps the tail
Binary3p1se(1.0 ≡ 0x02)

julia> randn(Xoshiro(360), Binary3p1se; projection = RNE_SN)
Binary3p1se(Inf ≡ 0x03)                         # opt out, get the overflow
```

The array and `!` forms (`rand(T, dims)`, `randn!(A)`, …) always use the defaults; for arrays under another projection, draw scalars: `[rand(rng, T; projection = ρ) for _ in 1:n]`.

General AI

Log-odds belief updating — and where tiny formats bite reasoning

Classic probabilistic reasoning: accumulate independent evidence as log-likelihood ratios. A sensor with hit rate 0.85 and false-alarm rate 0.30 contributes `log(0.85/0.30) ≈ 1.0415` per alarm. In `Binary8p3se` that datum is 1.0 — and the running belief stalls exactly the way the gradient accumulator does in the Deep Learning section:

```
using SmallFloats

function logodds_demo(nalarms)
    llr = Binary8p3se(log(0.85 / 0.30))      # quantized evidence weight
    acc = Binary8p3se(0.0)
    for _ in 1:nalarms
        acc = Add(Binary8p3se, RNE_SN, acc, llr)
    end
    post(l) = 1 / (1 + exp(-l))
    exact = nalarms * log(0.85 / 0.30)
    (decode(llr), round(exact; digits=2), decode(acc),
     round(post(exact); digits=5), round(post(decode(acc)); digits=5))
end
logodds_demo(12)
```

```
(1.0, 12.5, 8.0, 1.0, 0.99966)
# llr datum, exact log-odds, quantized log-odds, exact posterior, quantized posterior
```

The accumulator is wrong by a third (8.0 vs 12.5 — near 8 the ulp is 2.0, so 8 + 1 rounds straight back to 8), yet the *decision* is untouched: both posteriors are ≈ 1. Threshold decisions need order, not magnitude — which is why coarse formats work in reasoning pipelines far past the point where their arithmetic looks broken. When magnitude does matter, the cures are the same as for gradients: a wider accumulator format, or stochastic rounding.

Fuzzy inference in an unsigned format

Membership degrees live in [0, 1] and are never negative — a job for an unsigned finite format. `Binary8p6uf` gives a [0, 15.5] range with 1/32 resolution at 1; the classic fuzzy connectives are registry operations (`Minimum` = Gödel t-norm, `Maximum` = s-norm, `Multiply` = product t-norm):

```
F = Binary8p6uf
trap(x, a, b, c, d) = clamp(min((x - a) / (b - a), 1.0, (d - x) / (d - c)), 0.0, 1.0)
warm = F(trap(26.0, 15, 20, 24, 28))         # membership of 26 °C in "warm"
hot  = F(trap(26.0, 24, 30, 100, 101))       # … and in "hot"

(decode(warm), decode(hot),
 decode(Minimum(F, RNE_SN, warm, hot)),      # warm AND hot
 decode(Maximum(F, RNE_SN, warm, hot)),      # warm OR hot
 decode(Multiply(F, RNE_SF, warm, hot)))   # product t-norm
```

```
(0.5, 0.3359375, 0.3359375, 0.5, 0.16796875)
```

Because every connective is a bit-exact registry op, a fuzzy rule base evaluated in `Binary8p6uf` is *reproducible across machines to the code point* — a property an implementation that substitutes unmeasured approximations cannot promise.

Search heuristics: what quantized scores keep is the ordering

Game-tree and beam search consume evaluation scores only through comparisons. Quantizing 200 heuristic scores to 8 bits collapses them onto 78 distinct code points — yet the argmax survives, and sorting is exact (integer order keys, O(n) counting sort, the single NaN ordered first, deterministically):

```
using Random
rng = Xoshiro(21)
scores = randn(rng, 200) .* 3
q = Binary8p4se.(scores)

(argmax(scores), argmax(decode.(q)), argmax(scores) == argmax(decode.(q)),
 decode.(sort(q; rev=true)[1:5]), length(unique(codepoint.(q))))
```

```
(140, 140, true, [8.0, 7.5, 6.5, 6.0, 6.0], 78)
```

The two 6.0s are the caveat: quantization creates *ties* the exact scores did not have. `TotalOrder`'s deterministic tie behavior means the search expands the same node on every run — ties change which answer you get, not whether you can reproduce it.

Machine Learning

Quantizing a weight tensor and measuring the damage

```
using SmallFloats, Random, Statistics

rng = Xoshiro(42)
w = randn(rng, 10_000) .* 0.25          # typical trained-weight scale
q = Binary8p4se.(w)                     # project every weight
back = decode.(q)

mean((w .- back).^2), maximum(abs.(w .- back)), count(isinf, back) / length(back)
```

```
(4.41e-5, 0.0312, 0.0)                  # MSE, max |error|, overflow fraction
```

The max error is exactly half an ulp of the largest binade the data reached — the worst case nearest rounding permits.

Picking a format: precision vs range

For the same Gaussian tensor, K = 8 formats trade significand bits against exponent range. RMSE doubles per lost significand bit; MaxFinite grows explosively:

```
rng = Xoshiro(7); x = randn(rng, 50_000)
for F in (Binary8p5se, Binary8p4se, Binary8p3se, Binary8p2se)
    back = [decode(Convert(F, RNE_SF, xi)) for xi in x]
    println(rpad(formatname(F), 12), " rmse = ", round(sqrt(mean((x .- back).^2)); sigdigits=3),
            "   maxfinite = ", decode(MaxFiniteOf(F)))
end
```

```
Binary8p5se  rmse = 0.0133   maxfinite = 15.0
Binary8p4se  rmse = 0.0266   maxfinite = 224.0
Binary8p3se  rmse = 0.0528   maxfinite = 49152.0
Binary8p2se  rmse = 0.103    maxfinite = 2.147483648e9
```

For unit-scale data, spend bits on precision; buy range only when your data needs it (or get it from a block scale — see below).

Stochastic rounding is unbiased where nearest is not

Nearest rounding maps 0.30078125 to 0.3125 every single time — a systematic +0.0117 bias. Stochastic rounding is right *on average*:

```
target = 0.30078125                       # ν = 3/16 of an ulp above 0.296875
σ = RSA_SN(16)                          # StochasticA{16} with SatNone
rng = Xoshiro(1)
m = mean(decode(Convert(Binary8p4se, σ, target; rng)) for _ in 1:200_000)
(decode(Binary8p4se(target)), m)
```

```
(0.3125, 0.300765)                        # RNE result vs stochastic mean ≈ target
```

This is the property that makes stochastic rounding matter for accumulating many small contributions (gradients, activations statistics) in a low-precision format.

Deep Learning

MX-style block quantization — and the staging pitfall

Block formats pair a shared scale with narrow elements, tracking dynamic range that a bare element format cannot. Here each row of `W` lives at a different scale, spanning ~2^{16} across the matrix:

```
using SmallFloats, Random, Statistics

rng = Xoshiro(3)
W = randn(rng, 64, 64) .* 2 .* (2.0 .^ (collect(0:63) ./ 4))   # per-row scales

function mx_rows(W, ::Type{FST}, ::Type{FS}, ::Type{FE}, ::Val{B}) where {FST,FS,FE,B}
    n, m = size(W)
    [begin
        # stage through a WIDE-RANGE format so nothing clamps before scaling
        seg = ntuple(k -> Convert(FST, RNE_SF, W[i, (j-1)*B + k]), Val(B))
        ConvertToBlockMaxAbsFinite(FS, FE, RNE_SN, RNE_SN, seg)
    end for i in 1:n, j in 1:m ÷ B]
end

blocks = mx_rows(W, Binary8p2se, Binary8p1uf, Binary8p4se, Val(32))
recon = [let b = blocks[i, (j-1) ÷ 32 + 1]
             decode(b.s) * decode(b.x[(j-1) % 32 + 1])
         end for i in 1:64, j in 1:64]
plain = [decode(Convert(Binary8p4se, RNE_SF, w)) for w in W]

relerr(A) = sqrt(mean(((W .- A) ./ max.(abs.(W), 1e-9)).^2))
relerr(plain), relerr(recon)
```

```
(0.616, 0.109)      # plain 8p4se clamps the large rows; MX tracks them
```

Stage wide, then block-quantize

`ConvertToBlockMaxAbsFinite` takes already-`Binary` elements. If you stage your `Float64` data through the *element* format first, `SatFinite` clamps the large values **before** the block scale can absorb them, and blocks buy you nothing — we measured exactly that (0.617 vs 0.616) with `Binary8p4se` staging. Stage through a wide-range format (`Binary8p2se` above); the residual 0.109 here is the staging format's own precision, which bounds what any downstream scheme can keep.

At B = 32 with an 8-bit scale the storage cost is 8.25 bits/value.

Quantized dot products with one final rounding

`BlockDotProduct` computes every lane product and the accumulation *exactly*, then projects once:

```
rng = Xoshiro(9)
a64, b64 = randn(rng, 32), randn(rng, 32)
qb(v) = ConvertToBlockMaxAbsFinite(Binary8p1uf, Binary8p4se, RNE_SN, RNE_SN,
            ntuple(i -> Convert(Binary8p4se, RNE_SF, v[i]), Val(32)))
dq = BlockDotProduct(Binary8p4se, RNE_SN, qb(a64), qb(b64))
(a64'b64, decode(dq))
```

```
(5.0634, 4.5)       # difference is input quantization only, never accumulation error
```

Why training loops like stochastic rounding: the swamping demo

Accumulate 400 gradient steps of 0.011 into a `Binary8p3se` accumulator. Under nearest rounding, the moment the accumulator dwarfs the increment, every add rounds back to the accumulator — it **stalls**. Stochastic rounding keeps absorbing the increments in expectation:

```
function accumulate_demo(nsteps, g)
    σ = RSA_SN(16)
    rng = Xoshiro(11)
    acc_rne = Binary8p3se(0.0); acc_sto = Binary8p3se(0.0)
    for _ in 1:nsteps
        acc_rne = Add(Binary8p3se, RNE_SN, acc_rne, Convert(Binary8p3se, RNE_SN, g))
        acc_sto = Add(Binary8p3se, σ, acc_sto, Convert(Binary8p3se, σ, g; rng); rng)
    end
    (nsteps * g, decode(acc_rne), decode(acc_sto))
end
accumulate_demo(400, 0.011)
```

```
(4.4, 0.125, 5.0)   # exact sum, RNE accumulator (stalled!), stochastic accumulator
```

The stochastic result is noisy (5.0 vs 4.4 on this seed) but unbiased; the RNE result is *wrong by 35×* and no amount of steps will fix it. In practice you keep a wider accumulator when you can — and use stochastic rounding when you can't.

Activation functions are 256-byte lookup tables

For pure specs, a unary activation over an 8-bit format is a *finite function* — the whole nonlinearity is one 256-byte table, built bit-exactly through the scalar path and then gathered per element. This is precisely the activation LUT you would burn into an accelerator:

```
using SmallFloats, Random
empty_tables!()
rng = Xoshiro(4)
pre = Binary8p4se.(randn(rng, 4096) .* 2.5)       # pre-activations, ±2.5σ
act = Tanh(Binary8p4se, RNE_SN, pre)          # table-gather kernel

(table_bytes(),
 round(count(v -> abs(decode(v)) == 1.0, act) / 4096; digits=3),
 length(unique(codepoint.(act))),
 decode(NextLessThan(one(Binary8p4se))))
```

```
(256, 0.384, 100, 0.9375)
# LUT size; fraction saturated to ±1; distinct output codes; last value below 1
```

Two things worth staring at. First, **saturation is severe at this input scale**: 38% of pre-activations land exactly on ±1 — under RNE, `tanh` reaches 1 as soon as the true value rounds past the midpoint of the last gap, and the entire saturated tail becomes indistinguishable to the next layer. (Under `TowardNegative` the asymptote is honored instead: `tanh` of any finite positive input projects to 0.9375, never 1 — the projection mode is part of the activation design space.) Second, the choice of nonlinearity changes *information retention* in code points, not just shape:

```
actS = Softplus(Binary8p4se, RNE_SN, pre)
(length(unique(codepoint.(actS))), table_bytes())
```

```
(61, 512)          # softplus keeps 61 distinct codes here; two LUTs now cached
```

Auditing an activation for a format is this cheap: enumerate all 256 inputs, look at the output histogram, count what survives.

Packing a quantized model

```
n = 1_000_000
model = [rawvalue(Binary5p2se, UInt8(rand(0:31))) for _ in 1:n]
pv = PackedVector(model)
(sizeof(model), sizeof(pv.data))
```

```
(1000000, 625000)   # 8 bits → 5 bits per value; indexing and vmap work directly on pv
```

Chapter 36

Verification Sessions

36.1 Verification Sessions

Internals-level recipes: pipeline introspection, exhaustive verification of your own code, κ measurement, and doctrine-compliant benchmarking. These use unexported names (`SmallFloats.round_to_precision`, `SmallFloats.project`, `SmallFloats.get_table`, `SmallFloats.order_key`, ...), which are stable enough to learn from but are not covered by semantic-versioning guarantees.

Outputs are captured from real sessions.

Basic

Watching RoundToPrecision work

`round_to_precision` returns the draft's exact `(sign, S, Q)` decomposition before saturation ever sees it:

```
using SmallFloats
using SmallFloats: round_to_precision

r = round_to_precision(4, 8, NearestTiesToEven(), 2.30078125, 0, 0)   # P=4, bias=8
(r.sign, r.S, r.Q, r.S * 2.0^r.Q)
```

```
(1, 9, -2, 2.25)          # S·2^Q = 9/4: the nearest P=4 value
```

Symbolic sticky: projecting just below a number

The engine accepts `sticky ∈ {-1, 0, +1}` meaning "the true value is the carrier plus an infinitesimal of this sign". This is how enclosure endpoints and asymptotes (`tanh → 1⁻`) are projected exactly:

```
using SmallFloats: project
project(Binary8p4se, ProjSpec(TowardNegative(), SatNone()), 1.0; sticky=-1)
```

```
Binary8p4se(0.9375 ≡ 0x3f)     # == NextLessThan(1.0): the engine crossed the binade
```

Tables are the scalar path, memoized

```
using SmallFloats: get_table
empty_tables!()
tbl = get_table(:Exp, Binary8p4se, Binary8p4se, RNE_SN)
(tbl[Int(0x45) + 1],
 codepoint(Exp(Binary8p4se, RNE_SN, rawvalue(Binary8p4se, 0x45))),
 table_bytes())
```

```
(0x52, 0x52, 256)              # table entry ≡ scalar result; one 256-byte table cached
```

Order keys

Ordering is integer arithmetic: a sign-magnitude fold, monotone with the total order, NaN at the bottom (key 0). This is what comparisons and the O(n) counting sort run on:

```
using SmallFloats: order_key
[(v, order_key(v)) for v in (Binary8p4se(-1.0), Binary8p4se(0.0),
                             Binary8p4se(1.0), Binary8p4se(NaN))]
```

```
-1.0 → 64      0.0 → 129      1.0 → 193      NaN → 0
```

General AI

Ranking safety: an exhaustive monotonicity audit of score conversion

Decision and ranking systems consume scores through *order*. If scores are produced in one format and compared in another, the conversion must be monotone — and formats are small enough to prove it by enumeration rather than trust it. Walk every `Binary8p3se` datum in total order and check that its `Binary8p4se` image never goes backward on the integer order keys:

```
using SmallFloats
using SmallFloats: order_key

function monotone_conversion(::Type{From}, ::Type{To}) where {From,To}
    prev = nothing
    U = codeunit_type(From)
    values = [rawvalue(From, U(c))
              for c in UInt64(0):((UInt64(1) << bitwidth(From)) - 1)]
    for v in sort(values)
        isnan(decode(v)) && continue
        g = Convert(To, RNE_SN, v)
        prev !== nothing && order_key(g) < order_key(prev) && return false
        prev = g
    end
    true
end
monotone_conversion(Binary8p3se, Binary8p4se)
```

```
true
```

Run this for every `(From, To, ρ)` triple your system actually uses; it is a few hundred integer comparisons per triple. A ranking pipeline whose conversions all pass this audit cannot invert a preference by changing formats — a guarantee no amount of spot-testing provides.

κ-safe decision margins for approximate evaluators

Search under a compute budget often wants a cheap, approximate evaluation function. The κ registry turns "how approximate?" into a *measured* code-point bound — and code-point bounds compose into a decision rule: **if two defined evaluations differ by more than 2κ code points along the total order, the approximate evaluator cannot invert their comparison.** Verify the rule exhaustively for a κ = 2 evaluator:

```
using SmallFloats: order_key

fast(x) = (r = Exp(Binary8p4se, RNE_SN, x);
           isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)
κ, exhaustive = measure_kappa(fast, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN)

function margin_audit(fast, κ)
    codes = [rawvalue(Binary8p4se, UInt8(c)) for c in 0:255]
    safe = violations = 0
    for a in codes, b in codes
        da, db = Exp(Binary8p4se, RNE_SN, a), Exp(Binary8p4se, RNE_SN, b)
        (isnan(decode(da)) || isnan(decode(db))) && continue
        codedistance(da, db) > 2κ || continue
        safe += 1
        (order_key(fast(a)) < order_key(fast(b))) ==
            (order_key(da) < order_key(db)) || (violations += 1)
    end
    (safe, violations)
end
(κ, exhaustive, margin_audit(fast, κ)...)
```

```
(2.0, true, 49522, 0)
```

49,522 operand pairs clear the 2κ margin, and the approximate evaluator agrees with the defined ordering on every one of them — zero violations, exhaustively. Inside the margin, comparisons are genuinely undecidable at this κ; a search can treat sub-margin comparisons as ties to expand, or escalate those few nodes to the exact evaluator. Either way the pruning is *provably* sound, with κ measured at registration rather than promised.

Machine Learning

Verifying a quantizer exhaustively against an independent reference

Formats are small enough that "test on a few points" is never necessary. Here every in-range datum of `Binary8p3se` is projected into `Binary8p4se` and checked against a brute-force nearest search under 256-bit arithmetic (distance agreement; the draft's tie rule is the projection engine's job and is pinned elsewhere in the suite):

```
using SmallFloats

function ref_nearest_distance(::Type{T}, x) where {T}
    U = codeunit_type(T)
    last = (UInt64(1) << bitwidth(T)) - 1
    fins = [v for v in (rawvalue(T, U(c)) for c in UInt64(0):last)
            if isfinite(decode(v))]
    minimum(abs(setprecision(() -> BigFloat(decode(v)) - BigFloat(x), BigFloat, 256))
            for v in fins)
end

function verify_quantizer()
    ok = true
    for c in 0x00:0xff
        x = decode(rawvalue(Binary8p3se, c))
        (isfinite(x) && abs(x) <= decode(MaxFiniteOf(Binary8p4se))) || continue
        got = Convert(Binary8p4se, RNE_SN, x)
        ok &= abs(decode(got) - x) == Float64(ref_nearest_distance(Binary8p4se, x))
    end
    ok
end
verify_quantizer()
```

```
true
```

This pattern — enumerate the inputs, compare against an independently written reference — is how the entire package is tested, and it is available to *your* quantization code at trivial cost.

The κ workflow, including the part where it says no

Register an approximate kernel and the registry measures it; understate the bound and it refuses:

```
step2(x) = (r = Exp(Binary8p4se, RNE_SN, x);
            isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)

measure_kappa(step2, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN)
```

```
(2.0, true)               # κ = 2 code points, verified over all 256 inputs
```

```
register_approx!(:cheater, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN, step2; κ=1)
```

```
ERROR: ArgumentError: declared κ = 1.0 understates measured κ = 2.0 — registration rejected
```

Exporting the conformance declaration

```
d = conformance_dict()          # plain nested Dict{String,Any}
(d["package"], length(d["formats"]), length(d["operations"]),
 [s["op"] for s in d["cached_specializations"]])
```

Serialize d with any JSON/TOML writer to attach a machine-readable conformance statement to experiment artifacts.

Deep Learning

Proving your fused kernel exact: BlockDotProduct vs 512-bit truth

The block layer promises *one* projection with exact lane arithmetic. Trust, then verify — against big-float truth, over random blocks with mixed scales and an honest special-value mix:

```
using SmallFloats, Random

rng = Xoshiro(5)
mk() = Block(Binary8p1uf(2.0^rand(rng, -3:3)),
             ntuple(_ -> rawvalue(Binary8p4se, UInt8(rand(rng, 0:255))), 32))

function verify_dots(trials)
    for _ in 1:trials
        bx, by = mk(), mk()
        lx = [decode(bx.s) * decode(v) for v in bx.x]
        ly = [decode(by.s) * decode(v) for v in by.x]
        (any(!isfinite, lx) || any(!isfinite, ly)) && continue
        truth = setprecision(() -> sum(BigFloat(lx[i]) * BigFloat(ly[i]) for i in 1:32),
                             BigFloat, 512)
        got = BlockDotProduct(Binary8p4se, RNE_SN, bx, by)
        ref = setprecision(() -> SmallFloats.project(Binary8p4se, RNE_SN, truth), BigFloat, 512)
        codepoint(got) == codepoint(ref) || return false
    end
    true
end
verify_dots(200)
```

```
true
```

The same shape verifies any custom fused kernel you write: compose the truth in `BigFloat`, project once, compare code points.

Stochastic rounding, audited: the full-R sweep

Every stochastic projection is a deterministic function of the draw R, so its *distribution* is checkable exactly. For `x = 2 + 3/64` in `Binary8p4se` the fraction is ν = 3/16 of an ulp, so `StochasticA{4}` must round up for exactly 3 of the 16 draws:

```
σ4 = RSA_SN(4)                  # ≡ ProjSpec(StochasticA{4}(), SatNone())
x = 2.0 + 3/64
count(decode(SmallFloats.project(Binary8p4se, σ4, x; R)) == 2.25 for R in 0:15)
```

```
3
```

Sweeping R like this turns "is my stochastic pipeline unbiased?" from a statistical question into an exhaustive one — the pattern the shipped test suite uses.

An accelerator-style activation, κ-measured: hard-tanh vs Tanh

Hardware activation units often ship piecewise approximations. The κ machinery makes the substitution honest: measure the approximation's worst code-point deviation from the defined activation, exhaustively, and register it under that measured bound. Hard-tanh — `clamp(x, −1, 1)` — as a stand-in for Tanh:

```
one4 = Binary8p4se(1.0)
hardtanh(x) = Clamp(Binary8p4se, RNE_SN, x, Negate(one4), one4)

κ, exhaustive = measure_kappa(hardtanh, :Tanh, Binary8p4se, (Binary8p4se,), RNE_SN)
(κ, exhaustive)
```

```
(4.0, true)                # worst deviation: 4 code points, verified on all 256 inputs
```

Where is it worst? At `x = 1.0`: hard-tanh returns 1.0 while the defined `tanh(1.0)` is 0.75 — four code points along the total order. The registration round-trip, including the conformance surface:

```
impl = register_approx!(:hardtanh_act, :Tanh, Binary8p4se, (Binary8p4se,),
                        RNE_SN, hardtanh; κ=4)
(kappa(:hardtanh_act), kappa_measured(impl), :hardtanh_act in list_approx())
```

```
(4.0, 4.0, true)           # declared = measured; conformance_report() now lists it
```

(`unregister_approx!(:hardtanh_act)` removes it.) Declaring κ=3 would be *rejected* — understatement is impossible by construction. The decision of whether a κ = 4 activation is acceptable now belongs to your accuracy budget, not to hope: combined with the margin rule from the General AI section, any two pre-activations whose defined Tanh images sit more than 8 code points apart keep their order under hard-tanh, provably.

Benchmarking without measuring the dispatcher

The package's benchmark doctrine, in one snippet (needs the `benchmarking/` environment for Chairmarks). Format types enter as **type parameters**; operands come from Chairmarks' *untimed* `setup`; and if you retrieve functions reflectively (`getfield(SmallFloats, op)`), pass them through an argument barrier so they specialize:

```
using Chairmarks, SmallFloats, Random
using Statistics: median

function bench_add(::Type{T}) where {T<:Binary}          # T: type parameter, not a global
    U = codeunit_type(T)
    last = (UInt64(1) << bitwidth(T)) - 1
    pool = [rawvalue(T, U(rand(UInt64(0):last))) for _ in 1:4096]
    @be (rand(pool), rand(pool)) (t -> Add(T, RNE_SN, t[1], t[2]))(_)
end

b = bench_add(Binary8p4se)
(round(median(b).time * 1e9; digits=1), median(b).allocs)
```

On the host recorded in the benchmark report, the corresponding in-domain Add row has a 9.0 ns median and zero allocations. Treat that as a check on the methodology, not a portable expected value: the tuple printed by this snippet depends on your CPU, Julia build, and benchmark environment.

The 60× misapplication

The same call with T read from a non-`const` global measures ~1 µs — Julia's dynamic keyword dispatch, not this package. Two project post-mortems trace to exactly this mistake; the shipped `benchmarking/benchmarking.jl` asserts specialization (zero warm-path allocation) before it believes any number, and so should yours.

Checking the code-point algebra against the package

The identities on Formal Foundations are claims about every format, so they are checked over every format — the distinguished codes for all 504, and the decoding formula at every positive finite code point of each.

Two things this exercise teaches beyond the identities themselves. First, the combined-exponent rule: computing `(L+t)·2^(j−1)·η` as two multiplications overflows `Float64` at the wide formats, and the first run of this script reported 1405 "defects" that were all in the checking arithmetic. Second, the oracle has to have the range of the thing it is checking — `BigFloat` with one `ldexp` does; `Float64` does not.

```
using SmallFloats, SmallFloats.Formats
using SmallFloats: _NAMED, nan_code, posinf_code, MaxFiniteOf, rawvalue,
                   codeunit_type, codepoint

bad = 0
for (nm, T) in sort(collect(_NAMED); by = first)
    K, P = bitwidth(T), precision(T)
    S, D = issigned(T) ? 1 : 0, isextended(T) ? 1 : 0
    Q, H, L = 1 << K, 1 << (K - 1), 1 << (P - 1)
    B = 1 << (K - P - S)
    N = Q - 1 - S * (H - 1)                     # the NaN anchor
    M = N - 1 - D                               # max positive finite code

    B == expbias(T)               || (bad += 1)
    N == Int(nan_code(T))         || (bad += 1)
    M == Int(codepoint(MaxFiniteOf(T))) || (bad += 1)
    D == 1 && (N - 1 == Int(posinf_code(T)) || (bad += 1))

    setprecision(BigFloat, 256) do
        for c in 1:M                            # every positive finite code
            v = decode(rawvalue(T, codeunit_type(T)(c)))
            want = c < L ? ldexp(BigFloat(c), 2 - P - B) :
                           ldexp(BigFloat(L + (c % L)), (c ÷ L) - 1 + 2 - P - B)
            BigFloat(v) == want || (bad += 1)
        end
    end
end
bad
```

```
0        # 504 formats, every distinguished code, every positive finite code point
```

Chapter 37

Performance Sessions

37.1 Performance Evidence

Generated: 2026-07-30 12:32 UTC · Julia 1.12.6 · alderlake (32 logical CPUs, 1 Julia thread) · Float128 paths: enabled · Chairmarks 1.3.1

Reference format for per-operation tables: `Binary8p4se` under `(NearestTiesToEven, SatNone)`. Every table names its operand class: the scalar-operation tables appear in four variants — all code points (NaN and ±Inf sampled), NaN excluded, finite-only, and per-operation in-domain — and every other sampled table uses the all-code-points pool, identified in its note. Times are per call; minima first, medians alongside. Methodology per the recorded benchmark doctrine: type-parameterized barriers, untimed setup, specialization preflight.

Core primitives

The decode/compare/step/classify layer plus the projection engine. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`decode`	1.6 ns	1.6 ns	0
`order_key`	1.6 ns	1.8 ns	0
`x < y`	1.8 ns	1.8 ns	0
`TotalOrder`	2.1 ns	2.1 ns	0
`Class`	1.6 ns	2.5 ns	0
`NextGreaterThan`	1.8 ns	2.1 ns	0
`project (RNE·SatNone)`	2.2 ns	6.7 ns	0
`project (StochasticA[8], R drawn)`	2.1 ns	7.1 ns	0

Scalar operations

Each arity is measured under four operand classes — safe args, no NaN/Inf, no NaN, and all code points — so operand-class effects (NaN fast rows, ±Inf special rows) are separable.

Unary (30)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
Abs	4.9 ns	6.7 ns	0
Negate	4.8 ns	6.9 ns	0
Recip	7.9 ns	18.0 ns	0
Sqrt	5.7 ns	18.7 ns	0
RSqrt	9.6 ns	20.7 ns	0
Cosh	7.3 ns	21.4 ns	0
Sin	6.2 ns	21.7 ns	0
Cos	7.3 ns	21.7 ns	0
Exp2	7.3 ns	22.4 ns	0
Exp	7.4 ns	22.5 ns	0
ArcSin	5.7 ns	23.2 ns	0
ExpMinusOne	5.3 ns	23.2 ns	0
Tanh	5.8 ns	23.2 ns	0
ArcCos	5.7 ns	24.3 ns	0
ArcSinPi	5.5 ns	25.5 ns	0
Sinh	5.7 ns	25.6 ns	0
Log	5.5 ns	25.7 ns	0
Log2	5.9 ns	26.6 ns	0
Tan	5.3 ns	26.6 ns	0
LogOnePlus	5.4 ns	26.7 ns	0
ArcCosPi	6.1 ns	26.8 ns	0
CosPi	6.9 ns	27.8 ns	0
SinPi	5.6 ns	28.1 ns	0
ArcTan	5.2 ns	28.4 ns	0
ArcTanPi	5.1 ns	31.0 ns	0
ArcTanh	6.2 ns	31.1 ns	0
TanPi	6.2 ns	31.1 ns	0
ArcCosh	5.5 ns	33.3 ns	0
Softplus	13.8 ns	33.3 ns	0
ArcSinh	6.2 ns	33.4 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median. Transcendental rows mix special-row fast returns with enclosure-path evaluations, so these are *scalar-path* costs; bulk unary work routes through 256-byte tables (see Array kernels).

operation	min	median	allocs
Sqrt	1.9 ns	2.1 ns	0
ArcCosh	2.1 ns	2.1 ns	0
ArcSinPi	1.9 ns	2.2 ns	0
Abs	4.3 ns	6.7 ns	0
Negate	4.8 ns	6.9 ns	0
Log2	2.1 ns	8.6 ns	0
RSqrt	2.1 ns	9.5 ns	0
Recip	1.9 ns	17.8 ns	0
Cos	7.1 ns	21.2 ns	0
Sin	5.4 ns	21.5 ns	0
Cosh	7.0 ns	21.6 ns	0
Exp2	7.3 ns	22.3 ns	0
Exp	7.0 ns	22.4 ns	0
ExpMinusOne	4.5 ns	22.8 ns	0
ArcSin	2.3 ns	22.8 ns	0
Tanh	5.7 ns	23.1 ns	0
ArcCos	2.5 ns	23.8 ns	0
Log	2.3 ns	25.3 ns	0
Sinh	5.1 ns	25.6 ns	0
LogOnePlus	1.9 ns	26.3 ns	0
ArcCosPi	2.1 ns	26.5 ns	0
Tan	6.1 ns	26.6 ns	0
CosPi	6.4 ns	27.4 ns	0
SinPi	5.2 ns	27.7 ns	0
ArcTan	5.5 ns	28.3 ns	0
ArcTanh	2.1 ns	28.7 ns	0
TanPi	5.3 ns	30.8 ns	0
ArcTanPi	4.8 ns	30.8 ns	0
Softplus	14.3 ns	33.5 ns	0
ArcSinh	5.0 ns	33.6 ns	0

operation	min	median	allocs
ArcCosh	2.1 ns	2.1 ns	0
RSqrt	2.1 ns	2.2 ns	0
Log	2.3 ns	2.5 ns	0
ArcSin	2.5 ns	2.6 ns	0
Sqrt	1.9 ns	5.1 ns	0
Abs	4.4 ns	6.7 ns	0
Negate	4.4 ns	6.8 ns	0
ArcSinPi	2.0 ns	7.3 ns	0
Log2	2.1 ns	8.6 ns	0
Recip	1.9 ns	17.8 ns	0
Cosh	5.0 ns	21.2 ns	0
Sin	2.1 ns	21.4 ns	0
Cos	1.9 ns	21.5 ns	0
Exp2	5.5 ns	22.3 ns	0
Exp	5.0 ns	22.3 ns	0
Tanh	5.7 ns	23.0 ns	0
ExpMinusOne	4.6 ns	23.1 ns	0
ArcCos	2.6 ns	23.8 ns	0
Sinh	5.1 ns	25.6 ns	0
LogOnePlus	1.9 ns	26.2 ns	0
ArcCosPi	2.1 ns	26.6 ns	0
Tan	2.0 ns	27.2 ns	0
CosPi	2.5 ns	27.5 ns	0
SinPi	2.1 ns	27.7 ns	0
ArcTan	5.7 ns	28.3 ns	0
ArcTanh	2.1 ns	28.6 ns	0
ArcTanPi	4.7 ns	30.8 ns	0
TanPi	2.1 ns	30.8 ns	0
Softplus	4.8 ns	33.5 ns	0
ArcSinh	4.5 ns	33.6 ns	0

operation	min	median	allocs
Sqrt	1.9 ns	2.0 ns	0
ArcCosh	2.1 ns	2.1 ns	0
ArcSinPi	1.9 ns	2.1 ns	0
RSqrt	2.1 ns	2.1 ns	0
Log	2.3 ns	3.8 ns	0
Log2	2.1 ns	5.5 ns	0
ArcSin	2.1 ns	5.8 ns	0
ArcTanh	2.1 ns	6.6 ns	0
Abs	2.0 ns	6.7 ns	0
Negate	2.1 ns	6.8 ns	0
Recip	1.8 ns	17.8 ns	0
Cosh	2.3 ns	21.0 ns	0
Cos	1.9 ns	21.3 ns	0
Sin	2.1 ns	21.5 ns	0
Exp2	2.1 ns	22.3 ns	0
Exp	2.0 ns	22.3 ns	0
ExpMinusOne	2.0 ns	22.7 ns	0
Tanh	2.3 ns	23.1 ns	0
ArcCos	2.3 ns	23.7 ns	0
Sinh	2.3 ns	25.5 ns	0
LogOnePlus	1.8 ns	26.4 ns	0
Tan	2.0 ns	26.5 ns	0
ArcCosPi	1.9 ns	26.6 ns	0
CosPi	2.3 ns	27.5 ns	0
SinPi	2.0 ns	27.7 ns	0
ArcTan	2.2 ns	28.2 ns	0
ArcTanPi	2.1 ns	30.8 ns	0
TanPi	1.9 ns	30.8 ns	0
Softplus	2.0 ns	33.5 ns	0
ArcSinh	2.1 ns	33.6 ns	0

Binary (18)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.9 ns	7.1 ns	0
MaximumMagnitude	6.9 ns	7.1 ns	0
CopySign	4.8 ns	7.2 ns	0
Maximum	4.5 ns	7.2 ns	0
Minimum	4.6 ns	7.2 ns	0
MaximumFinite	4.8 ns	7.2 ns	0
MinimumFinite	4.9 ns	7.3 ns	0
MinimumNumber	4.8 ns	7.3 ns	0
MaximumNumber	5.0 ns	7.3 ns	0
MaximumMagnitudeNumber	7.1 ns	7.3 ns	0
MinimumMagnitudeNumber	4.8 ns	7.3 ns	0
Multiply	4.9 ns	7.4 ns	0
Add	7.1 ns	9.0 ns	0
Subtract	7.4 ns	9.2 ns	0
Divide	6.4 ns	18.3 ns	0
Hypot	7.8 ns	28.0 ns	0
ArcTan2	8.0 ns	32.7 ns	0
ArcTan2Pi	7.2 ns	35.3 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.8 ns	7.1 ns	0
MaximumMagnitude	6.9 ns	7.1 ns	0
Minimum	4.7 ns	7.2 ns	0
Maximum	4.6 ns	7.2 ns	0
CopySign	4.7 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MaximumFinite	5.1 ns	7.2 ns	0
MinimumNumber	6.1 ns	7.3 ns	0
MaximumNumber	4.8 ns	7.3 ns	0
MinimumMagnitudeNumber	4.8 ns	7.3 ns	0
MaximumMagnitudeNumber	7.2 ns	7.3 ns	0
Multiply	4.9 ns	7.4 ns	0
Add	7.1 ns	9.0 ns	0
Subtract	7.9 ns	9.5 ns	0
Divide	2.6 ns	18.3 ns	0
Hypot	8.2 ns	28.0 ns	0
ArcTan2	9.0 ns	32.7 ns	0
ArcTan2Pi	7.1 ns	35.4 ns	0

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	4.6 ns	7.1 ns	0
MaximumMagnitude	5.0 ns	7.1 ns	0
CopySign	5.0 ns	7.2 ns	0
Minimum	4.6 ns	7.2 ns	0
Maximum	4.6 ns	7.2 ns	0
MaximumFinite	5.0 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MinimumNumber	5.1 ns	7.3 ns	0
MaximumNumber	4.6 ns	7.3 ns	0
MinimumMagnitudeNumber	4.6 ns	7.3 ns	0
MaximumMagnitudeNumber	5.1 ns	7.3 ns	0
Multiply	5.0 ns	7.4 ns	0
Add	6.3 ns	9.0 ns	0
Subtract	7.2 ns	9.5 ns	0
Divide	2.6 ns	18.3 ns	0
Hypot	6.1 ns	28.0 ns	0
ArcTan2	7.1 ns	32.7 ns	0
ArcTan2Pi	6.7 ns	35.4 ns	0

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	min	median	allocs
MinimumMagnitude	2.1 ns	7.1 ns	0
MaximumMagnitude	1.9 ns	7.1 ns	0
CopySign	2.1 ns	7.2 ns	0
Minimum	2.1 ns	7.2 ns	0
Maximum	1.8 ns	7.2 ns	0
MaximumFinite	5.6 ns	7.2 ns	0
MinimumFinite	5.0 ns	7.2 ns	0
MinimumNumber	5.1 ns	7.3 ns	0
MaximumNumber	4.6 ns	7.3 ns	0
MinimumMagnitudeNumber	4.6 ns	7.3 ns	0
MaximumMagnitudeNumber	5.0 ns	7.4 ns	0
Multiply	2.1 ns	7.4 ns	0
Add	3.7 ns	9.0 ns	0
Subtract	4.6 ns	9.5 ns	0
Divide	2.5 ns	18.3 ns	0
Hypot	3.0 ns	28.0 ns	0
ArcTan2	4.6 ns	32.7 ns	0
ArcTan2Pi	3.8 ns	35.3 ns	0

Ternary (3)

Safe args

Finite operands within each operation's safe domain — the fully unmasked per-operation scalar cost. The argument-restricted ops (Sqrt, RSqrt, Log, Log2, LogOnePlus, Recip, Divide, ArcSin, ArcCos, ArcCosh, ArcTanh) draw from explicit per-argument safe-domain predicates; every other op uses finite operand tuples whose defined result is not NaN (oracle-derived). Sorted by median.

operation	min	median	allocs
`Clamp`	5.1 ns	8.0 ns	0
`FMA`	8.1 ns	9.6 ns	0
`FAA`	8.3 ns	9.9 ns	0

No NaN, Inf args

Operands exclude NaN and ±Inf; finite datums only (zeros and subnormals kept). Domain-restricted ops still take NaN fast rows on out-of-domain finite operands. Sorted by median.

operation	min	median	allocs
`Clamp`	5.5 ns	8.0 ns	0
`FMA`	8.3 ns	9.7 ns	0
`FAA`	8.1 ns	9.7 ns	0

No NaN args

Operands exclude the NaN code point; ±Inf and every finite datum are sampled. Sorted by median.

operation	min	median	allocs
`Clamp`	5.2 ns	8.0 ns	0
`FMA`	6.9 ns	9.7 ns	0
`FAA`	7.1 ns	9.7 ns	0

All code points

Operands drawn uniformly over ALL code points — NaN and ±Inf are sampled; medians of domain-restricted ops are diluted by instant NaN rows. Sorted by median.

operation	min	median	allocs
`Clamp`	2.1 ns	8.0 ns	0
`FMA`	3.7 ns	9.7 ns	0
`FAA`	4.1 ns	9.7 ns	0

Sensitivity studies

How the scalar costs move with the format and with the projection specification.

Format sensitivity

Same three binary ops across formats; `Binary8p1uf` exercises the wide-exponent-spread escalations, small-K formats the tiny-table regime. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`Add(Binary8p4se)`	3.7 ns	9.0 ns	0
`Divide(Binary8p4se)`	2.5 ns	18.3 ns	0
`Multiply(Binary8p4se)`	2.1 ns	7.4 ns	0
`Add(Binary8p3sf)`	3.9 ns	8.7 ns	0
`Divide(Binary8p3sf)`	2.6 ns	16.4 ns	0
`Multiply(Binary8p3sf)`	2.1 ns	7.2 ns	0
`Add(Binary8p1uf)`	4.7 ns	74.0 ns	0
`Divide(Binary8p1uf)`	2.6 ns	8.4 ns	0
`Multiply(Binary8p1uf)`	2.1 ns	6.9 ns	0
`Add(Binary5p2se)`	4.0 ns	9.1 ns	0
`Divide(Binary5p2se)`	2.5 ns	9.0 ns	0
`Multiply(Binary5p2se)`	1.9 ns	7.4 ns	0
`Add(Binary3p1se)`	4.0 ns	7.6 ns	0
`Divide(Binary3p1se)`	2.6 ns	6.3 ns	0
`Multiply(Binary3p1se)`	2.1 ns	5.9 ns	0

Projection by rounding/saturation mode

`project(Binary8p4se, ρ, x)` over the mode vocabulary (stochastic budgets N = 8). Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`NearestTiesToEven`	2.0 ns	6.7 ns	0
`NearestTiesToAway`	2.8 ns	7.7 ns	0
`TowardPositive`	3.0 ns	6.0 ns	0
`TowardNegative`	2.6 ns	5.9 ns	0
`TowardZero`	2.5 ns	5.3 ns	0
`ToOdd`	2.0 ns	6.6 ns	0
`StochasticA[8]`	2.0 ns	7.0 ns	0
`StochasticB[8]`	2.5 ns	7.1 ns	0
`StochasticC[8]`	2.3 ns	7.1 ns	0
`RNE · SatFinite`	2.1 ns	7.6 ns	0
`RNE · SatPropagate`	2.1 ns	7.2 ns	0

Kernels and storage

Array-shaped work: the table-gather and compute kernels, the ternary bitwidth policy, sorting, table builds, blocks, and packed/converted storage.

Array kernels (vmap)

Warm caches: table specializations prebuilt, so table rows measure the gather; scalar-loop rows measure the full compute pipeline per element. The ternary row here is `Binary8p4se` (K=8, always the compute path); see the next section for how the ternary bitwidth policy behaves across K. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per element
`vmap unary (table gather), n=65536`	8.56 µs	8.71 µs	0	0.13 ns/elem — 7.52 Gelem/s
`vmap binary (table gather), n=65536`	16.45 µs	16.81 µs	0	0.26 ns/elem — 3.9 Gelem/s
`vmap ternary (scalar loop), n=65536`	1.01 ms	1.02 ms	0	15.54 ns/elem — 0.06 Gelem/s
`vmap binary stochastic (scalar loop), n=65536`	646.57 µs	655.78 µs	0	10.01 ns/elem — 0.1 Gelem/s
`vmap unary through PackedVector, n=65536`	76.97 µs	90.29 µs	773	1.38 ns/elem — 0.73 Gelem/s

Ternary bitwidth tiers (FMA/FAA)

`FMA`/`FAA`/`Clamp` are total functions on `2^(K1+K2+K3)` code points, but that count spans 512 B (K=3) to 16 MiB (K=8), so the array kernel tables small operand formats eagerly, tables mid-size ones adaptively (after enough elements amortize the build; not shown here — see the adaptive-cache gate in `test/ternary_opt.jl`), and always runs the scalar compute kernel at K=8, threaded above a size cutoff when `Threads.nthreads() > 1`. Each tier's optimized row is paired with a scalar-loop baseline (policy Refs forced off around the measurement, restored after) so the win is visible per tier; this process has 1 Julia thread. No threaded/sequential comparison below — rerun with `julia -t N` (N > 1) to see it. Same reference format, ρ, and operand pool discipline as Array kernels above.

operation	min	median	allocs	per element
`FMA K=4 (eager table), n=65536`	19.58 µs	20.46 µs	0	0.31 ns/elem — 3.2 Gelem/s
`FMA K=4 (eager table), scalar-loop baseline, n=65536`	967.88 µs	974.03 µs	0	14.86 ns/elem — 0.07 Gelem/s
`FMA K=6 (eager table), n=65536`	24.03 µs	24.34 µs	0	0.37 ns/elem — 2.69 Gelem/s
`FMA K=6 (eager table), scalar-loop baseline, n=65536`	1.0 ms	1.01 ms	0	15.46 ns/elem — 0.06 Gelem/s
`FMA K=8 (compute), n=65536`	990.69 µs	1.0 ms	0	15.31 ns/elem — 0.07 Gelem/s
`FMA K=8 (compute), scalar-loop baseline, n=65536`	990.11 µs	1.0 ms	0	15.27 ns/elem — 0.07 Gelem/s

Sorting (64 K values)

Counting sort is installed as the default algorithm for `Binary` vectors. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs
`sort! (counting sort via defalg), n=65536`	155.36 µs	157.4 µs	2
`sort! (stock comparison sort), n=65536`	1.36 ms	1.37 ms	3
`sort! rev=true (counting sort), n=65536`	155.48 µs	157.95 µs	2

Table builds (oracle + projection, Float128-first)

Cold cache per sample (`empty_tables!` in untimed setup); JIT pre-warmed. The warm-hit column is the steady-state cost of `get_table` when the specialization is already cached (min / median). Table entries enumerate every code point by construction (NaN and ±Inf included).

operation	min	median	allocs	warm hit
`Exp(8p4se) (256 entries)`	30.31 µs	31.16 µs	257	65.8 ns / 67.4 ns
`Tanh(8p4se) (256 entries)`	29.49 µs	30.93 µs	257	65.4 ns / 67.5 ns
`Add(8p4se×8p4se) (64 K entries)`	13.77 ms	13.93 ms	458754	65.0 ns / 66.6 ns
`Divide(8p4se×8p4se) (64 K)`	14.02 ms	14.14 ms	458754	65.1 ns / 66.6 ns
`Add(8p1uf×8p1uf) (64 K, wide-spread)`	27.08 ms	29.5 ms	995304	65.8 ns / 67.8 ns

Block and scaled operations

Elements `Binary8p4se`, scales `Binary8p1uf`, B = 32. Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per lane
`BlockAdd (B=32)`	963.7 ns	1.17 µs	35	36.65 ns/lane
`BlockDotProduct → 8p4se (B=32)`	211.3 ns	265.3 ns	3	8.29 ns/lane
`BlockReduceAdd → 8p4se (B=32)`	43.5 ns	489.4 ns	0	15.29 ns/lane
`ConvertToBlockMaxAbsFinite (B=32)`	3.79 µs	3.95 µs	111	123.3 ns/lane

Conversions and packed storage

Operands: all code points — NaN and ±Inf sampled.

operation	min	median	allocs	per element
`T(::UInt8) code-point constructor`	1.8 ns	1.8 ns	0	
`rawvalue (unchecked kernel route)`	1.8 ns	1.8 ns	0	
`T(::Float64) numeric constructor (projects)`	3.2 ns	7.7 ns	0	
`Convert 8p4se → 8p3se (scalar)`	2.1 ns	6.7 ns	0	
`Float64 → 8p4se (project)`	2.0 ns	6.7 ns	0	
`PackedVector pack, n=65536`	33.47 µs	35.43 µs	3	0.54 ns/elem
`PackedVector unpack (collect), n=65536`	28.86 µs	29.05 µs	3	0.44 ns/elem

All numbers from this machine/run; absolute values vary by host. Regenerate with `julia --project=benchmarking benchmarking/benchmarking.jl benchmarking/benchmark_report.md`.

Part VII

Implementation

Chapter 38

Architecture and Invariants

38.1 Architecture and Invariants

The implementation is organized around two independent classifications: the carrier rung needed to hold a format exactly and the rigor class attached to a computed result. The remaining internal pages follow values through those classifications and into the one projection path.

The first axis is the **carrier rung** a format's datums decode to, chosen by the format's exponent bias: `Float64` for the 432 formats whose full dynamic range fits Float64's normal range (rung 1), `Float128` for formats that need more exponent room but stay in binary floating point (rung 2), and an exact dyadic carrier for the formats — 72 of them — whose range or precision breaks even Float128 (rung 3). The choice is a dispatch decision derived from the exponent bias, never a caller's, and every rung is exact for the datums it carries. The second axis is the **rigor class** attached to every non-`Float64` computed result: Class R (unconditional — provable without any accuracy assumption) and Class E (envelope-conditional — faithful-but-not-CR estimates validated by a generous, measured slack). Together the two axes answer, for any operation on any format, "what carries the value" and "what justifies the result."

The pages that follow work through the machinery each axis produces: the encoding and projection engine that every carrier rung feeds into (Encoding and Decoding), the oracle and its two rigor classes (Oracle and Rigor Classes), the table and kernel layer that makes the common rung fast (Function Tables and Array Kernels), the block layer's exactness argument (Exact Block Reductions), and the Verification Strategy that keeps all of it honest.

Layer map

Source files load in dependency order, each layer speaking only downward:

layer	file	provides
formats	`formats.jl`	`Binary{K,P,SGN,EXT}`, the 504 named aliases, Group M queries
specs	`projspec.jl`	rounding/saturation singletons, `ProjSpec{R,S}`, the predefined spec grid
defaults	`defaults.jl`	settable session defaults (`DefaultType`, `DefaultProjection`, …) behind `Refs`; consumed via the speculation guard
codec	`decode_encode.jl`	decode (generated tables + bit-composed compute), encode, order keys, counting sort, `Class`, Next ops
engine	`project.jl`	`round_to_precision` (mask-based Float64 core + generic core), `saturate`, `project`, `project_interval`
ops	`ops_scalar.jl`	result-kind protocol, `apply_op`, the operation registry, the spec register
compat	`juliacompat.jl`	the Base register: declarative op ⇒ Base-function map
oracle	`oracle.jl`	ω-semantics for all 52 operations
tables	`tables.jl`	the pure-ρ result-table cache (unary/binary + bitwidth-gated ternary)
kernels	`kernels.jl`	Shape-A gathers, Shape-B scalar/threaded loops, `vmap`
blocks	`blocks.jl`	`Block`, block/scaled ops, exact reductions
packed	`packed.jl`	sub-byte `PackedVector` storage
approx	`approx.jl`	κ measurement/registry, conformance declaration
rand	`rand.jl`	Random-API hooks: uniform-[0,1) floor projection, clamped normal

Continue through the implementation

Encoding and Decoding, Oracle and Rigor Classes, Function Tables and Array Kernels, Exact Block Reductions, Verification Strategy.

Chapter 39

Encoding and Decoding

39.1 Encoding and Decoding

The public promise is simple: a code point identifies one exact datum, and decoding that identity never rounds. The implementation chooses different carriers and decode strategies by format, but those choices may change cost, never meaning.

Invariant

For every valid code point `c` of a concrete representation `T`:

```
codepoint(rawvalue(T, c)) == c
```

and `decode(rawvalue(T, c))` equals the P3109 datum assigned to `c`. For every non-NaN datum, encoding the exact decoded value returns the corresponding code point under an exact-grid projection.

Representation and storage

`Binary{K,P,SGN,EXT}` describes the abstract format. A concrete alias such as `Binary8p4se` supplies the machine representation. Code points use `UInt8` at $K \le 8$ and `UInt16` above that split; unused high bits are not part of the format's identity.

The bit layout follows the draft: sign where present, biased exponent, and trailing significand. The negative-zero slot names the format's single NaN; there is one zero rather than signed zeros.

Decode policies

`decodepolicy(T)` selects an implementation shape.

For $K \le 8$, decoding uses a generated constant tuple containing all datums. The tuple is built from the computational decoder, so lookup and computation have one source of meaning. A runtime decode becomes one indexed load, while a constant input can fold completely.

Above $K = 8$, materializing a 65,536-entry constant for every format is not a useful trade. The computational decoder runs directly.

Datum carriers

The decoded datum must be exact even when its magnitude exceeds `Float64`. `datumcarrier(T)` selects among three carrier rungs from format properties:

- rung 1 uses `Float64` where its normal range and precision cover every datum;

- rung 2 uses `Float128` when greater exponent room is required;
- rung 3 uses an exact dyadic carrier when a hardware-style binary float cannot cover the format exactly.

The caller does not select the datum carrier. The invariant is stronger than “most values fit in Float64”: every selected carrier is exact for every datum of its format.

Ordering keys

Comparisons and counting sort use an unsigned integer key obtained by a sign-magnitude fold. Key zero is reserved for the single NaN, which P3109's total order places below the other datums. Every ordinary datum key is at least one.

The key type has room for the entire code lattice plus the NaN sentinel. The mapping is monotone, so comparison and sort can operate on integers without decoding floating values.

Source map

- format parameters, representations, masks, and carrier traits: `src/formats.jl`;
- computational decoding and order keys: `src/decode_encode.jl`;
- wide exact datum carriers: `src/dyadic.jl`, `src/dyadic3.jl`, and carrier support files;
- constructor normalization and representation aliases: format and conversion entry points in `src/`.

Evidence

The verification suite compares computational decoding against an independent big-float transliteration over the supported lattice. It also checks lookup versus computation, round trips, carrier widening, ordering monotonicity, and the format/code-point algebra.

Extension seam

A new representation or carrier policy must preserve the public invariant and add equivalence tests against an existing wider path. Do not make a carrier choice from runtime caller preference; that would turn representation into semantics.

Next

Projection Engine.

Chapter 40

Projection Engine

40.1 Projection Engine

Projection is the only write path into a code point. Constructors, conversions, operations, table construction, and block results all converge here.

```
exact result → RoundToPrecision → Saturate → Encode
```

Public promise

Given a result format, projection specification, and exact mathematical value, the engine returns the P3109-defined code point. No faster path is allowed to substitute a second rounding routine.

RoundToPrecision

The rounding stage produces a canonical `Rounded(kind, sign, S, Q)`. `S` and `Q` describe a scaled significand exactly; `kind` distinguishes finite and special results.

Two implementations cover the supported carriers:

- the generic core scales an exact or enclosed carrier and compares the discarded fraction with the selected mode's decision points;
- the `Float64` mask core extracts sign, exponent, and significand fields and performs the same decisions with integer operations.

The mask core is an optimization only. Specials, subnormal carrier inputs, or cases outside its proof obligations fall back to the generic core. Equivalence tests establish that the two implementations choose the same rounded result.

Symbolic sticky

An interval or asymptotic computation sometimes knows that the true value is immediately above or below an exactly representable endpoint without possessing a distinct carrier value for that infinitesimal displacement.

`sticky ∈ {-1,0,+1}` carries that relation into every rounding comparison. The delicate “just below a datum” case crosses into the preceding binade and uses a fraction of 1^{-}; treating it as the endpoint itself would give the wrong answer for directed modes and ties.

Saturate

The saturation stage classifies the rounded result against the result format's range and applies the draft's saturation rows. A disposition can be:

- as-is;
- minimum or maximum finite;
- positive or negative infinity;
- NaN.

Classification uses canonical integer fields rather than an inexact carrier comparison. Signed and unsigned formats, finite and extended domains, rounding direction, and special operation results all participate in the row choice.

Encode

Encoding assembles the final code point with integer operations. It handles significand carry, normal/subnormal boundaries, exponent fields, special encodings, and the single-zero rule. Encode does not make another numerical decision; all policy has already been resolved.

Source map

- projection orchestration and saturation: `src/project.jl`;
- rounding cores and rounded representation: the rounding implementation files under `src/`;
- operation-to-exact-result dispatch: `src/oracle.jl` and operation files;
- encoding and raw value construction: `src/decode_encode.jl`.

Evidence

Required gates include deterministic and stochastic rounding predicates, generic/mask equivalence, boundary saturation rows, carrier escalation, symbolic-sticky cases, and exhaustive code-point results where the input space is affordable. Array tables are compared entry-for-entry with this scalar path.

Extension seam

An optimization may bypass work only after proving that it returns the same canonical rounded form or final code point. A new rounding or saturation mode belongs in the shared `ProjSpec` machinery and its exhaustive predicate tests, not in an operation-specific shortcut.

Next

Oracle and Rigor Classes.

Chapter 41

Oracle and Rigor Classes

41.1 Oracle and Rigor Classes

Every operation's defined result is computed by `ωeval`, which returns one of five result kinds; `apply_op` fast-splits the common one and finishes the rest:

kind	meaning	finished by
`Float64`	exact (specials; representable arithmetic)	direct `project`
`Dyadic`	exact at rung 3: an `Int128` significand and an `Int64` exponent, `isbits`	direct `project` (dyadic carrier)
`Float128`	exact by **width analysis**	direct `project` (Float128 carrier)
`StickyF`	wide-spread FMA/FAA tail: head value (`Float64` or `Float128`) + tail sign	direct `project` with `sticky` set — no allocation
`BigExactF`	exact at a precision **derived from the operands** — `bigprec`, not a constant (wide-spread tail for Add; the near-impossible FAA distillation miss)	`project` on `BigFloat`
`Enclose128`	correctly-rounded Float128 **bracket**	sticky agreement, MPFR fallback
`EncloseF`	MPFR directed enclosure `f(prec)`, optional Float128 pre-filter `fq`, optional eager Float64 estimate `yd`	three-stage: `yd` → `fq` → interval protocol

The Dyadic↔Rational bridge

The dyadic carrier converts to and from `Rational` **exactly**, and that bridge is the cleanest way to reason about a rung-3 value:

```
using SmallFloats.DyadicNumbers: dyadic_to_rational, rational_to_dyadic, isdyadic

dyadic_to_rational(x)          # Rational{BigInt}, exact — a Dyadic IS S·2^Q
rational_to_dyadic(3 // 4)     # exact, or throws
isdyadic(1 // 3)               # false — no Dyadic represents it
```

Three properties are worth knowing. The infinities convert (`±1//0`, matching `Rational{BigInt}(Inf)`) and NaN does not, because `Rational` has no NaN slot. A non-dyadic rational is **refused rather than rounded** — rounding it here would be a rounding outside `project`. And the round trip is a law about *values*, not fields: `Dyadic`'s significand is deliberately unnormalized, so $6\cdot2^{-2}$ and $3\cdot2^{-1}$ are the same value and round-trip to the canonical one.

The full design, with every edge case and the three laws, is in `other/dyadic_rational.md`.

The two rigor classes

Two **rigor classes** govern every non-`Float64` path, and their arguments are never mixed:

Class R (unconditional) rests on two facts that hold without any accuracy assumption:

- **Width analysis.** Sums of decoded datums are *exactly representable* in `Float128` whenever operand bits + exponent spread fit 113 bits — checked in advance by integer exponent arithmetic (`Add` at $\Delta E \le 100$, `FMA` ≤ 92, `FAA` span ≤ 98). Beyond the threshold, `Add` escalates to the exact MPFR path — at a precision **derived from the operands** by `bigprec`, not the retired 2200-bit constant, which was ample at $K \le 8$ and insufficient for 50 of the 504 formats (gate G2 measures this) — while `FMA`/`FAA` take a non-allocating **sticky-head** shortcut: past that spread the smaller term is provably too small (by construction of the threshold) to affect anything but the tail direction of the larger one, so the result is `StickyF(head, sign)` — no `BigFloat` involved. `FAA`'s three-term case runs a bounded Float128 2sum-distillation (Priest-style, ≤ 6 sweeps) to find that head/tail split directly; the residual `BigExactF` MPFR fallback exists only for the near-impossible case the distillation doesn't converge.
- **IEEE correct rounding.** Division and `sqrt` are correctly rounded at *every* binary width, by mandate. `Divide`/`Recip` therefore use the **Float64** quotient directly: it is within half an ulp of truth, so it serves as `EncloseF`'s eager `yd` estimate with no Float128 arithmetic on the path at all. `Sqrt`/`RSqrt` use the **Float128** CR result: an inexact nearest-CR value `q` brackets the truth in the *open* interval `(prevfloat(q), nextfloat(q))`. Exactness itself is detected by an `fma` residual test.

Class E (envelope-conditional). Faithful-but-not-CR evaluations stand in for an enclosure only inside a generous envelope, resolved by the two-sided sticky gate. Two stages, cheapest first:

- *Float64 stage:* for operations whose Float64 libm is faithful (≤ 1 ulp $\approx 2^{-52}$ relative — the exp/log families, the hyperbolics, forward trig inside the $|x| \le 10^{15}$ reduction window, `atan`, `asinh`, the softplus composition), the eager estimate `yd` carries envelope $E = |yd| \cdot 2^{-45}$, $\ge 2^{7}$ slacker than any faithful-libm error (measured margin on this machine: $\ge 2^{6.6}$).
- *Float128 stage:* libquadmath's estimate `y = fq()` carries $E = |y| \cdot 2^{-90}$, at least 2^{18} slacker than any published libquadmath bound.

In both stages, if the sticky-projected endpoints agree, that code point is the answer; if not, the next stage (ultimately the MPFR ladder) decides. An envelope failure can therefore only cost speed, never correctness — unless both endpoints agree *and* the envelope is wrong, which the differential-build tests rule out empirically by building every standard table twice (Float128-first and pure-MPFR) and byte-diffing, and which the exhaustive oracle cross-check re-verifies per operation against the 3072-bit protocol.

The interval protocol

The interval protocol (`project_interval`) is the termination backbone: evaluate the MPFR enclosure at precision `p`; if `lo == hi` the value is exactly representable — project it; otherwise project `lo` with `sticky = +1` and `hi` with `sticky = −1`; agreement (projection is monotone at fixed `R`) yields the answer; disagreement doubles `p`, clamped to the `maxprec` ceiling (**derived from the format**, `max(4096, bigprec(T) + 64)` — a flat 4096 was ample at $K \le 8$ and could not decide `ArcCosPi` at `Binary15p2ue` under a directed or stochastic rule, honored exactly even when it is not a power-of-two multiple of the 256-bit start). Termination requires that the enclosure not be chasing a value the grid can actually hit — which is why the π-scaled operations carry **Niven peels**: for dyadic arguments, `tan(πr)` takes rational values only at the quarter-integers (± 1) and `atan2/π` only on the diagonals ($\pm ¼$, $\pm ¾$); those cases are answered exactly before any enclosure is built, and Niven's theorem proves the peel set complete.

Source and gates

The operation registry and ω evaluation live in `src/oracle.jl`; exact dyadic carriers and their rational bridge live in `src/dyadic.jl` and `src/dyadic3.jl`. Changes here must preserve enclosure containment, termination, and agreement with a wider independent path. The gate and tier files under `test/` record whether each check is exhaustive or sampled.

Continue through the implementation

Encoding and Decoding, Function Tables and Array Kernels, Add an Operation.

Chapter 42

Function Tables and Array Kernels

42.1 Function Tables and Array Kernels

Function Tables

For pure specs, unary and binary operations are **finite functions** — 256 or 65 536 entries — so the kernel layer materializes them once per `(op, formats, ρ)` into a locked cache (`Dict{TableKey, Memory{UInt8}}`, double-checked locking, builds outside the lock) and serves every later array call as a gather: Shape-A, one load per element, measured 0.27 ns/elem unary and 0.5 ns/elem binary. Tables are built *through the scalar path*, so they inherit its bit-exactness; the suite asserts table ≡ scalar over every entry.

Ternary (`FMA`, `FAA`, `Clamp`) is a finite function too — 2^(K1+K2+K3) entries — but that count spans four orders of magnitude across the 3–16 bitwidth range (512 B at K=3, 16 MiB at K=8), so one policy doesn't fit the whole range. A separate `TernaryKey → TernaryEntry` cache (`_ternary_table_for`, in `tables.jl`) tiers by Σ bitwidth:

- **Eager** (≤ 18 bits, all K ≤ 6 combinations, ≤ 256 KiB): builds and caches on the first array call.
- **Adaptive** (≤ 21 bits, the K = 7 band, up to 2 MiB): accumulates a per-signature element count across calls and builds only once a signature has processed enough elements to amortize the build; a byte-bounded LRU eviction (`TERNARY_CACHE_BYTES`) guards against many hot signatures coexisting.
- **Compute** (K = 8, 16+ MiB — a table is never worth it): `vmap!` runs Shape-B, the fully specialized scalar pipeline per element, optionally split across `Threads.@threads` for long enough arrays (each ternary draw is independent under a fixed, non-stochastic ρ, so lanes cannot interact).

Every ternary table entry, eager or adaptive, is still built *through the scalar path* — the tiering changes when/whether the cache exists, never what it contains. Stochastic calls of any arity always take Shape-B, with the RNG resolved once per array rather than per element.

Sorting

Ordering runs on **integer order keys**: a sign-magnitude fold into an unsigned type wide enough for the format's 2^K + 1 keys (`UInt16` at K ≤ 8, wider above), monotone with the total order. Key `0` is reserved for the single NaN, which the draft orders below −Inf (§4.12.1); every datum key is therefore ≥ 1. Same-format `TotalOrder`, `isless`, and the numeric comparisons are key comparisons (~1 ns); since a format has at most 2^K + 1 distinct keys, vectors sort with an **O(n) counting sort** installed via `Base.Sort.defalg` (forward and reverse orderings; anything exotic falls back to the stock algorithm).

Source and gates

`src/tables.jl` owns table keys, construction, cache policy, and ternary tiers; `src/kernels.jl` owns gather and scalar-loop execution. A change is complete only when table entries agree with scalar results over their full affordable domain, cold and warm behavior are measured separately, stochastic calls bypass pure tables, and cache byte accounting remains correct.

Continue through the implementation

Encoding and Decoding, Oracle and Rigor Classes, Verification Sessions.

Chapter 43

Exact Block Reductions

43.1 Exact Block Reductions

A block reduction has to add B products exactly and round **once**. The textbook way to do that is a *superaccumulator* — a fixed-point register wide enough to hold every representable product without loss, typically hundreds of bits, updated per lane. SmallFloats.jl does not have one, and does not need one.

This page explains what it does instead, why that is exact rather than merely accurate, and where the two guards that make it exact actually live.

The problem

A block is `(scale, elements)` — one scale factor shared by B elements (draft §5):

```
julia> b = Block(Binary8p1uf(0.5), (Binary8p4se(80.0), Binary8p4se(-0.375),
                                    Binary8p4se(-5.5), Binary8p4se(2.25)));

julia> scaleformat(b), elemformat(b), blocksize(b)
(Binary8p1uf, Binary8p4se, 4)
```

Reducing it means summing the four *lane values* $\texttt{scale} \times \texttt{element}_i$. Do that lane by lane in the result format and every partial sum rounds:

```
julia> SmallFloats.blockdecode(b)              # the exact lane values
(40.0, -0.1875, -2.75, 1.125)

julia> BlockReduceAdd(Binary8p4se, RNE_SN, b)  # exact sum 38.1875, projected ONCE
Binary8p4se(40.0 ≡ 0x6a)

julia> lanes = SmallFloats.blockdecode(b);

julia> foldl((a, v) -> Add(Binary8p4se, RNE_SN, a, Convert(Binary8p4se, RNE_SN, v)),
             lanes; init = zero(Binary8p4se))  # rounding at every step
Binary8p4se(36.0 ≡ 0x69)
```

One code point apart, from four lanes. The exact sum is 38.1875; the ulp in `[32, 64)` is 4, so the neighbours are 36 and 40 and the correct answer is 40. Accumulated rounding lands on 36 — not catastrophically wrong, but *not the defined result*, and the error grows with B.

So the reduction must be exact before it is projected. The question is what to accumulate in.

Step 1: decode each lane exactly

`blockdecode` produces `ωMultiply(decode(s), decode(xᵢ))` per lane, exactly, on a carrier chosen from **both** formats.

That last point is the whole content of the function, and it prevents a subtle misapplication. A block's scale and element formats are independent, and what has to fit is their *product*. The carrier rule is `ΣBᵢ ≤ emax`, so two formats that are each comfortable alone can compose into a lane product that is not: a `Binary16p1uf` scale reaches `2^32768`. Selecting the carrier from either format's own rung would overflow to `±Inf` — a plausible-looking special value rather than an error, which is the worst available failure mode. The carrier is therefore keyed on `rung(Val(:Multiply), FS, FE)`.

Step 2: accumulate exactly — two strategies

Once the lanes are exact, the sum must be too. There are two ways to get there, and the package tries the cheap one first.

The fast path: a span filter

`Float128` carries a 113-bit significand. A sum of B values is exactly representable in it when

```
(lane significand bits) + (exponent span) + ⌈log₂B⌉  ≤  113
```

— the significand bits of the widest lane, plus the number of binades the lanes straddle, plus the carries from adding B of them. Every term is an *integer*, and the exponent span is one pass over the lanes. So the package computes the span and decides:

reduction	shipped guard
`BlockReduceAdd`	`span + ⌈log₂B⌉ ≤ 92`
`BlockDotProduct`	`span + ⌈log₂B⌉ ≤ 76`

If the guard passes, a plain `Float128` accumulation **is** the exact answer — no rounding occurs at any step, so there is nothing to be careful about. The dot product's budget is tighter because each of its terms is a product of *four* datums (two scales, two elements) rather than two.

Why the filter is sound

The guards look generous against the worst case you would compute on paper. Two `Binary16p11se` datums multiply to 22 significand bits, which leaves 91 for `span + ⌈log₂B⌉` — one less than the 92 the sum guard admits.

The resolution is that **lane width and exponent span are anti-correlated, not independent.** A datum only reaches the bottom of a format's exponent range by being *subnormal*, and a subnormal datum carries proportionally fewer significand bits. In `Binary16p11se`, full 11-bit significands exist only at exponents −15…15; below that, every bit of extra reach costs a bit of width. The two quantities in the inequality cannot both be maximal.

That argument is easy to get wrong, so it is checked rather than trusted: an audit over wide format pairs admitted **17 040** sums and **16 479** dot products through the guards and compared every one against exact `BigFloat` truth. Zero were inexact.

Two guards, not one

The span filter is about the **significand**. It says nothing about whether the lanes *fit in `Float128` at all* — and at rung 3 they do not, because a `Binary16p1uf` scale reaches `2^32768`, which is `Inf` in `Float128` before any accumulation begins.

So the accumulator is selected by the **head** as well: `_f128acc` answers `false` for `HeadExact` unconditionally. Width and range are independent conditions and both must hold. An earlier version checked only the width, and the exponent condition was a premise nobody had stated.

The fallback: precision derived from the formats

When either guard fails, an exact `BigFloat` accumulation takes over at a precision **derived from the block's two formats and its length** — never a constant:

```
sums     2(B_S + B_E + P_S + P_E) + 64 + ⌈log₂B⌉
products B(P_S + P_E) + 128
```

Two different formulas because two different analyses. For a sum, the lanes straddle `2(B_S + B_E)` binades and each carries `P_S + P_E` significand bits, plus carries. For a product reduction the exponent merely shifts, but the significand is the *sum* over lanes.

Both replaced magic numbers, and the history is the reason they are written out. The product form was `16B + 128`, where `16 = 8 + 8` is `P_S + P_E` with `P = 8` hardcoded — correct at $K \le 8$ and understating by up to $2\times$ above it. Its sibling `_REDPREC = 2400` was ample through $B \approx 512$ and silently truncating above. Both failure modes produce a plausible wrong number rather than an error, which is exactly the class of defect this package treats as unacceptable.

Step 3: project once

Whichever path produced it, the exact result meets the projection engine exactly once. `ωBlockProject` follows the draft's special rows for the scale (`NaN`, `0`, `±Inf`), then divides each element result by the scale through a cheapest-first cascade:

```
exact Float64 quotient  →  correctly rounded Float128
                        →  bracket / pre-filter  →  MPFR interval
```

mirroring the scalar quotient group's rigor arguments. Each rung is tried only if the one before it cannot decide the code point.

P = 1 scales divide exactly, for free

A `P = 1` format has no stored significand bits, so every datum is ±2^e. Dividing by a power of two is exact, and the first rung of the cascade always succeeds — the draft calls this out in its own NOTE. This is why the MX-style scale format `Binary16p1uf` is not just conventional but cheap: the whole enclosure ladder collapses to one correctly rounded division.

Why no superaccumulator

A superaccumulator is the right answer when you cannot inspect the data — a fixed register sized for the worst case, paid for on every lane. Here the data *is* inspectable: one integer pass over the lane exponents answers the question the accumulator width was insuring against, and answers it exactly. The common case then runs in hardware `Float128`, and the uncommon case gets arbitrary precision sized to the actual formats.

A register-resident superaccumulator remains a tracked optimization for the Phase-3 work — as a *speed* change on the fallback path, not a correctness one.

Source and gates

`src/blocks.jl` owns block construction, scale/element operations, span filtering, exact reductions, and `BlockVector`. Review changes against three obligations: decoded lane values are exact, the chosen accumulator covers the proven span, and exactly one final projection occurs. Compare fused reductions with a deliberately wider reference, including mixed scales and special values.

Continue through the implementation

- Blocks and Scaled Operations — the API contract.
- Working with Blocks — the task-oriented guide.
- Blocks for Dynamic Range — when to reach for one.
- Oracle and Rigor Classes — where the enclosure cascade comes from.
- Proving your fused kernel exact — `BlockDotProduct` against 512-bit truth.
- Add an Operation — adding block variants.

Chapter 44

Verification Strategy

44.1 Verification Strategy

What Is Verified

The value sets are small enough that sampling is never necessary, so the suite enumerates — ≈ 8.9 M assertions in all:

- formats against an independent draft transliteration (14 679);
- ordering over all 2.5 M same-format pairs plus Next-op edge tables (7.6 M);
- every unary operation on every input against a 3072-bit protocol run; divide and the ternaries exhaustively;
- stochastic R-sweeps with directed-asymptote pins;
- table ≡ scalar over every entry, including the ternary tiers (eager and adaptive) against the scalar path;
- the sticky-head FMA/FAA escalation against the MPFR reference across every rounding-mode family and adversarial cancellation cases;
- blocks against a from-scratch reference composition;
- Float128 carrier ≡ Float64 carrier, and Float128-first ≡ MPFR differential builds;
- the mask rounding core ≡ the generic core over datums and reachable sums/products at boundary stochastic budgets N ∈ {45, 60};
- packed round-trips, and κ/conformance behavior.

Deterministic **specialization regressions** (concrete inferred return types at the public entry points, zero warm-path allocation) stand in for timing assertions.

The Approach to Benchmarking

Recorded after two measurement post-mortems: a benchmark closure over any non-`const` global measures Julia's dispatch machinery, not the code under test — and it distorts *ratios*, not just absolutes, because dispatch cost varies with call shape (a dynamic keyword call costs ~1 µs; a dynamic positional call far less; six unresolved interior sites cost six times one).

The rules the shipped `benchmarking/benchmarking.jl` enforces structurally:

- format types enter as type parameters, never as globals;

- operands come from untimed setup;
- functions retrieved reflectively pass through argument barriers to specialize;
- a preflight aborts the run if warm scalar paths allocate — including the wide-spread FMA/FAA sticky-head path.

The table-build section reports both the cold build (cache evicted per sample) and the steady-state warm cache hit, since callers amortize the former through the latter. Vary one binding per variant; verify specialization before believing a number.

Deliberate limitations

No implicit cross-format arithmetic (promotion is to `Float64`, explicitly). No in-place packed arithmetic. Threading is opt-in and narrow: only the untabled ternary (`K = 8`) compute kernel threads, and only above a size cutoff and when `Threads.nthreads() > 1`; every other kernel is single-threaded. `Irrational`/`Rational` inputs to `Convert` are rejected rather than double-rounded silently. The `Float128` machinery never changes results — disabling it (`SmallFloats_Float128=disable`) is a tested no-op semantically.

Reading a verification result

Treat the roll-call label as part of the result. **Exhaustive** means every member of the stated finite domain was checked. **Sampled** means the report must also state the generator, seed, sample count, and why that sample addresses the risk. A pass without its domain and method is not a portable claim.

The gate files under `test/` own correctness assertions; benchmark scripts own timing methodology. Documentation examples are checked separately so a green package test cannot conceal stale printed output.

Continue through the implementation

Function Tables and Array Kernels, Benchmark Correctly, Verify Custom Code.

Chapter 45

Add an Operation

45.1 Add an Operation

How to add a new operation to SmallFloats.jl so that it inherits everything the existing catalog has: bit-exact defined results, the full projection-mode vocabulary, generated scalar/array/block surfaces, table caching, exhaustive test coverage, benchmark pickup, and a truthful conformance declaration. Read Architecture and Invariants and Oracle and Rigor Classes first — this page assumes their vocabulary (the result-kind protocol, the two rigor classes, the interval protocol, the yd envelope stage).

What's generated vs what you write

Generated for you, from one registry line: the spec-register method `Op(fr, ρ, operands…; rng, R)`, the same-format convenience method `Op(x…)`, array methods (Shape-A table gathers or Shape-B loops), the `Block`/`Scaled` variants, the export, table cacheability, the conformance listing, and test-harness pickup.

You must write: the ω-mathematics (`ωeval` in `src/oracle.jl`) and the six universal duties below — special rows, the result-kind protocol, termination (the Niven duty), envelope justification, ladder validity, and running the gates.

The architecture is registry-driven: for the supported arities, **most of the surface is generated**. Adding an operation is mostly (1) one registry line and (2) one rigorous `ωeval` method — plus the *duties* that keep the package's guarantees true. Every subsection below states the duties explicitly; they are not optional.

Every new operation

1. **Special rows are spec, not optimization.** Write out the NaN/±Inf/zero rows explicitly in `ωeval`, in draft style, before any general evaluation. Mark any behavior the draft text under-determines with `[interp]`.

2. **Result-kind protocol.** `ωeval` must return one of `Float64 | Float128 | StickyF | BigExactF | EncloseF | Enclose128F`, with `Float64`/`Float128` reserved for *provably exact* results and `StickyF` for exact-head-plus-tail-direction results whose soundness bound is discharged at the emitting site (see the FMA/FAA wide-spread escalations). Keep the return union narrow (ideally `{Float64, EncloseF}`) — a wide union defeats inference and costs allocations (measured: a 4-type union turned a 50 ns op into 240 ns with 2 allocations).

3. **Termination (the Niven duty).** `project_interval` terminates only if the enclosure is not chasing a value the projection grid can actually hit. Any input whose *exact* result is a representable dyadic must be **peeled or detected** before an enclosure is built. For transcendental ops this is usually a finite set of special rows (prove completeness, as the π-family does with Niven's theorem); for algebraic ops it is an exactness test.

4. **Envelope justification.** If you supply an eager yd estimate, you owe an error bound: the true value must lie within $|yd| \cdot 2^{-45}$ (_F64_RELEXP). Acceptable bases, strongest first: IEEE-CR hardware ops (÷, sqrt: ≤ ½ ulp, unconditional); short compositions of CR ops (≤ ~1.5 ulp); faithful libm (≤ 1 ulp, an empirical-but-published claim); short faithful compositions with condition number ≤ 1 (≤ ~3 ulp). **Measure it**: sample your estimator against ≥ 256-bit MPFR truth over the op's domain, including the treacherous regions (near zeros of the result, domain boundaries), and demand ≥ 2^6 slack under 2^{-45}. The same duty applies to a Float128 `fq` estimator against its 2^{-90} envelope. Beware **argument-error amplification**: an estimate computed as `f(g(x))` where g rounds (e.g. `sin(π·r)` in Float64) can be catastrophically worse near zeros of f than a native result-domain evaluation (`sinpi(r)`) — measure the composition you actually ship.

5. **Ladder validity.** The MPFR closure must return a genuine directed enclosure. Whole-expression `setrounding` is valid only when the composition is monotone in every intermediate (state the argument in a comment, as `Softplus` does); otherwise round each step in the correct direction explicitly (as `_mpfr_rsqrt` does for the anti-monotone 1/√x).

6. **Gates.** Run the full suite. The unary exhaustive ×hp gate and the coverage check pick up registry ops automatically; extend the hand-curated lists where applicable (monotonicity ops, binary exhaustive sweeps). Then regenerate the benchmark report — `bench_scalar_ops` iterates the registry lists, so your op appears in all four operand-class tables automatically; **add a `_SAFE_DOMAINS` entry** in `benchmarking/benchmarking.jl` if the op is argument-restricted, or the safe-args table will use the oracle-derived fallback pool.

Scalar operations

The registry lives in `src/ops_scalar.jl`. Adding a name to `_UNARY_OPS` / `_BINARY_OPS` / `_TERNARY_OPS` causes, with no further work:

- registration in `OP_REGISTRY` (group `:B`/`:C`/`:A` per the loop defaults — override with an explicit `register_op!` if the default group is wrong);
- the generated spec-register method `Op(fr, ρ, operands…; rng, R)` and the same-format convenience `Op(x…)`;
- generated array methods routing through `vmap` (Shape-A tables for pure ρ — always at arity ≤ 2, bitwidth-gated at arity 3 — Shape-B scalar/threaded loops otherwise);
- generated `BlockOp` and `ScaledOp` variants (`src/blocks.jl`);
- the export (`src/SmallFloats.jl` loops the registry);
- table cacheability, conformance listing, and test-harness coverage.

What is *not* generated is the mathematics: `ωeval` in `src/oracle.jl`.

Adding a unary operation

Worked example: **`Cbrt`** (cube root), chosen because it exercises the hardest duty — exactness detection for an algebraic op.

Step 1 — registry (`src/ops_scalar.jl`): append `:Cbrt` to `_UNARY_OPS`. The loop registers it as group `:B` with arity 1.

Step 2 — ω-semantics (`src/oracle.jl`), with every duty discharged.

Type the row '::AbstractFloat', not '::Float64'

An enclosure-ladder row is reached at **every rung**. `_LADDER_OPS` is derived by subtraction in `oracle.jl` — whatever is neither an exact selection, nor exact arithmetic, nor `Convert` — so a new registry operation automatically gets `_wf128`, `_wexact` and `_wdyadic` rows that forward to *this* method with `Float128`, `BigFloat` or converted-from-`Dyadic` operands.

A row typed `::Float64` therefore works for the 432 formats at rung 1 and raises a `MethodError` for the other 72. The package types 22 of its own ladder rows `::AbstractFloat` for exactly this reason; the 6 that are `::Float64` are the exact-arithmetic rows, which have per-head siblings written separately.

Gate G10 catches the omission — it calls every registry operation at every rung and asserts only that the call returns — and G4 asserts that `_EXACT_SELECTION`, `_EXACT_ARITH`, `_LADDER_OPS` and `Convert` partition `OP_REGISTRY` exactly, so an operation cannot be classified into none of them and silently lose its wide-rung rows.

```
# Exact-cube detection (termination duty): x = ±m·2^(3k) with m an odd perfect
# cube ⇒ cbrt(x) is a Float64-exact dyadic that project_interval must never
# chase. Oracle-path cost is irrelevant; correctness of the peel is everything.
function _exact_cbrt(x::Float64)
    fr, e = frexp(abs(x))               # |x| = fr·2^e, fr ∈ [0.5, 1)
    m = Int64(ldexp(fr, 53)); e -= 53   # |x| = m·2^e exactly
    tz = trailing_zeros(m); m >>= tz; e += tz
    e % 3 == 0 || return nothing
    r = round(Int64, cbrt(Float64(m)))
    for c in max(r - 1, 1):(r + 1)      # cbrt is faithful ⇒ true root ∈ r ± 1
        Int128(c)^3 == Int128(m) && return flipsign(ldexp(Float64(c), e ÷ 3), x)
    end
    nothing
end

function _eval(::Val{:Cbrt}, x::AbstractFloat)     # ::AbstractFloat, NOT ::Float64
    isnan(x) && return NaN                      # special rows first: they are spec
    isinf(x) && return x                        # cbrt(±∞) = ±∞
    iszero(x) && return 0.0
    c = _exact_cbrt(x)
    c === nothing || return c                   # exact dyadic result — peeled
    # yd: Float64 cbrt is faithful (≤ 1 ulp ≈ 2⁻⁵²; measure it) ⇒ ≥ 2⁷ slack
    # under the 2⁻⁴⁵ envelope. fq: libquadmath cbrtq under the 2⁻⁹⁰ envelope.
    # Ladder: cbrt is monotone ⇒ whole-expression directed rounding encloses.
    _encl1(cbrt, x; fq=() -> cbrt(Float128(x)), yd=cbrt(x))
end
```

Step 3 — optional surface: a Base veneer (`Base.cbrt(x::Binary) = Cbrt(x)` next to the others in `ops_scalar.jl`), a docstring, and — since cbrt is monotone — its symbol in the test suite's monotonicity list.

Step 4 — validate: envelope measurement for `cbrt` (include tiny and huge x; cbrt has no dangerous regions — the result is never near zero except at zero, which is peeled), then the full suite: the exhaustive `×hp` gate now checks `Cbrt` on every code point of the reference format under four modes against the 3072-bit protocol, automatically.

Adding a binary operation

Worked example: **LogAddExp** — `log(eˣ + eʸ)` — chosen because it exercises composed estimators and the ladder-monotonicity argument.

Step 1 — registry: append `:LogAddExp` to `_BINARY_OPS` (default group `:C`; the special five arithmetic names get `:A`).

Step 2 — ω-semantics. One generic evaluator serves all three precisions — Float64 (`yd`), Float128 (`fq`), and BigFloat (the ladder):

```
# Stable form: h + log1p(exp(l − h)), h = max, l = min. Every step is monotone
# nondecreasing in (x, y), so whole-expression directed rounding in _mpfr2 is a
# valid enclosure (the same argument Softplus records).
_logaddexp(a, b) = (h = max(a, b); l = min(a, b); h + log1p(exp(l - h)))

function ωeval(::Val{:LogAddExp}, x::AbstractFloat, y::AbstractFloat)
    (isnan(x) | isnan(y)) && return NaN
    (x == Inf || y == Inf) && return Inf        # ∞ dominates
    x == -Inf && return y                       # log(0 + eʸ) = y, exactly
    y == -Inf && return x
    # yd: two faithful libm calls plus additions, condition ≤ 1 throughout
    # (result ≥ max(x, y)) ⇒ ≤ ~3 ulp ≈ 2⁻⁵⁰ − 2⁵ slack; measure it, including
    # x ≈ y (where log1p(exp(0)) dominates) and widely separated operands.
    _encl2(_logaddexp, x, y;
           fq=() -> _logaddexp(Float128(x), Float128(y)),
           yd=_logaddexp(x, y))
end
```

Termination note: can `logaddexp(x, y)` be exactly dyadic for dyadic operands with the special rows peeled? `log(eˣ + eʸ) = d` requires `eˣ + eʸ = e^d` — by Lindemann–Weierstrass this forces relations that dyadic arguments cannot satisfy except on the peeled rows, so no further peel is needed; **record that argument in a comment**. When you cannot prove such a statement for your op, hunt for exact cases the way `_exact_cbrt` does.

Step 3 — tests: binary ops are not in the unary exhaustive gate; add a divide-style exhaustive sweep (256×256 operand pairs against a ladder-only reference, a handful of modes) to the suite, and an envelope-sanity loop entry if you introduced a new libm dependence.

Adding a ternary operation

Worked example: **`FMS`** — fused multiply-subtract, `x·y − z` — chosen because the cheapest correct implementation is *delegation*, and delegation is a first-class technique here.

Step 1 — registry: append `:FMS` to `_TERNARY_OPS` (group `:A` by the loop). The generated array method routes through the same ternary bitwidth policy every other ternary op uses (`tables.jl`'s `_ternary_table_for`): small operand formats table (eagerly or adaptively), `K = 8` runs the — optionally threaded — Shape-B scalar loop, and stochastic ρ always draws per element. No op-specific work needed; the policy keys on formats and ρ, not on which op it is.

Step 2 — ω-semantics by delegation to FMA's already-verified analysis (width thresholds, `_twosum` exactness, wide-spread fallback):

```
# x·y − z ≡ FMA(x, y, −z). Negation of z follows the Subtract convention:
# preserve NaN, and map −0 to +0 so the single-zero datum model is respected.
ωeval(::Val{:FMS}, x::Float64, y::Float64, z::Float64) =
    ωeval(Val(:FMA), x, y, isnan(z) ? z : (iszero(z) ? 0.0 : -z))
```

Delegation inherits FMA's result kinds, its exactness thresholds, *and* its test pedigree — but verify the sign algebra of your reduction on the special rows (here: `FMS(∞, 1, ∞)` must be NaN, which the delegation produces because `FMA(∞, 1, −∞)` is `∞ − ∞`). Add the delegated op to the ternary exhaustive sweep in the suite.

Adding a quaternary operation

Arity 4 is **not yet plumbed** — the generation loops, kernels, and block layer handle arities 1–3. Adding a quaternary op is therefore two tasks: extend the plumbing (once), then add the op. Worked example: **`FMMA`** — `x·y + z·w`, a two-term dot product.

Plumbing (one-time), four sites:

1. `src/ops_scalar.jl` — register manually after the arity loops (`register_op!(:FMMA, 4, :A)`), and add an `op.arity == 4` branch to the spec-register generation loop, mirroring the ternary branch with a fourth operand:

```
# …continuing the existing `if op.arity == …` cascade:
if op.arity == 4
    @eval begin
        @inline function $name(fr::Type{<:Binary}, ρ::ProjSpec,
                               x::Binary, y::Binary, z::Binary, w::Binary;
                               rng::MaybeRNG=nothing, R::Union{Nothing,Int}=nothing)
            apply_op($V(), fr, ρ, _drawR(ρ, rng, R),
                     decode(x), decode(y), decode(z), decode(w))
        end
        @inline $name(x::T, y::T, z::T, w::T; kw...) where {T<:Binary} =
            $name(T, default_projspec(T), x, y, z, w; kw...)
    end
end
```

   (`apply_op` and `ωeval` are already variadic — no change needed there.)

2. `src/kernels.jl` — a four-array `vmap!` method. The plain Shape-B loop (copy the *body* of the ternary method's compute branch, add operand `D`) is enough to get correct results; the ternary method's table-tiering and threading are a bitwidth-driven optimization layered on top (`tables.jl`'s `_ternary_table_for` and its LRU cache), not required plumbing — extend to arity 4 only if a table win at that arity is worth the 2^(4K) growth. Also add an arity-4 branch in the registry-generated array-surface loop and the matching rng-threading overload (mirroring the three-array one) so stochastic ρ dispatches correctly.

3. `src/blocks.jl` — either an arity-4 branch in the Block/Scaled generation loop (four operand blocks, four `blockdecodes`) or an explicit skip (`op.arity > 3 && continue`) if a block form is not wanted; be deliberate.

4. `src/approx.jl` — `conformance_report` prints arities with `for a in 1:3`; widen to `1:4`.

The op itself follows the FAA/FMA width-analysis playbook exactly:

```
const _DE_FMMA = 92     # two exact ≤17-bit products: 18 + ΔE ≤ 113, margin 3

function ωeval(::Val{:FMMA}, x::Float64, y::Float64, z::Float64, w::Float64)
    (isnan(x) | isnan(y) | isnan(z) | isnan(w)) && return NaN
    (((iszero(x) && isinf(y)) || (isinf(x) && iszero(y))) ||
     ((iszero(z) && isinf(w)) || (isinf(z) && iszero(w)))) && return NaN
    p = x * y; q = z * w                          # exact: ≤8-bit significands
    if isinf(p) || isinf(q)
        (isinf(p) && isinf(q) && p != q) && return NaN
        return isinf(p) ? p : q
    end
    s, e = _twosum(p, q)
    e == 0.0 && return iszero(s) ? 0.0 : s        # exact in Float64
    (_f128() && !iszero(p) && !iszero(q) && _expdiff(p, q) <= _DE_FMMA) &&
        return Float128(p) + Float128(q)          # exact by width
    BigExactF(() -> setprecision(() ->
        BigFloat(x) * BigFloat(y) + BigFloat(z) * BigFloat(w),
        BigFloat, bigprec_prod(x, y, z, w)))     # DERIVED, never a constant
end
```

Derive the precision from the operands — never a constant

This escalation used to read `_BIGP`, a 2200-bit constant. It was ample for every format that existed at $K \le 8$ and **insufficient for 50 of the 504**, and it did not announce that: it returned a plausible wrong number, which gate G2 was written red to record (`1//1` where the exact answer is `2//1`, the operand returned unchanged because a residual 30 000 binades below the rounding position fell off the accumulator).

`bigprec(F…)` derives the precision from the formats; `bigprec(x…)` and `bigprec_prod(x…)` derive it from the operands in flight. Gate G2 asserts the derivation is sufficient across the whole grid, so a new operation that uses it inherits that proof. A new constant would need its own.

Testing duty is heavier at arity 4: the full cross-product is 2^{32} tuples, so the suite entry must be *sampled* (seeded, ladder-referenced, $\ge 10^5$ tuples across several modes) plus hand-picked exhaustion of the special-row algebra and the width-threshold boundary (`ΔE ∈ {91, 92, 93}` constructions). Document the sampling in the test, since it breaks the suite's "enumerate, never sample" norm — that exception must be visible, not silent.

Block operations

The block layer (`src/blocks.jl`) implements the draft's elementwise schema **once** — `blockdecode → ω-op lanewise → blockproject` — and generates every registry op's `BlockOp`/`ScaledOp` from it. The reductions are hand-written on a second shared pattern. Which subsection you need depends on whether your operation fits one of those two patterns.

Adding a scaled block operation

If the scalar operation is (or can be) a **registry op, the scaled form is free**: registering `:Cbrt` above already produced `ScaledCbrt(fr, ρ, s, x)` and `BlockCbrt(fr, ρ, b, sr)` with the draft's §5.4 semantics — decode `scale × element` exactly per lane, apply the ω-op, `BlockProject` against the result scale through the division cascade. **Prefer this route**; it is the package's non-divergence mechanism.

Write a scaled op by hand only when it is *not* an elementwise image of a registry op. The primitive to compose with is `_bp_element(fr, ρ, R, res, Sdat)` — it owns the draft's scale-special rows (NaN/0/±Inf scale) and the entire division-by-scale rigor cascade. Example, a scaled affine $s_1 \cdot x + s_2 \cdot y$ projected against a *unit* result scale:

```
function ScaledAffine(fr::Type{<:Binary}, ρ::ProjSpec,
                      s1::Binary, x::Binary, s2::Binary, y::Binary;
                      rng::MaybeRNG=nothing)
    # Each (scale, element) pair is a TWO-FACTOR MONOMIAL and gets its own head
    # from the two formats — the same rule `blockdecode` applies to a block.
    h1 = rung(Val(:Multiply), typeof(s1), typeof(x))
    h2 = rung(Val(:Multiply), typeof(s2), typeof(y))
    a = ωeval(h1, Val(:Multiply), lift(h1, decode(s1)), lift(h1, decode(x)))::carriertype(h1)
    b = ωeval(h2, Val(:Multiply), lift(h2, decode(s2)), lift(h2, decode(y)))::carriertype(h2)
    # The two products may land on DIFFERENT carriers, so the add runs at their
    # join with each operand lifted into it.
    h = _joinheads(a, b)
    res = ωeval(h, Val(:Add), lift(h, a), lift(h, b))
    _bp_element(fr, ρ, _drawR(ρ, rng, nothing), res, 1.0)
end
```

The assertion is `::carriertype(h)`, and writing `::Float64` there is a real defect rather than a stylistic choice. It is true for the 432 formats at rung 1 and an outright `TypeError` for the other 72, whose lanes decode to `Float128` or to the exact dyadic carrier. The generated `Scaled*` surface in `blocks.jl` carried exactly that mistake and it is recorded as §11 M44 in the execution log — the guidance here used to teach it.

What the assertion *is* load-bearing for survives the correction: `Multiply` is closed on the carrier at every rung (gate G4 pins this), so the row cannot escalate to a lazy `BigExactF` here, and saying so is what lets `_joinheads` below see carrier values only. JET finds the missing statement before any test can.

The Add result may be any kind in the protocol — `_bp_element` finishes all of them. Dividing by a scale of `1.0` is the identity fast path.

Adding an elementwise block operation

Same schema, whole-block granularity. The three obligations: decode operand blocks with `blockdecode` (exact), evaluate lanewise with `ωeval` (never with ad-hoc Float64 math), and finish with `blockproject` against the supplied result scale (never with per-lane `project` — the scale-division rows belong to `blockproject`). Example — `BlockRelu`, an elementwise `max(x, 0)` that reuses the verified `MaximumNumber` semantics:

```
"""BlockRelu(fr, ρ, b, sr): lanewise max(xᵢ, 0) — NaN lanes propagate NaN per
MaximumNumber's number-preferring rows only when both operands are NaN, i.e.
never here; state your lane semantics this explicitly for any new op."""
function BlockRelu(fr::Type{<:Binary}, ρ::ProjSpec, b::Block{B}, sr::Binary;
                   rng::MaybeRNG=nothing) where {B}
    X = blockdecode(b)
    Z = ntuple(i -> ωeval(Val(:MaximumNumber), X[i], 0.0), Val(B))
    blockproject(fr, ρ, sr, Z; rng)
end
```

Add the name to the block-surface export list and to `conformance()`'s `blocknames` extension vector so the declaration stays truthful. Test against a from-scratch reference composition (the suite's existing block tests show the pattern) — element format × scale format × B, exhaustively where feasible.

Adding a reductive block operation

Reductions do **not** fit the elementwise schema; they follow the second pattern, visible in `BlockReduceAdd`/`BlockDotProduct`: (1) resolve the ∞/NaN **fold algebra** on Float64 classifications first; (2) an integer **span filter** decides whether the whole reduction is exactly representable in `Float128` — if so, plain `Float128` accumulation *is* the exact answer; (3) otherwise a `BigExactF` exact accumulator at provably sufficient precision; (4) exactly **one** projection, at the very end, via `_finish`. Never round intermediates.

Worked example — **`BlockSumOfSquares`**, Σ x_i^2:

```
"""BlockSumOfSquares(fr, ρ, b): project(Σ decode-lane_i^2) — one rounding total.
Fold algebra: any NaN lane ⇒ NaN; else any ±Inf lane ⇒ +Inf (squares cannot
cancel — no opposite-infinity NaN case exists, unlike ReduceAdd); else exact."""
function BlockSumOfSquares(fr::Type{<:Binary}, ρ::ProjSpec, b::Block{B};
                           rng::MaybeRNG=nothing, R::Union{Nothing,Int}=nothing) where {B}
    X = blockdecode(b)
    res = if any(isnan, X)
        NaN
    elseif any(isinf, X)
        Inf
    elseif all(iszero, X)
        0.0
    else
        # span filter: lanes carry ≤17-bit significands ⇒ squares ≤34 bits and
        # a square's exponent is 2·(lane exponent); the sum is exact in Float128
        # when 34 + 2·span + ⌈log₂B⌉ + 1 ≤ 113  ⇒  2·span + ⌈log₂B⌉ ≤ 78.
        if _f128() && 2 * _expspan(X) + _log2ceil(B) <= 78
            acc = Float128(0)
            for v in X
                q = Float128(v)
                acc += q * q                    # exact squares, exact sum
            end
            acc
        else
            BigExactF(() -> setprecision(BigFloat, _REDPREC) do
                acc = BigFloat(0)
                for v in X
                    acc += BigFloat(v)^2        # exact at _REDPREC
                end
                acc
            end)
        end
    end
    _finish(fr, ρ, _drawR(ρ, rng, R), res)
end
```

The two lines that demand your care in any new reduction are the **fold algebra** (derive it from the draft's reduce definition on the extended reals — here the absence of an opposite-infinities NaN case is a *theorem about squares*, stated in the docstring) and the **span-filter inequality** (derive it from operand widths, write the derivation in the comment, and add boundary-condition tests at the threshold, exactly as the suite does for the existing `_DE_*` constants). Everything else — the exact accumulator, the single `_finish`, stochastic R plumbing — is pattern.

Checklist

- Full test suite green, including your new sweep entries.
- Envelope measurements recorded for any new yd/fq estimator.
- Benchmark report regenerated; `_SAFE_DOMAINS` entry added if the op is argument-restricted; sanity-check the op's row in the safe-args table.
- Docstring on the public name; `[interp]` markers where you interpreted.
- `conformance_report()` shows the op (automatic for registry ops; manual block additions extend `blocknames`).

Continue through the implementation

Oracle and Rigor Classes, Exact Block Reductions, Verify Custom Code.

Chapter 46

Verify Custom Code

46.1 Verify Custom Code

Trust, then verify — these patterns turn "is my code exact?" into an enumeration. Formats are small enough that "test on a few points" is never necessary; the recipes below enumerate the input space (or, where the space is too large to enumerate, sweep it exhaustively along the dimension that matters) and compare against an independently computed reference. Several examples use unexported internals (`SmallFloats.round_to_precision`, `SmallFloats.project`, …); those are stable enough to learn from but are not covered by semantic-versioning guarantees.

Verify a quantizer

Here every in-range datum of `Binary8p3se` is projected into `Binary8p4se` and checked against a brute-force nearest search under 256-bit arithmetic (distance agreement; the draft's tie rule is the projection engine's job and is pinned elsewhere in the suite):

```
using SmallFloats

function ref_nearest_distance(::Type{T}, x) where {T}
    fins = [v for v in (rawvalue(T, UInt8(c)) for c in 0:255) if isfinite(decode(v))]
    minimum(abs(setprecision(() -> BigFloat(decode(v)) - BigFloat(x), BigFloat, 256))
            for v in fins)
end

function verify_quantizer()
    ok = true
    for c in 0x00:0xff
        x = decode(rawvalue(Binary8p3se, c))
        (isfinite(x) && abs(x) <= decode(MaxFiniteOf(Binary8p4se))) || continue
        got = Convert(Binary8p4se, RNE_SN, x)
        ok &= abs(decode(got) - x) == Float64(ref_nearest_distance(Binary8p4se, x))
    end
    ok
end
verify_quantizer()
```

```
true
```

This pattern — enumerate the inputs, compare against an independently written reference — is how the entire package is tested, and it is available to *your* quantization code at trivial cost.

Prove a fused kernel

The block layer promises *one* projection with exact lane arithmetic. Trust, then verify — against big-float truth, over random blocks with mixed scales and an honest special-value mix:

```
using SmallFloats, Random

rng = Xoshiro(5)
mk() = Block(Binary8p1uf(2.0^rand(rng, -3:3)),
             ntuple(_ -> rawvalue(Binary8p4se, UInt8(rand(rng, 0:255))), 32))

function verify_dots(trials)
    for _ in 1:trials
        bx, by = mk(), mk()
        lx = [decode(bx.s) * decode(v) for v in bx.x]
        ly = [decode(by.s) * decode(v) for v in by.x]
        (any(!isfinite, lx) || any(!isfinite, ly)) && continue
        truth = setprecision(() -> sum(BigFloat(lx[i]) * BigFloat(ly[i]) for i in 1:32),
                             BigFloat, 512)
        got = BlockDotProduct(Binary8p4se, RNE_SN, bx, by)
        ref = setprecision(() -> SmallFloats.project(Binary8p4se, RNE_SN, truth), BigFloat, 512)
        codepoint(got) == codepoint(ref) || return false
    end
    true
end
verify_dots(200)
```

```
true
```

The same shape verifies any custom fused kernel you write: compose the truth in `BigFloat`, project once, compare code points.

Audit stochastic rounding

Every stochastic projection is a deterministic function of the draw R, so its *distribution* is checkable exactly. For `x = 2 + 3/64` in `Binary8p4se` the fraction is ν = 3/16 of an ulp, so `StochasticA{4}` must round up for exactly 3 of the 16 draws:

```
σ4 = RSA_SN(4)                  # ≡ ProjSpec(StochasticA{4}(), SatNone())
x = 2.0 + 3/64
count(decode(SmallFloats.project(Binary8p4se, σ4, x; R)) == 2.25 for R in 0:15)
```

```
3
```

Sweeping R like this turns "is my stochastic pipeline unbiased?" from a statistical question into an exhaustive one — the pattern the shipped test suite uses.

Applying monotonicity

Decision and ranking systems consume scores through *order*. If scores are produced in one format and compared in another, the conversion must be monotone — and formats are small enough to prove it by enumeration rather than trust it. Walk every `Binary8p3se` datum in total order and check that its `Binary8p4se` image never goes backward on the integer order keys:

```
using SmallFloats
using SmallFloats: order_key

function monotone_conversion(::Type{From}, ::Type{To}) where {From,To}
    prev = nothing
    for v in sort(From.(0x00:UInt8(2^bitwidth(From) - 1)))
        isnan(decode(v)) && continue
        g = Convert(To, RNE_SN, v)
        prev !== nothing && order_key(g) < order_key(prev) && return false
        prev = g
    end
    true
end
monotone_conversion(Binary8p3se, Binary8p4se)
```

```
true
```

Run this for every `(From, To, ρ)` triple your system actually uses; it is a few hundred integer comparisons per triple. A ranking pipeline whose conversions all pass this audit cannot invert a preference by changing formats — a guarantee no amount of spot-testing provides.

Measuring Approximation

Search under a compute budget often wants a cheap, approximate evaluation function. The κ registry turns "how approximate?" into a *measured* code-point bound — and code-point bounds compose into a decision rule: **if two defined evaluations differ by more than 2κ code points along the total order, the approximate evaluator cannot invert their comparison.** Verify the rule exhaustively for a κ = 2 evaluator:

```
using SmallFloats: order_key

fast(x) = (r = Exp(Binary8p4se, RNE_SN, x);
           isfinite(decode(r)) ? NextGreaterThan(NextGreaterThan(r)) : r)
κ, exhaustive = measure_kappa(fast, :Exp, Binary8p4se, (Binary8p4se,), RNE_SN)

function margin_audit(fast, κ)
    codes = [rawvalue(Binary8p4se, UInt8(c)) for c in 0:255]
    safe = violations = 0
    for a in codes, b in codes
        da, db = Exp(Binary8p4se, RNE_SN, a), Exp(Binary8p4se, RNE_SN, b)
        (isnan(decode(da)) || isnan(decode(db))) && continue
        codedistance(da, db) > 2κ || continue
        safe += 1
        (order_key(fast(a)) < order_key(fast(b))) ==
            (order_key(da) < order_key(db)) || (violations += 1)
    end
    (safe, violations)
end
(κ, exhaustive, margin_audit(fast, κ)...)
```

```
(2.0, true, 49522, 0)
```

49,522 operand pairs clear the 2κ margin, and the approximate evaluator agrees with the defined ordering on every one of them — zero violations, exhaustively. Inside the margin, comparisons are genuinely undecidable at this κ; a search can treat sub-margin comparisons as ties to expand, or escalate those few nodes to the exact evaluator. Either way the pruning is *provably* sound, with κ measured at registration rather than promised.

Continue through the implementation

Add an Operation, Benchmark Correctly, Verification Sessions.

Chapter 47

Benchmark Correctly

47.1 Benchmark Correctly

Recorded after two measurement post-mortems: a benchmark closure over any non-`const` global measures Julia's dispatch machinery, not the code under test — and it distorts *ratios*, not just absolutes, because dispatch cost varies with call shape (a dynamic keyword call costs ~1 µs; a dynamic positional call far less; six unresolved interior sites cost six times one).

The rules

The rules the shipped `benchmarking/benchmarking.jl` enforces structurally:

- format types enter as type parameters, never as globals;
- operands come from untimed setup;
- functions retrieved reflectively pass through argument barriers to specialize;
- a preflight aborts the run if warm scalar paths allocate — including the wide-spread FMA/FAA sticky-head path.

The table-build section reports both the cold build (cache evicted per sample) and the steady-state warm cache hit, since callers amortize the former through the latter. Vary one binding per variant; verify specialization before believing a number.

Set up a measurement that is actually measuring your code

The package's benchmark doctrine, in one snippet (needs the `benchmarking/` environment for Chairmarks). Format types enter as **type parameters**; operands come from Chairmarks' *untimed* `setup`; and if you retrieve functions reflectively (`getfield(SmallFloats, op)`), pass them through an argument barrier so they specialize:

```
using Chairmarks, SmallFloats, Random
using Statistics: median

function bench_add(::Type{T}) where {T<:Binary}          # T: type parameter, not a global
    pool = [rawvalue(T, rand(UInt8)) for _ in 1:4096]
    @be (rand(pool), rand(pool)) (t -> Add(T, RNE_SN, t[1], t[2]))(_)
end

b = bench_add(Binary8p4se)
(round(median(b).time * 1e9; digits=1), median(b).allocs)
```

```
(16.3, 0.0)          # ns per full scalar Add, zero allocations
```

The 60× benchmarking slowdown

The same call with T read from a non-`const` global measures ~1 µs — Julia's dynamic keyword dispatch, not this package. Two project post-mortems trace to exactly this mistake; the shipped `benchmarking/benchmarking.jl` asserts specialization (zero warm-path allocation) before it believes any number, and so should yours.

Continue through the implementation

Verification Strategy, Verify Custom Code, Verification Sessions — whose *Benchmarking without measuring the dispatcher* is the complete version of the snippet above, with the two measurement post-mortems that produced it.

Part VIII

Help

Chapter 48

Troubleshooting

48.1 Troubleshooting

Symptom-first reference. Find what you are seeing, not what you did wrong — each entry names the symptom, the cause, the fix, and where to read the full explanation.

Values and Construction

Symptom: `T(0x02)` gives a different value than `T(2)`, and it looks like a bug. Cause: an `Unsigned` argument is a *code point*, at every bitwidth — `T(0x02)` constructs code point `0x02`, while `T(2)` projects the numeric value two. Signedness of the integer type is what distinguishes them, so this holds for `UInt8`, `UInt16`, and any other `Unsigned`. Fix: use `T(Int(c))` when you mean the number, `T(c::Unsigned)` when you mean the code point, or call `Convert` when intent should be unmistakable. Full explanation: Cheat Sheet, Values, Code Points & Conversion.

Symptom: `Binary8p4se(0x02)` and `convert(Binary8p4se, 0x02)` disagree. Cause: `convert` is the deliberate exception to the Unsigned-is-a-code-point rule and is value-preserving — `Binary8p4se(0x02)` is code point 2, while `convert(Binary8p4se, 0x02)` is the value 2.0. Promotion, `similar`, and `fill` depend on `convert` behaving this way. Fix: know which one you are calling; prefer `T(x)` or `Convert(T, ρ, x)` in your own code and reserve bare `convert` for places Base itself calls it. Full explanation: Cheat Sheet, Values, Code Points & Conversion.

Symptom: constructing from a `Rational` throws. Cause: `Rational` is not one of the supported construction carriers. Fix: convert explicitly to an exact supported carrier (a `Float64`, `Dyadic`, or similar) or knowingly to `Float64` first, then construct. Full explanation: Cheat Sheet, Values and code points.

Symptom: `T(-0.0) === zero(T)` surprises code that expects signed zero. Cause: every P3109 format has exactly one zero; there is no negative zero to distinguish. Fix: stop testing for negative zero; if you need "approached from below," use the projection engine's `sticky` argument instead of relying on a stored sign. Full explanation: Core Model, Encoding and Decoding.

Symptom: a value built with `rawvalue` decodes to garbage or a non-existent datum. Cause: `rawvalue(T, c)` is unchecked code-point construction — it skips the range validation that `T(c::Unsigned)` performs, so an out-of-range integer produces a bit pattern with no corresponding intent-checked datum. Fix: prefer validated `T(c::Unsigned)` outside performance kernels; reserve `rawvalue` for hot loops where the caller has already proven `c` is in range. Full explanation: Cheat Sheet, Values and code points.

Symptom: dividing by zero gives `NaN`, not `Inf` as IEEE-754 code expects. Cause: P3109 defines `x / 0 → NaN` (and `Recip(±Inf) = 0`), which is not IEEE division-by-zero semantics. Fix: do not port IEEE-754 zero-division assumptions across; check for zero divisors explicitly if your algorithm depends on signed infinities from division. Full explanation: Cheat Sheet, Values and code points; P3109 in One Chapter.

Arithmetic and Semantics

Symptom: `Multiply/Add` overflowed and you expected a clamp, but got `Inf` (or vice versa). Cause: `SatNone` does not mean "no saturation" in the clamping sense — it applies the draft's domain-, signedness-, and direction-dependent rows, which can still produce `Inf`, `MaxFinite`, or `NaN` depending on the case. Only `SatFinite` guarantees clamping to the finite range. Fix: choose `SatFinite` whenever clamping is actually required; do not assume `SatNone` behaves like `clamp`. Full explanation: Cheat Sheet, Projection specifications.

Symptom: mixing two `Binary` formats in one expression throws a promotion error instead of silently widening. Cause: this is deliberate — the package refuses silent cross-format promotion so that no rounding step happens somewhere nobody wrote. Fix: convert explicitly, `Convert(T, ρ, x)`, to the format you actually want before combining. Full explanation: Cheat Sheet, Values and code points.

Symptom: `rem`, `mod`, or a similar Base numeric function refuses to run, or errors in a way that looks like a missing method. Cause: some generic Base numeric functions are defined in terms of operations or promotion behavior this package deliberately does not supply by default (for example, functions that would require an implicit cross-format promotion or an IEEE-754-only special case). This is a refusal, not an omission bug. Fix: express the computation with the package's explicit operation catalog (`Add`, `Subtract`, `Multiply`, `Divide`, `FMA`, ...) and `Convert` calls instead of relying on the generic fallback. Full explanation: Cheat Sheet, Scalar operation catalog.

Arrays and Performance

Symptom: an array of `Binary` values is slow, or every element looks boxed. Cause: the element type is the *abstract* format `Binary{K,P,Σ,Δ}`, not a concrete alias — the abstract format is not `isbits`, so a generic-element array boxes every entry. Fix: use a concrete element type, `Vector{Binary8p4se}` or `Vector{format(K,P,Σ,Δ)}`. Full explanation: Cheat Sheet, Common pitfalls (below), and Architecture and Invariants.

Symptom: `Vector{Binary{K,P,Σ,Δ}}(undef, n)` silently boxes every element, even though you thought you had a concrete type. Cause: `Binary{K,P,Σ,Δ}` with type-parameter variables is abstract, and `Vector{…}(undef, n)` cannot be intercepted or redirected to a concrete representation the way constructors can. Fix: use `Vector{Binary8p4se}` (a concrete alias) or `Vector{format(K,P,Σ,Δ)}` (concrete once `K,P,Σ,Δ` are literal values); `similar` normalizes correctly and is safe. Full explanation: Cheat Sheet, Common pitfalls.

Symptom: benchmarks report ~1 microsecond per call for an operation you know is sub-nanosecond warm. Cause: the format type or projection spec was read from a non-`const` global, so every call pays Julia's dynamic dispatch instead of running the specialized method. This is a benchmarking-methodology bug, not a package performance regression — it has been traced to this exact mistake in real post-mortems. Fix: keep format types and projection specs in `const` bindings, function arguments, or type parameters; put runtime-selected-format code behind a function barrier; verify zero warm-path allocation before trusting a number. Full explanation: Verification Sessions, Benchmarking without measuring the dispatcher; Verification Strategy.

Symptom: the first array call for a given format/operation is much slower than every call after it. Cause: deterministic unary/binary operations build a cached result table on first use for that specialization; you measured the build, not the steady state. Fix: warm the specialization before benchmarking, and benchmark warm calls separately from the first call. Use `table_bytes()` to inspect the cache footprint and `empty_tables!()` to reset it. Full explanation: Cheat Sheet, Performance checklist (pointer); Function Tables and Array Kernels.

Random and Stochastic

Symptom: a stochastic-rounding pipeline gives different results every run, even though you want to compare against a fixed baseline. Cause: stochastic projections (RSA, RSB, RSC) consume random bits from an RNG (or an explicit draw), and without a fixed source that draw changes every run. Fix: supply a seeded `rng` (for example `Xoshiro(1)`), or pass an explicit `R` in `0:(2^N - 1)` for exact, reproducible draws — ideal in tests. Full explanation: Cheat Sheet, Stochastic projections; Reproducible Stochastic Rounding.

Symptom: you passed a seed but two calls in the same pipeline still diverge. Cause: the uniform draw and the stochastic rounding draw must come from the *same* rng stream to stay reproducible together; if you construct a second `Xoshiro` mid-pipeline instead of threading the same `rng` through, the streams desynchronize. Fix: thread one `rng` object through every call in the pipeline that needs to agree. Full explanation: Cheat Sheet, Stochastic projections.

Related help

Cheat Sheet, Glossary, Performance Model, Verification Strategy.

Chapter 49

Cheat Sheet

49.1 Cheat Sheet

A compact reference for the SmallFloats.jl operations you are most likely to write. For semantics and explanation, follow the links out of each section; for implementation detail, see Insights.

Start here

```
using SmallFloats
using Random: Xoshiro

T = Binary8p4se
ρ = RNE_SN

x = T(1.6)
y = T(0.25)
z = Add(T, ρ, x, y)
```

The explicit operation shape is always:

```
OperationName(result_format, projection_spec, operands...; rng, R)
```

For same-format operands under the default projection, ordinary Julia syntax is available:

```
z == x + y
exp(x)
fma(x, y, z)
```

Format names

```
Binary K p P (s|u) (e|f)
       │   │  │     └─ extended (Inf) or finite domain
       │   │  └─────── signed or unsigned
       │   └────────── significand precision, including the implicit bit
       └──────────────  total bitwidth, 3 through 16
```

Examples:

Name	Meaning
`Binary8p4se`	8-bit, precision 4, signed, extended
`Binary8p4sf`	8-bit, precision 4, signed, finite
`Binary6p3ue`	6-bit, precision 3, unsigned, extended
`Binary5p5uf`	5-bit, precision 5, unsigned, finite
`Binary16p6se`	16-bit, precision 6, signed, extended (opt-in export)
`Binary16p1uf`	16-bit, precision 1, unsigned, finite — the MX scale shape

The alias is the *representation* of the parametric format, related by `<:` and **not** by `===`:

```
Binary8p4se <: Binary{8, 4, true, true}    # true
Binary8p4se === Binary{8, 4, true, true}   # FALSE — `Binary` is abstract
format(16, 6, true, true)                  # the alias, from runtime parameters
```

`using SmallFloats` exports the 120 aliases at $K \leq 8$. For the other 384: `using SmallFloats.Formats`, or `SmallFloats.Binary16p6se`, or `format(...)`.

Useful format queries:

```
bitwidth(T)       # K
precision(T)      # P
issigned(T)
isextended(T)
expbias(T)
expbitwidth(T)
trailingsigbits(T)

MaxFiniteOf(T)
MinFiniteOf(T)
MinPositiveOf(T)
MinNormalOf(T)
MaxSubnormalOf(T)
```

Every query above also takes a **value** instead of a type — `bitwidth(x) ≡ bitwidth(typeof(x))` — and folds to the same constant:

```
x = Binary8p4se(1.6)
bitwidth(x)       # 8
MaxFiniteOf(x)    # Binary8p4se(224.0 ≡ 0x7e)
```

Values and code points

```
x = T(1.6)                    # numeric value: project into T
x = Convert(T, ρ, 1.6)       # explicit projection

c = codepoint(x)              # the code point, in the format's storage unit
x = T(c)                      # an Unsigned means validated CODE POINT
x = rawvalue(T, c)            # unchecked code-point construction

d = decode(x)                 # the exact datum, on the format's own carrier
```

'Unsigned' means code point — at every width

`T(0x02)` constructs code point `0x02`; `T(2)` projects the numeric value two. Signedness of the integer type is what distinguishes them, so the rule holds for `UInt8`, `UInt16` and any other `Unsigned`: `Binary16p6se(0x0002)` is code point 2, not the value 2.0. Use `Convert` when intent should be unmistakable.

`codepoint` returns the format's *storage unit* — `UInt8` at K ≤ 8, `UInt16` above — and that is `codeunit_type(T)`. A code-point argument is range-checked against `2^K` whatever its width, so passing a wider `Unsigned` than necessary is lossless and safe.

`convert` is the deliberate exception and is value-preserving: `Binary8p4se(0x02)` is code point 2, while `convert(Binary8p4se, 0x02)` is the value 2.0. Promotion, `similar` and `fill` depend on that.

Random values (uniform [0,1) floor-projected; normal round-to-nearest, tails clamped to MaxFinite; `randn` signed formats only):

```
rand(T); rand(T, dims); rand!(A)      # uniform on [0, 1)
randn(T); randn(T, dims); randn!(A)   # standard normal, always finite
rand(Xoshiro(1), T)                   # explicit rng as usual

rand(rng, T;  projection=RTP_SN)   # scalar forms: land under any ProjSpec
randn(rng, T; projection=RTZ_SF) # (stochastic ρ draws R from same rng)
```

Classification and stepping:

```
isnan(x); isinf(x); isfinite(x); iszero(x)
signbit(x); issubnormal(x)
Class(x)
NextGreaterThan(x); NextLessThan(x)
nextfloat(x); prevfloat(x)
TotalOrder(x, y)
```

Projection specifications

A projection specification is `(rounding mode, saturation mode)`:

```
ρ = ProjSpec(TowardPositive(), SatFinite())
roundingmode(ρ)
saturationmode(ρ)
```

Deterministic projections

Rounding	SatFinite	SatPropagate	SatNone
nearest, ties even	RNE_SF	RNE_SP	RNE_SN
nearest, ties away	RNA_SF	RNA_SP	RNA_SN
toward +∞	RTP_SF	RTP_SP	RTP_SN
toward −∞	RTN_SF	RTN_SP	RTN_SN
toward zero	RTZ_SF	RTZ_SP	RTZ_SN
round to odd	RTO_SF	RTO_SP	RTO_SN

RNE_SN is the package-wide default. Saturation modes mean:

Mode	Out-of-range behavior
SatFinite	clamp everything to the finite range
SatPropagate	preserve representable infinities; clamp other overflow
SatNone	apply the draft's domain-, signedness-, and direction-dependent rows

Stochastic projections

Constructors are grouped by stochastic variant:

Variant	SatFinite	SatPropagate	SatNone
A	RSA_SF(N)	RSA_SP(N)	RSA_SN(N)
B	RSB_SF(N)	RSB_SP(N)	RSB_SN(N)
C	RSC_SF(N)	RSC_SP(N)	RSC_SN(N)

N is the random-bit budget, `1 ≤ N ≤ 60`. Omitting it uses `N = 8`.

```
σ = RSA_SN(8)

Add(T, σ, x, y; rng=Xoshiro(1))  # reproducible stream
Add(T, σ, x, y; R=17)            # exact draw, ideal for tests

isstochastic(σ)                   # true
nrandbits(σ)                      # 8
```

For an N-bit mode, explicit R must be in `0:(2^N - 1)`.

Session defaults

Read with `DefaultX()`, set with `DefaultX!(v)`:

```
DefaultType()            # Binary8p2se     DefaultType!(Binary8p4se)
DefaultReturnType()      # Binary8p2se     DefaultReturnType!(Binary8p3se)
DefaultRoundingMode()    # NearestTiesToEven()
DefaultSaturationMode()  # SatNone()
DefaultProjection()      # RNE_SN
DefaultRNG()             # Xoshiro         DefaultRNG!(Xoshiro(42))
DefaultRbits()           # 8               DefaultRbits!(16)
```

Setting a rounding/saturation component rebuilds `DefaultProjection`; setting `DefaultProjection!` directly updates both components. Always: `DefaultProjection() === ProjSpec(DefaultRoundingMode(), DefaultSaturationMode())`.

The convenience methods (`a + b`, `Exp(x)`, `T(2.1)`) **do** consult `DefaultProjection`, so changing it changes their results globally. They read it through a speculation guard: allocation-free while the default holds its initial value, one dispatch per call after you change it. Explicit forms (`Add(T, ρ, x, y)`) are unaffected by the session default.

Consume a default via the combinators — never by computing on a bare `DefaultX()` read. No dispatch while the default is unchanged, one barrier dispatch after a change; zero-alloc when `f`'s result type doesn't depend on the default (a default-typed result boxes once at escape):

```
with_default_type((T, x) -> T(x), 1.5)          # Binary8p2se(1.5 ≡ 0x41)
with_default_projection((ρ, x, y) -> Add(T, ρ, x, y), x, y)
with_default_returntype(f, args...)              # f(DefaultReturnType(), args...)
```

Scalar operation catalog

```
# explicit result format and projection
Add(T, ρ, x, y)
FMA(T, ρ, x, y, z)
Convert(T, ρ, external_value)

# same-format default-projection convenience
Add(x, y)
FMA(x, y, z)
```

Arity	Operations
Unary	`Abs`, `Negate`, `Sqrt`, `RSqrt`, `Recip`, `Exp`, `Exp2`, `ExpMinusOne`, `Log`, `Log2`, `LogOnePlus`, `Softplus`, `Sin`, `Cos`, `Tan`, `ArcSin`, `ArcCos`, `ArcTan`, `Sinh`, `Cosh`, `Tanh`, `ArcSinh`, `ArcCosh`, `ArcTanh`, `SinPi`, `CosPi`, `TanPi`, `ArcSinPi`, `ArcCosPi`, `ArcTanPi`
Binary	`CopySign`, `Add`, `Subtract`, `Multiply`, `Divide`, `Hypot`, `ArcTan2`, `ArcTan2Pi`, `Maximum`, `Minimum`, `MaximumNumber`, `MinimumNumber`, `MaximumMagnitude`, `MinimumMagnitude`, `MaximumMagnitudeNumber`, `MinimumMagnitudeNumber`, `MinimumFinite`, `MaximumFinite`
Ternary	`FMA`, `FAA`, `Clamp`
Conversion	`Convert`

Common Base spellings under the session projection (initially `RNE_SN`) — the full register is in Julia Compatibility:

```
x + y; x - y; x * y; x / y
-x; abs(x); inv(x); sqrt(x)
exp(x); exp2(x); expm1(x)
log(x); log2(x); log1p(x)
sin(x); cos(x); tan(x)
asin(x); acos(x); atan(x); atan(y, x)
sinh(x); cosh(x); tanh(x)
min(x, y); max(x, y); clamp(x, y, z)
fma(x, y, z); muladd(x, y, z)
```

Arrays

Every registered operation has elementwise array methods:

```
A = T.([1.0, 1.5, 2.0])
B = T.([0.25, 0.5, 0.75])

C = Add(T, ρ, A, B)
E = Exp(T, ρ, A)
F = FMA(T, ρ, A, B, C)

C = vmap(:Add, T, ρ, A, B)
vmap!(C, Val(:Add), T, ρ, A, B)
```

Operands and destination must have matching axes. For table-eligible K ≤ 8 formats, deterministic unary/binary operations use cached result tables and affordable ternary signatures may also use tables. Wider signatures compute directly. Stochastic operations always compute each element and consume one draw per projection.

```
table_bytes()
empty_tables!()
```

Blocks and scaled operations

```
FS = Binary8p1uf
FE = Binary8p4se

b = Block(FS(4.0), FE(1.5), FE(-0.75), FE(2.0), FE(0.5))

blocksize(b)
scaleformat(b)
elemformat(b)

ConvertFromBlock(T, ρ, b)
BlockReduceAdd(T, ρ, b)
BlockReduceMultiply(T, ρ, b)
BlockDotProduct(T, ρ, b, b)
```

Every scalar operation except `Convert` has generated block and scaled forms:

```
BlockAdd(T, ρ, b1, b2, result_scale)
BlockExp(T, ρ, b, result_scale)

ScaledAdd(T, ρ, scale1, x1, scale2, x2)
ScaledExp(T, ρ, scale, x)
```

Quantize against a supplied or automatically selected scale:

```
ConvertToBlock(FS, FE, ρ, values_tuple, scale)
ConvertToBlockMaxAbsFinite(FS, FE, scale_ρ, element_ρ, values_tuple)
```

`BlockVector` stores many equal-shape blocks in a structure-of-arrays layout.

Packed storage

```
v = Binary5p2se.([0.5, 1.0, 1.5, 2.0])
pv = PackedVector(v)

pv[2]
pv[2] = Binary5p2se(0.75)
collect(pv)

out = vmap(:Exp, Binary5p2se, ρ, pv)
```

`PackedVector` stores each code point in exactly `bitwidth(F)` bits. Computation unpacks tiles internally; packed arithmetic is deliberately not in-place.

Conformance and approximations

```
conformance()
conformance_dict()
conformance_report()
draft_revision()

κ, exhaustive = measure_kappa(fn, :Exp, T, (T,), ρ)
register_approx!(:fast_exp, :Exp, T, (T,), ρ, fn; κ)
impl = approx(:fast_exp)
kappa(impl)
kappa_measured(impl)
list_approx()
unregister_approx!(:fast_exp)
```

Approximate implementations are never substituted into the default API.

Common missteps

Misapplication	Correct pattern
Treating an `Unsigned` as a number	`T(Int(c))` for a numeric integer; `T(c::Unsigned)` for a code point — at every width
`Vector{Binary{K,P,Σ,Δ}}(undef, n)`	**Silently boxes every element** — the abstract format is not `isbits`. Use `Vector{Binary8p4se}` or `Vector{format(K,P,Σ,Δ)}`. `similar` normalizes; `Vector{…}(undef, n)` cannot be intercepted
Silently mixing formats	Convert explicitly: `Convert(T, ρ, x)`
Assuming `SatNone` always clamps	Choose `SatFinite` when clamping is required
Expecting IEEE division-by-zero	P3109 semantics here define `x / 0 → NaN`
Expecting negative zero	Every format has one zero; `T(-0.0) === zero(T)`
Expecting `sort` to put NaN last	The draft's total order puts the single NaN **first**, below −Inf (§4.12.1) — the opposite end from `Float64`
Passing a `Rational`	Convert explicitly to an exact supported carrier or knowingly to `Float64`
Reproducibility with stochastic rounding	Supply a seeded `rng`, or an explicit `R` in tests
Using `rawvalue` on unchecked input	Prefer validated `T(c::Unsigned)` outside kernels

Symptom-first versions of these, with the error text you would actually see, are in Troubleshooting.

Performance checklist

- Keep format types and projection specs in `const` bindings, function arguments, or type parameters.
- Put code using runtime-selected formats behind a function barrier.
- Expect the first deterministic array call for a specialization to build a table; benchmark warm calls separately.
- Use `PackedVector` when storage bandwidth matters more than direct byte access.
- Use `table_bytes()` to inspect cache footprint and `empty_tables!()` to reset it.
- Do not replace explicit projection semantics with `@fastmath`.

The reasoning behind each is in the Performance Model; the measurement rules are in Verification Strategy.

For worked applications, continue to the Examples Index. For correctness and benchmark methodology, continue to Architecture.

Chapter 50

Glossary

50.1 Glossary

Alphabetical, one or two lines per term, with links to the page where the term is defined or used in full.

Related help

Cheat Sheet, Troubleshooting, Architecture and Invariants.

Term	Definition	See
Carrier	The concrete numeric type an exact intermediate or datum is held on during computation — `Float64` at rung 1, `Float128` at rung 2, or an exact dyadic carrier at rung 3, chosen by the format's exponent bias, never by the caller.	Architecture and Invariants; Projection Engine
Code point	The 2^K-valued integer identity of a value within its format — what a `Binary` value actually stores, in bijection with the format's datum set.	Core Model
Code unit	The storage word a format's code points are packed into — `UInt8` at `K ≤ 8`, `UInt16` above — returned by `codeunit_type(T)` and by `codepoint(x)`.	Cheat Sheet, Values, Code Points, and Conversion
Datum	The exact mathematical value a code point denotes — a member of the closed extended reals (finite dyadic values, ±Inf, NaN).	Core Model
Datum carrier	The specific carrier type that is exact for a given format's datums, named by `datumcarrier(T)`; not to be confused with the promotion carrier used for mixed-type Base arithmetic.	Projection Engine
Dyadic	An exact rational-like carrier, `S · 2^Q` with an `Int128` significand and `Int64` exponent, `isbits`, used at rung 3 where even `Float128` cannot hold a format's range or precision; converts to and from Julia's `Rational` exactly.	Oracle and Rigor Classes; Standard and Design Notes
κ (kappa)	The maximum code-point distance, measured along the total order, between an approximate implementation's result and the defined result — measured by exhaustive enumeration at registration time, never declared unchecked.	Oracle and Rigor Classes; Defined, Stochastic, and Approximate Results
Niven peel	The termination argument for π-scaled trigonometric operations: a finite set of exactly-representable cases split off ("peeled") before interval enclosure begins, with Niven's theorem proving the peel set is complete.	Oracle and Rigor Classes
Omega-operation (ω)	The auxiliary, mathematically exact operation (`ωeval`) that a defined operation's result is the projection of — the thing that is computed *before* rounding and saturation ever see it.	Oracle and Rigor Classes
Order key	An integer, sign-magnitude-folded encoding of a value used for comparisons and sorting — monotone with the total order, NaN mapped to key `0` (below every datum, per §4.12.1), enabling O(n) counting sort.	Projection Engine
Projection	The single write path that produces every code point in the package: exact result → RoundToPrecision → Saturate → Encode. There is no second rounding routine anywhere in the package.	Projection Engine
Projection specification (ProjSpec)	The pair (`rounding mode, saturation mode`) that parameterizes a projection, constructed as `ProjSpec(rounding, saturation)` or via a named constant like `RNE_SN`.	Cheat Sheet, Projection Specifications
Promotion carrier	The ordinary Julia float type (`Float64` at rung 1, `BigFloat` at rung 2/3) that a `Binary` value promotes to when combined with an ordinary number — named by `promotecarrier(T)`; distinct from the datum carrier used internally.	Standard and Design Notes; Julia Compatibility Register
Result kind	One of the five categories `ωeval` returns a computed result as (for example exact-`Float64`, exact-`Dyadic`, or an enclosure needing further work), which `apply_op` dispatches on to finish the projection.	Oracle and Rigor Classes
Rigor class (R/E)	The two grades of justification for a non-`Float64` computed result: Class R (unconditional, provable without any accuracy assumption) and Class E (envelope-conditional, a faithful-but-not-correctly-rounded estimate validated by a measured slack).	Architecture and Invariants; Oracle and Rigor Classes
Rounding mode	One of the six deterministic modes (`NearestTiesToEven`, `NearestTiesToAway`, `TowardPositive`, `TowardNegative`, `TowardZero`, `ToOdd`) or three stochastic variants (`RSA`, `RSB`, `RSC`) that make up half of a projection specification.	Cheat Sheet, Projection Specifications
Rung	One of three carrier tiers a format is assigned to by its exponent bias: rung 1 (`Float64`, 432 formats), rung 2 (`Float128`, formats whose range needs more exponent room), rung 3 (exact dyadic carrier, formats whose range or precision breaks `Float128`).	Architecture and Invariants
Saturation mode	One of three policies for out-of-range projection results: `SatFinite` (clamp to the finite range), `SatPropagate` (preserve representable infinities, clamp other overflow), `SatNone` (apply the draft's domain-, signedness-, and direction-dependent rows).	Cheat Sheet, Projection Specifications
Saturation rows	The draft's twenty-row table mapping a rounded value's range classification to its final disposition (as-is, MaxFinite, MinFinite, ±Inf, NaN), consulted by the Saturate stage of projection.	Projection Engine
Span filter	An integer pre-pass over lane exponents in a block reduction that proves a narrower carrier already covers every value the reduction can produce, licensing a faster accumulation path without loss of exactness.	Exact Block Reductions
Sticky (symbolic)	A `sticky ∈ {-1, 0, +1}` argument to the projection engine meaning "the true value is the carrier value plus an infinitesimal of this sign" — how enclosure endpoints and asymptotes (`tanh → 1⁻`) project exactly without inventing a fake carrier value.	Projection Engine
Table gather	The array-kernel strategy of computing a result by indexing a precomputed, cached lookup table (built once through the scalar path) rather than recomputing per element.	Function Tables and Array Kernels
Total order	The draft's total ordering over a format's value set, with the single NaN placed first, below −Inf (§4.12.1); the basis for `TotalOrder`, `isless`, comparisons, and counting sort.	Projection Engine

Chapter 51

Standard and Design Notes

51.1 Standard and Design Notes

An index to the curated `docs/other/` collection: the standard's concept map, the design rationales behind specific implementation choices, and the working papers that planned the K > 8 extension. Each entry carries a status label — read it before treating a document as current.

Designing to the Draft Standard

P3109 Draft Notes — `IEEE_D1_concepts.md`. A concept map of the whole IEEE P3109/D1 draft (July 2026): format-defining parameters, value space, operation machinery, and projection, with every node linked back to the draft's line numbers. Status: normative-description.

The full draft transliteration, `IEEE_D1.md`, is repo-only and is not part of this curated set — it is linked out from the concept map and from the working papers below (see Decision D1). Find it at `IEEE_D1.md` in the repository.

Design Rationales

dyadic_rational.md — `dyadic_rational.md`. The `Dyadic` ↔ `Rational` bridge: how the rung-3 exact carrier converts to and from Julia's `Rational` exactly, and the three laws that conversion must obey (round-trip on every finite dyadic, ±∞ agreement with Base's `1//0`, and rejection only at the true NaN slot). Status: design-rationale.

promoterules.md — `promoterules.md`. Works out why `promote_rule` between two P3109 formats cannot mean "widen to a bigger format" the way `Int`/`Float64` promotion does, and what the promotion carrier is instead. Status: design-rationale, partially superseded by the promotion-carrier section in the Cheat Sheet and Julia Compatibility guide — those pages reflect the shipped `promotecarrier(T)` behavior, and this document should be read as the reasoning behind it rather than as the current API description.

Float32inside.md — `Float32inside.md`. A design study asking whether `Float32` can replace `Float64` as the internal carrier; verdict is no as a universal replacement, yes as a surface type and as a gated compute carrier for specific kernels. Status: exploratory.

Float32more.md — `Float32more.md`. Companion to Float32inside.md: a detailed, empirically validated implementation plan for the Float32/BFloat16 surface, the exactness-gated compute path, and κ-registered Float32 kernels, with measured pass/fail counts per milestone. Status: exploratory.

Development notes

Four documents plan and re-plan the extension of the format grid from $K \leq 8$ (120 at $K \leq 8$, retained as the default exports) to $3 \leq K \leq 16$ (504 formats). Read in this order; each says explicitly where it overrides its predecessor.

extendingK.md — `extendingK.md`. The first pass: what the draft permits, the two axes (storage unit and evaluation carrier/rung) that govern the extension, and format counts by starting rung. Status: exploratory/historical.

doingtheextensions.md — `doingtheextensions.md`. Turns the plan into an architecture — the representation lattice, the carrier dispatch lattice, and generated per-format trait methods — staged so every step compiles and passes independently. Status: exploratory/historical.

implementextensions.md — `implementextensions.md`. The execution plan: a line-by-line audit of the source tree that found ten findings (errors or gaps) against the inherited plan, followed by a ten-stage, single-commit-per-stage implementation schedule. Status: exploratory/historical.

morejulian.md — `morejulian.md`. A separate audit of where the package's Julia interface departs from idiomatic Julia — ranked by user impact rather than by extension stage — covering broken `AbstractFloat` contract methods and deliberate departures alike. Status: exploratory/historical.

Related help

Cheat Sheet, Glossary, Architecture and Invariants.